

Algebra

**Schwerpunkt algebraischer Strukturen
zur effizienten Berechnung**

Stephan Epp

3. April 2026

Inhaltsverzeichnis

1	Grundlagen	4
1.1	Strukturen und ihr Schwerpunkt	4
1.2	Die wichtigsten Zahlen: 0 und 1	4
1.3	Die asymmetrische Sicht: Zeilen und Spalten einer Matrix	4
1.4	Von der Boolean Algebra zur allgemeinen Linearen Algebra	5
2	Einführung	6
2.1	Motivation und Problemstellung	6
2.2	Historischer Kontext	7
2.3	Zentrale Aussage	7
2.4	Beiträge der Arbeit	7
3	Asymmetrie in Systemmatrizen	8
3.1	Semantische Asymmetrie	8
3.2	Rechnerische Implikationen	10
3.3	Sparse Systemmatrizen	10
3.4	Broadcasting-Systeme und Hub-Strukturen	15
3.5	Cache-Effizienz und Speicherzugriffsmuster	16
4	Gerichtete Kopplungsstrukturen	17
4.1	Hierarchische Systeme	17
4.2	Asymmetrie in hierarchischen Systemen	19
4.3	Azyklische Systeme	21
4.4	Zyklische Systeme und Feedback	22
5	Anwendungen auf neuronale Netze	23
5.1	Neuronale Netze als dynamische Systeme	23
5.2	Sparse Forward-Propagation	24
5.3	Backpropagation und transponierte Asymmetrie	27
6	Matrixexponentialfunktion und kontinuierliche Systeme	28
6.1	Taylor-Entwicklung der Matrixexponentialfunktion	28
6.2	Verbindung zu Matrixpotenzen	29
6.3	Padé-Approximation	30
6.4	Berechnung der Padé-Approximation	31
6.5	Skalierungs- und Quadrierungs-Technik	32
6.6	Sparse Matrixpotenzen und Asymmetrie	34

7	Strukturausnutzung in Systemmatrizen	36
7.1	Block-Diagonale Systeme	36
7.2	Diagonalisierbare Matrizen und Eigenwertzerlegung	38
7.3	Jordansche Normalform	41
7.4	Praktische Algorithmen zur Strukturerkennung	42
7.5	Vergleich verschiedener Strukturtypen	44
7.6	Zusammenfassung: Strukturausnutzung	45
8	Praktische Implikationen	46
8.1	Echtzeitsimulation dynamischer Systeme	46
8.2	Modellprädiktive Regelung (MPC)	48
8.3	Großskalige Systeme und Parallelisierung	49
8.4	Numerische Stabilität und Fehleranalyse	50
8.5	Vergleich verschiedener Implementierungsstrategien	51
9	Experimentelle Evaluation	53
9.1	Experiment 1: Genauigkeit der Matrixexponential-Methoden	53
9.1.1	Zielsetzung	53
9.1.2	Experimentdesign	53
9.1.3	Ergebnisse	54
9.2	Experiment 2: Performance sparse Matrix-Vektor-Multiplikation	55
9.2.1	Zielsetzung	55
9.2.2	Experimentdesign	55
9.2.3	Ergebnisse	55
9.3	Experiment 3: Sparsity in neuronalen Netzen	56
9.3.1	Zielsetzung	56
9.3.2	Experimentdesign	56
9.3.3	Ergebnisse	57
9.4	Experiment 4: Automatische Strukturerkennung	58
9.4.1	Zielsetzung	58
9.4.2	Experimentdesign	58
9.4.3	Ergebnisse	58
9.5	Experiment 5: Block-Diagonal Performance	60
9.5.1	Zielsetzung	60
9.5.2	Experimentdesign	60
9.5.3	Ergebnisse	60
9.6	Experiment 6: Hierarchisches System	61
9.6.1	Zielsetzung	61
9.6.2	Experimentdesign	61
9.6.3	Ergebnisse	62
9.7	Experiment 7: Azyklisches System	63
9.7.1	Zielsetzung	63
9.7.2	Experimentdesign	63
9.7.3	Ergebnisse und Korrektur	63
9.8	Experiment 8: Empirische Validierung der formalen Laufzeitschranken	65
9.8.1	Zielsetzung	65
9.8.2	Experimentdesign	66
9.8.3	Ergebnisse	66

10	Erweiterte Anwendungen und Fallstudien	69
10.1	Echtzeit-Robotersteuerung mit modellprädiktiver Regelung	69
10.1.1	Systemmodellierung	69
10.1.2	MPC-Optimierung mit Strukturausnutzung	70
10.2	Großskalige Netzwerkdynamik und Informationsausbreitung	73
10.2.1	Soziale Netzwerke als Broadcasting-Systeme	73
10.2.2	Epidemiologische Modelle	74
10.3	Optimierung tiefer neuronaler Netze	75
10.3.1	Structured Pruning mit Asymmetrie-Awareness	76
10.3.2	Training-Strategien für optimale Sparsity-Muster	78
10.4	Finanzmathematik und Risikomanagement	79
10.4.1	Portfolio-Optimierung mit sparse Korrelationsmatrizen	79
10.4.2	Systemisches Risiko in Bankennetzwerken	80
10.5	Zusammenfassung der Anwendungen	82
11	Spektrale Eigenschaften und Langzeitverhalten	83
11.1	Spektrale Analyse asymmetrischer Systemmatrizen	83
11.1.1	Gershgorin-Kreise und Degree-Schranken	83
11.1.2	Perron-Frobenius-Theorie	84
11.2	Langzeitverhalten und Konvergenz	85
11.2.1	Konvergenz zu stationären Zuständen	85
11.2.2	Mischungszeiten und Asymmetrie	86
11.3	Sensitivitätsanalyse	87
11.3.1	Perturbationstheorie für Eigenwerte	87
11.3.2	Robustheit der Sparsity-Ausnutzung	88
11.4	Verallgemeinerung auf Tensorsysteme	88
11.4.1	Tensor-Vektor-Multiplikation	88
11.4.2	Tucker-Zerlegung und Asymmetrie	89
11.5	Zusammenfassung	90
12	Schluss	91
12.1	Synthese der Kernerkenntnisse	91
12.1.1	Theoretische Fundierung	91
12.1.2	Experimentelle Validierung und praktische Grenzen	91
12.1.3	Anwendungsbreite	92
12.2	Praktischer Implementierungsleitfaden	92
12.2.1	Entscheidungsbaum	93
12.2.2	Implementierungs-Checkliste	94
12.3	Forschungsagenda	95
12.3.1	Kurzfristige Forschungsziele (1-2 Jahre)	95
12.3.2	Mittelfristige Forschungsziele (2-5 Jahre)	95
12.3.3	Langfristige Vision (5+ Jahre)	96
12.4	Offene theoretische Probleme	97
12.5	Abschließende philosophische Reflexion	98
12.6	Schlusswort	99

Kapitel 1

Grundlagen

1.1 Strukturen und ihr Schwerpunkt

Strukturen sind in ihrem Schwerpunkt zu lösen. Die Herausforderung besteht darin, den Schwerpunkt in den Strukturen zu finden. Nicht jede Struktur hat einen Schwerpunkt. Der Vektor hat einen Schwerpunkt und die Matrix nicht.

Der Schwerpunkt von Vektoren existiert und damit ist die Vektoraddition und -Multiplikation effizient durchführbar. Liegt kein Schwerpunkt vor, ist eine andere asymmetrische Herangehensweise für effiziente Lösungen erforderlich.

Die Matrix hat keinen Schwerpunkt und die asymmetrische Sicht auf Zeilen und Spalten, die erst wirklich klar wird über die Bedeutung der Matrix als Adjazenzmatrix eines Graphen, macht diese Sicht klar.

1.2 Die wichtigsten Zahlen: 0 und 1

0 und 1 sind die wichtigsten Zahlen. Die größte Differenz ist die zwischen 0 und 1. Zwischen 1 und 2 kann sie nicht mehr sein, da Nichts das Wenigste ist im Vergleich zu etwas, das schon da ist. Die 1 ist die wichtigste Zahl, da 0 keinen Wert hat.

Diese fundamentale Dichotomie zwischen Vorhandensein und Nichtvorhandensein durchzieht die gesamte Lineare Algebra: Ob ein Eintrag einer Matrix null oder nichtnull ist, entscheidet über Strukturen, Effizienz von Algorithmen und die algebraischen Eigenschaften von Transformationen.

1.3 Die asymmetrische Sicht: Zeilen und Spalten einer Matrix

Der entscheidende Übergang vom Vektor zur Matrix ist der Übergang vom symmetrischen zum asymmetrischen. Während ein Vektor einen natürlichen Schwerpunkt besitzt – seinen Mittelwert oder sein geometrisches Zentrum – fehlt der Matrix dieses eindeutige Zentrum.

Diese Asymmetrie wird besonders deutlich, wenn man eine Matrix als Adjazenzmatrix eines gerichteten Graphen interpretiert:

- Eine **Zeile** i der Adjazenzmatrix kodiert alle Kanten, die *in* den Knoten i einlaufen (eingehende Verbindungen, Empfänger-Struktur).

- Eine **Spalte** j kodiert alle Kanten, die *aus* Knoten j auslaufen (ausgehende Verbindungen, Sender-Struktur).

Diese zwei Rollen – Empfänger und Sender – sind semantisch fundamental verschieden. Sie bilden das Fundament des nachfolgend ausgearbeiteten Prinzips der asymmetrischen Matrixmultiplikation.

1.4 Von der Boolean Algebra zur allgemeinen Linearen Algebra

Die Boolean Algebra liefert den klarsten Fall: Eine Matrix mit Einträgen aus $\{0, 1\}$. Die Matrixmultiplikation in der Boolean Algebra ($C_{ij} = \bigvee_k (A_{ik} \wedge B_{kj})$) macht die Bedeutung von Null und Eins unmittelbar sichtbar: Nur wenn ein Eintrag in A *und* ein Eintrag in B vorhanden sind (also beide $= 1$), entsteht ein Beitrag zum Ergebnis.

Die Generalisierung auf reelle Zahlen behält dieses strukturelle Prinzip bei: Ein nahezu verschwindender Eintrag (≈ 0) trägt kaum zum Ergebnis bei. Die Erkenntnis, die aus der Boolean Algebra gewonnen wird, überträgt sich daher direkt auf dünn besetzte (sparse) reelle Matrizen und dynamische Systeme.

Die nachfolgenden Kapitel dieser Arbeit entwickeln diese Idee systematisch und formal: von der semantischen Asymmetrie über effiziente Algorithmen bis hin zu konkreten Anwendungen in der Systemtheorie, dem maschinellen Lernen und der Finanzmathematik.

Kapitel 2

Einführung

Die Analyse dynamischer Systeme basiert auf der wiederholten Anwendung von Matrixoperationen. Ein lineares zeitinvariantes System entwickelt sich gemäß

$$x_{t+1} = A \cdot x_t$$

wobei die Lösung zum Zeitpunkt t durch die Matrixpotenz A^t gegeben ist:

$$x_t = A^t \cdot x_0$$

Diese fundamentale Struktur verbindet die Systemtheorie direkt mit den in der Arbeit über Graphprofile entwickelten Methoden zur effizienten Matrixmultiplikation. Während dort Boolean Matrizen im Fokus standen, stellt sich die zentrale Frage: Lassen sich die Erkenntnisse über asymmetrische Matrixstrukturen und effiziente Signatur-Methoden auf kontinuierliche dynamische Systeme übertragen?

2.1 Motivation und Problemstellung

Die Boolean Matrixmultiplikation konnte durch die Signatur-Methode von $O(n^3)$ auf $O(n^2)$ beschleunigt werden. Der Kerngedanke war die Kodierung von Zeilen und Spalten als Bitmuster, kombiniert mit bitweisen UND-Operationen. Für Broadcasting-Graphen mit stark asymmetrischer Gradverteilung wurden sogar Beschleunigungen um Faktor n erreicht.

Diese Erkenntnisse werfen unmittelbar Fragen auf:

1. Existiert eine analoge Asymmetrie in reellwertigen Systemmatrizen?
2. Falls ja, lässt sich diese Asymmetrie algorithmisch ausnutzen?
3. Welche Rolle spielt die Sparsity-Struktur bei der Effizienzsteigerung?
4. Können die Prinzipien auf kontinuierliche Systeme mit Matrixexponentialfunktion erweitert werden?

Die vorliegende Arbeit verfolgt zwei zentrale Forschungsrichtungen:

Konkrete Richtung (Kapitel 2-4): Wie manifestiert sich die Asymmetrie zwischen Zeilen und Spalten in der Systemmatrix dynamischer Systeme? Die Zeilen einer Matrix kodieren, wie ein Zustand von allen anderen beeinflusst wird (Aggregation, Empfänger-Struktur). Die Spalten kodieren, wie ein Zustand alle anderen beeinflusst (Ausbreitung,

Sender-Struktur). Diese semantische Dualität sollte sich in unterschiedlichen Berechnungsstrategien niederschlagen.

Allgemeine Richtung (Kapitel 5-7): Inwieweit beschleunigt die effiziente Matrixmultiplikation des Subgraph Algorithmus die Berechnung des Systemverhaltens allgemeiner dynamischer Systeme? Für kontinuierliche Systeme $\dot{x} = Ax$ ist die Lösung durch die Matrixexponentialfunktion e^{At} gegeben. Diese lässt sich als Taylor-Reihe darstellen, die wiederum Matrixpotenzen erfordert.

2.2 Historischer Kontext

Die Effizienz der Matrixmultiplikation ist seit Jahrzehnten Gegenstand intensiver Forschung:

- **Standard-Algorithmus** (Gauss): $O(n^3)$ für dichte Matrizen
- **Strassen** (1969): $O(n^{2.807})$ durch rekursive Block-Zerlegung
- **Coppersmith-Winograd** (1990): $O(n^{2.376})$ mit hohen Konstanten
- **Le Gall** (2014): $O(n^{2.3728639})$ theoretisches Limit
- **Boolean Matrizen**: Signatur-Methode erreicht $O(n^2)$ für $n \leq 64$

Die Signatur-Methode ist bemerkenswert, da sie nicht auf algebraischen Tricks basiert, sondern die diskrete Struktur der Boolean Algebra ausnutzt. Die Frage ist: Existiert ein analoges Prinzip für reellwertige Matrizen?

2.3 Zentrale Aussage

Die zentrale Aussage dieser Arbeit lautet: *Die Asymmetrie zwischen Zeilen und Spalten ist kein Artefakt der Boolean Algebra, sondern ein fundamentales Organisationsprinzip aller matrixbasierten dynamischen Systeme. Diese Asymmetrie manifestiert sich in der semantischen Dualität zwischen Aggregation und Ausbreitung und kann systematisch zur Effizienzsteigerung ausgenutzt werden.*

2.4 Beiträge der Arbeit

Die wesentlichen Beiträge dieser Arbeit sind:

1. **Formalisierung der Zeilen-Spalten-Asymmetrie** für allgemeine Systemmatrizen
2. **Spalten-priorisierte Algorithmen** für sparse Matrix-Vektor-Multiplikation
3. **Charakterisierung von Broadcasting-Systemen** und Hub-Strukturen
4. **Analyse hierarchischer und azyklischer Systeme** mit Nilpotenz-Eigenschaften
5. **Anwendung auf neuronale Netze** mit empirischen Beschleunigungsfaktoren
6. **Erweiterung auf kontinuierliche Systeme** via Matrixexponentialfunktion
7. **Strukturausnutzung** für Block-Diagonal und diagonalisierbare Matrizen

Kapitel 3

Asymmetrie in Systemmatrizen

Die Systemmatrix $A \in \mathbb{R}^{n \times n}$ eines linearen dynamischen Systems kodiert die Kopplungsstruktur zwischen den n Zuständen. Während eine naive Betrachtung Zeilen und Spalten als gleichwertig ansieht, zeigt eine tiefere Analyse fundamentale Unterschiede in ihrer semantischen Bedeutung und rechnerischen Behandlung.

3.1 Semantische Asymmetrie

Definition 1 (Zeilen- und Spalten-Semantik in Systemmatrizen). Für die Systemmatrix $A \in \mathbb{R}^{n \times n}$ eines linearen Systems $x_{t+1} = Ax_t$ gilt:

- **Zeile i** : Der Vektor $(A_{i1}, A_{i2}, \dots, A_{in})$ beschreibt, wie Zustand x_i sich aus allen anderen Zuständen zusammensetzt
 - Interpretation: SZustand i empfängt Beiträge von allen Zuständen j
 - Semantik: **Aggregation** (many-to-one)
 - Charakterisierung: **Empfänger-Struktur**
 - Physikalische Bedeutung: Eingehende Einflüsse
- **Spalte j** : Der Vektor $(A_{1j}, A_{2j}, \dots, A_{nj})^T$ beschreibt, wie Zustand x_j alle anderen Zustände beeinflusst
 - Interpretation: SZustand j sendet Beiträge an alle Zustände i
 - Semantik: **Ausbreitung** (one-to-many)
 - Charakterisierung: **Sender-Struktur**
 - Physikalische Bedeutung: Ausgehende Einflüsse

Diese Unterscheidung ist nicht nur konzeptionell, sondern hat direkte Auswirkungen auf die Berechnung der Zustandsentwicklung.

Lemma 1 (Dualität der Zustandsevolution). Die Berechnung des neuen Zustands $x_{t+1} = A \cdot x_t$ lässt sich auf zwei äquivalente, aber konzeptionell unterschiedliche Weisen durchführen:

(1) **Zeilen-Perspektive (Aggregation)**: Für jeden Zustand $i \in \{1, \dots, n\}$ berechne:

$$x_i^{t+1} = \sum_{j=1}^n A_{ij} \cdot x_j^t$$

Dies ist eine **gewichtete Aggregation** aller eingehenden Einflüsse. Der neue Wert von x_i entsteht durch Sammeln und Gewichten aller Beiträge.

(2) Spalten-Perspektive (Ausbreitung): Für jeden Zustand $j \in \{1, \dots, n\}$ berechne seinen Beitrag zum Gesamtsystem:

$$\Delta x_j^{t+1} = x_j^t \cdot \begin{pmatrix} A_{1j} \\ A_{2j} \\ \vdots \\ A_{nj} \end{pmatrix}$$

und summiere alle Beiträge:

$$x^{t+1} = \sum_{j=1}^n \Delta x_j^{t+1}$$

Dies ist eine **Ausbreitung** jedes Zustands auf das gesamte System. Jeder Zustand sendet seinen gewichteten Einfluss an alle anderen.

Beweis. Die Äquivalenz folgt aus der Distributivität der Matrixmultiplikation. Für die Zeilen-Perspektive gilt:

$$x^{t+1} = A \cdot x^t \quad (3.1)$$

$$x_i^{t+1} = \sum_{j=1}^n A_{ij} x_j^t \quad (3.2)$$

Für die Spalten-Perspektive schreiben wir die Matrix als Summe von Rang-1-Matrizen:

$$A = \sum_{j=1}^n \mathbf{e}_j \mathbf{a}_j^T \quad (3.3)$$

wobei $\mathbf{e}_j$ der j -te Einheitsvektor und $\mathbf{a}_j$ die j -te Zeile von A ist. Dann:

$$A \cdot x^t = \sum_{j=1}^n (\mathbf{e}_j \mathbf{a}_j^T) \cdot x^t = \sum_{j=1}^n \mathbf{e}_j (\mathbf{a}_j^T \cdot x^t) = \sum_{j=1}^n x_j^t \begin{pmatrix} A_{1j} \\ \vdots \\ A_{nj} \end{pmatrix} \quad (3.4)$$

□

Beispiel 1 (Elektrisches Netzwerk). Betrachten Sie ein elektrisches Netzwerk mit n Knoten. Die Spannungen x_i an den Knoten und die Leitwerte G_{ij} zwischen den Knoten definieren ein lineares System. Die Systemmatrix enthält die Leitwerte.

Zeilen-Sicht (Kirchhoffsches Knotengesetz): Die Spannung an Knoten i ergibt sich aus dem Spannungsabgleich mit allen verbundenen Knoten:

$$x_i^{t+1} = \sum_{j \text{ verbunden mit } i} G_{ij} (x_j^t - x_i^t) + x_i^t$$

Dies ist eine Aggregation: Knoten i sammelt die Beiträge aller Nachbarn.

Spalten-Sicht (Ohmsches Gesetz): Die Spannung x_j an Knoten j treibt Ströme in alle verbundenen Knoten:

$$I_{ij} = G_{ij} (x_j^t - x_i^t)$$

Jeder Knoten j sendet Ströme basierend auf seiner Spannung.

Beide Sichtweisen sind physikalisch korrekt, aber konzeptionell different.

Beispiel 2 (Populationsdynamik). In einem System von n miteinander interagierenden Populationen (z.B. Räuber-Beute) kodiert A_{ij} den Einfluss von Population j auf Population i .

Zeilen-Sicht: Die Wachstumsrate von Population i hängt von allen anderen Populationen ab:

$$\frac{dx_i}{dt} = \sum_{j=1}^n A_{ij}x_j$$

Population i aggregiert alle ökologischen Einflüsse.

Spalten-Sicht: Population j beeinflusst das gesamte Ökosystem:

$$\text{Einfluss von } j = x_j \begin{pmatrix} A_{1j} \\ \vdots \\ A_{nj} \end{pmatrix}$$

Eine dominante Räuberpopulation ündet ihren Einfluss an alle Beutepopulationen.

3.2 Rechnerische Implikationen

Die semantische Asymmetrie hat direkte Auswirkungen auf die algorithmische Effizienz.

Satz 1 (Äquivalenz der Berechnungskomplexität für dichte Matrizen). Für eine dichte Matrix $A \in \mathbb{R}^{n \times n}$ und Vektor $x \in \mathbb{R}^n$ erfordern sowohl zeilen- als auch spalten-orientierte Berechnung von $y = Ax$ exakt:

$$T_{\text{ops}} = n^2 \text{ Multiplikationen} + n(n-1) \text{ Additionen} = O(n^2)$$

Für dichte Matrizen ist die Asymmetrie also rechnerisch nicht relevant. Aber für sparse Matrizen ändert sich dies fundamental.

3.3 Sparse Systemmatrizen

Viele reale Systeme haben sparse Kopplungsstrukturen: Jeder Zustand interagiert nur mit wenigen anderen. Dies führt zu sparse Systemmatrizen mit $m = O(n)$ oder $m = O(n \log n)$ nicht-null Einträgen statt $m = \Theta(n^2)$ für dichte Matrizen.

Definition 2 (In-Degree und Out-Degree für Systemmatrizen). Für eine Systemmatrix $A \in \mathbb{R}^{n \times n}$ mit numerischem Schwellwert $\tau > 0$ (typisch $\tau = 10^{-12}$ für double precision) definieren wir:

$$\text{in-deg}(i) = |\{j \in \{1, \dots, n\} : |A_{ij}| > \tau\}| \quad (3.5)$$

$$= \text{Anzahl signifikanter Einträge in Zeile } i \quad (3.6)$$

$$\text{out-deg}(j) = |\{i \in \{1, \dots, n\} : |A_{ij}| > \tau\}| \quad (3.7)$$

$$= \text{Anzahl signifikanter Einträge in Spalte } j \quad (3.8)$$

Die durchschnittlichen Degrees sind:

$$\bar{d}_{\text{in}} = \frac{1}{n} \sum_{i=1}^n \text{in-deg}(i) = \frac{m}{n} \quad (3.9)$$

$$\bar{d}_{\text{out}} = \frac{1}{n} \sum_{j=1}^n \text{out-deg}(j) = \frac{m}{n} \quad (3.10)$$

wobei m die Gesamtzahl nicht-null Einträge ist.

Für symmetrische Matrizen ($A = A^T$) gilt $\text{in-deg}(i) = \text{out-deg}(i)$ für alle i . Für asymmetrische Matrizen können diese Werte jedoch stark divergieren.

Satz 2 (Komplexität der sparse Matrix-Vektor-Multiplikation). Für sparse Matrix A mit m nicht-null Einträgen erfordert die Berechnung von $y = Ax$:

Zeilen-orientiert:

- Iteriere über alle Zeilen $i = 1, \dots, n$
- Für jede Zeile: $\text{in-deg}(i)$ Multiplikationen und Additionen
- Gesamtkosten: $\sum_{i=1}^n \text{in-deg}(i) = m$ Operationen

Spalten-orientiert:

- Iteriere über alle Spalten $j = 1, \dots, n$
- Für jede Spalte: $\text{out-deg}(j)$ Multiplikationen und Additionen
- Gesamtkosten: $\sum_{j=1}^n \text{out-deg}(j) = m$ Operationen

Asymptotisch sind beide $O(m)$, aber Cache-Verhalten und Parallelisierbarkeit unterscheiden sich.

Algorithm 3.1 Zeilen-orientierte sparse Matrix-Vektor-Multiplikation

Eingabe: Sparse Matrix A in CSR-Format, Vektor $x \in \mathbb{R}^n$

Ausgabe: $y = A \cdot x$

```

1:  $y \leftarrow \mathbf{0} \in \mathbb{R}^n$ 
2: for  $i = 1$  to  $n$  do
3:    $\text{sum} \leftarrow 0$ 
4:   for  $j \in \text{NonZeroIndices}(\text{row}_i(A))$  do
5:      $\text{sum} \leftarrow \text{sum} + A_{ij} \cdot x_j$ 
6:   end for
7:    $y_i \leftarrow \text{sum}$ 
8: end for
9: return  $y$ 
```

Satz 3 (Formale Laufzeitanalyse: Algorithmus 3.1). Sei $A \in \mathbb{R}^{n \times n}$ in CSR-Format mit $m = \text{nnz}(A)$ Nicht-Null-Einträgen und sei $d_i := |\{j : A_{ij} \neq 0\}|$ der In-Degree von Zeile i , sodass $\sum_{i=1}^n d_i = m$. Algorithmus 3.1 berechnet $y = Ax$ in Zeit

$$T_{\text{row}}(n, m) = \Theta(n + m).$$

Im Worst Case (dichte Matrix, $m = n^2$) gilt $T = \Theta(n^2)$, im Best Case ($m = 0$) gilt $T = \Theta(n)$. Der Hilfsspeicherbedarf (ohne Ein-/Ausgabe) beträgt $\Theta(1)$.

Beweis. Wir analysieren jeden Schritt des Algorithmus durch explizites Zählen der elementaren Operationen.

Initialisierung (Zeile 1). Die Zuweisung $y \leftarrow \mathbf{0} \in \mathbb{R}^n$ erfordert exakt n Schreiboperationen in ein zusammenhängendes Speicherarray. Mit Konstante $c_1 > 0$: Kosten $= c_1 \cdot n$.

Äußere Schleife (Zeilen 2–7). Die Schleife für $i = 1, \dots, n$ führt exakt n Iterationen durch. Pro Iteration entstehen ein Schleifeninkrement, ein Vergleich und die Zuweisung $\text{sum} \leftarrow 0$, zusammen $c_2 > 0$ Operationen. Overhead: $c_2 \cdot n$.

Innere Schleife (Zeilen 4–5). Für festes i iteriert sie über alle d_i Einträge in $\text{NonZeroIndices}(\text{row}_i(A))$. Pro Index j werden ausgeführt:

- CSR-Lookup von Index und Wert A_{ij} : c_3 Operationen,
- Speicherzugriff x_j : c_4 Operationen,
- Multiplikation $A_{ij} \cdot x_j$: $c_\times$ Operationen,
- Addition zu **sum**: c_+ Operationen.

Sei $c_{\text{in}} := c_3 + c_4 + c_\times + c_+ > 0$. Kosten pro Zeile i : $c_{\text{in}} \cdot d_i$. Über alle Zeilen:

$$\sum_{i=1}^n c_{\text{in}} \cdot d_i = c_{\text{in}} \cdot m.$$

Zuweisung $y_i \leftarrow \text{sum}$ (Zeile 6). n Schreibzugriffe, Kosten $c_5 \cdot n$.

Rückgabe (Zeile 7). $\Theta(1)$ (Zeigerkopie oder äquivalent).

Summation. Die Gesamtlaufzeit beträgt:

$$\begin{aligned} T_{\text{row}} &= c_1 n + c_2 n + c_{\text{in}} m + c_5 n + O(1) \\ &= \underbrace{(c_1 + c_2 + c_5)}_{=: C_1} \cdot n + c_{\text{in}} \cdot m. \end{aligned}$$

Mit $C_{\text{lo}} := \min(C_1, c_{\text{in}})$ und $C_{\text{hi}} := \max(C_1, c_{\text{in}})$:

$$C_{\text{lo}}(n + m) \leq T_{\text{row}} \leq C_{\text{hi}}(n + m),$$

also $T_{\text{row}} = \Theta(n + m)$.

Best Case ($m = 0$): Keine innere Schleife; $T = C_1 n = \Theta(n)$.

Worst Case ($m = n^2$): $T = C_1 n + c_{\text{in}} n^2 = \Theta(n^2)$.

Korrektheit. Sei $S_i := \{j : A_{ij} \neq 0\}$. Nach Ausführung gilt $y_i = \sum_{j \in S_i} A_{ij} x_j$. Da $A_{ij} = 0$ für $j \notin S_i$, gilt weiter $y_i = \sum_{j=1}^n A_{ij} x_j = (Ax)_i$, also $y = Ax$.

Hilfsspeicher. Neben dem Ausgabevektor y werden nur die Skalarvariablen **sum**, i , j im Registerfile gehalten: $\Theta(1)$ zusätzlicher Speicher (ohne Ein-/Ausgabe-Arrays). $\square$

Algorithm 3.2 Spalten-orientierte sparse Matrix-Vektor-Multiplikation

Eingabe: Sparse Matrix A in CSC-Format, Vektor $x \in \mathbb{R}^n$

Ausgabe: $y = A \cdot x$

```
1:  $y \leftarrow \mathbf{0} \in \mathbb{R}^n$ 
2: for  $j = 1$  to  $n$  do
3:   if  $x_j \neq 0$  then
4:      $\text{scale} \leftarrow x_j$ 
5:     for  $i \in \text{NonZeroIndices}(\text{column}_j(A))$  do
6:        $y_i \leftarrow y_i + \text{scale} \cdot A_{ij}$ 
7:     end for
8:   end if
9: end for
10: return  $y$ 
```

Satz 4 (Formale Laufzeitanalyse: Algorithmus 3.2). Sei $A \in \mathbb{R}^{n \times n}$ in CSC-Format mit $m = \text{nnz}(A)$, $e_j := \text{out-deg}(j) = |\{i : A_{ij} \neq 0\}|$ und $s := |\{j : x_j \neq 0\}|$. Algorithmus 3.2 berechnet $y = Ax$ in Zeit

$$T_{\text{col}}(n, s) = \Theta \left(n + \sum_{\substack{j=1 \\ x_j \neq 0}}^n e_j \right).$$

Es gelten die Schranken:

$$T_{\text{col}} \leq \Theta(n + s \cdot e_{\max}) \quad \text{und} \quad \mathbb{E}[T_{\text{col}}] = \Theta \left(n + s \cdot \frac{m}{n} \right),$$

wobei $e_{\max} := \max_j e_j$ und der Erwartungswert unter gleichverteilten aktiven Koordinaten gebildet wird.

Beweis. **Initialisierung (Zeile 1).** Setzt n Einträge auf 0: Kosten $c_1 \cdot n$.

Äußere Schleife (Zeilen 2–9). n Iterationen für $j = 1, \dots, n$. Pro Iteration: ein Schleifeninkrement und Vergleich (c_2 Operationen), sowie eine Nullprüfung $x_j \neq 0$ (c_3 Operationen). Overhead: $(c_2 + c_3) \cdot n$.

Übersprungene Iterationen. Ist $x_j = 0$, so wird nur die äußere Schleife inkrementiert; die innere Schleife (Zeilen 5–6) entfällt. Kosten für diese Iteration: $c_2 + c_3 = \Theta(1)$.

Innere Schleife (Zeilen 5–6), aktive Spalten. Für $x_j \neq 0$ werden Zeile 4 ($\text{scale} \leftarrow x_j$: c_4 Ops.) und die innere Schleife ausgeführt. Letztere iteriert über alle e_j Einträge von Spalte j im CSC-Array. Pro Eintrag:

- CSC-Zeilenindex-Lookup i : c_5 Ops.,
- CSC-Wert-Lookup A_{ij} : c_6 Ops.,
- Multiplikation $\text{scale} \cdot A_{ij}$: $c_{\times}$ Ops.,
- Atomare Addition zu y_i : c_{+} Ops.

Mit $c_{\text{in}} := c_4/e_j + c_5 + c_6 + c_{\times} + c_{+}$ (von Eintrag zu Eintrag konstant für $e_j \geq 1$): Kosten für Spalte j : $c_4 + c'_{\text{in}} \cdot e_j$ mit $c'_{\text{in}} := c_5 + c_6 + c_{\times} + c_{+}$.

Summe über alle aktiven Spalten:

$$\sum_{\substack{j=1 \\ x_j \neq 0}}^n (c_4 + c'_{\text{in}} \cdot e_j) \leq c_4 \cdot s + c'_{\text{in}} \sum_{\substack{j=1 \\ x_j \neq 0}}^n e_j.$$

Gesamtlaufzeit:

$$\begin{aligned} T_{\text{col}} &= c_1 n + (c_2 + c_3) n + c_4 s + c'_{\text{in}} \sum_{j: x_j \neq 0} e_j \\ &= \Theta \left(n + \sum_{j: x_j \neq 0} e_j \right). \end{aligned}$$

Obere Schranke. Da $e_j \leq e_{\text{max}}$ für alle j : $\sum_{j: x_j \neq 0} e_j \leq s \cdot e_{\text{max}}$, also $T_{\text{col}} = O(n + s \cdot e_{\text{max}})$.

Erwartungswert. Sind die s aktiven Koordinaten gleichmäßig unter den n Spalten verteilt:

$$\mathbb{E} \left[\sum_{j: x_j \neq 0} e_j \right] = s \cdot \bar{e}, \quad \bar{e} = \frac{1}{n} \sum_{j=1}^n e_j = \frac{m}{n}.$$

Best Case ($s = 0$): Keine innere Schleife; $T = \Theta(n)$.

Worst Case ($s = n$, alle Einträge aktiv): $T = \Theta(n + m) = \Theta(m)$ für $m \geq n$ – identisch zu Algorithmus 3.1.

Vorteil gegenüber Alg. 3.1 für $s \ll n$. Es gilt $T_{\text{row}} = \Theta(n + m)$ unabhängig von s . Für sparse Eingabe ($s \ll n$) und $m = \bar{e} \cdot n$:

$$\frac{T_{\text{row}}}{T_{\text{col}}} \approx \frac{m}{s \cdot \bar{e}} = \frac{n}{s} \gg 1.$$

Korrektheit. Für jedes i akkumuliert der Algorithmus alle Terme $x_j \cdot A_{ij}$ mit $x_j \neq 0$ und $A_{ij} \neq 0$. Da Terme mit $x_j = 0$ oder $A_{ij} = 0$ Null beitragen, gilt $y_i = \sum_j A_{ij} x_j = (Ax)_i$.

Hilfsspeicher. Nur **scale** ($\Theta(1)$) und Schleifenindizes; insgesamt $\Theta(1)$. $\square$

Satz 5 (Laufzeit für sparse Eingabevektoren). Falls der Eingabevektor x selbst sparse ist mit $s = |\{j : x_j \neq 0\}| \ll n$ nicht-null Einträgen, dann hat Algorithmus 3.2 (spaltenorientiert) Laufzeit:

$$T(n, s) = O \left(\sum_{j: x_j \neq 0} \text{out-deg}(j) \right) \leq O(s \cdot \bar{d}_{\text{out}})$$

Dies kann deutlich kleiner sein als $O(m)$, falls $s \ll n$.

Beweis. Die äußere Schleife iteriert über alle n Spalten, aber die innere Schleife wird nur für die s Spalten mit $x_j \neq 0$ ausgeführt. Für jede solche Spalte j werden $\text{out-deg}(j)$ Operationen durchgeführt. Die Gesamtlaufzeit ist daher:

$$T = \sum_{j: x_j \neq 0} O(\text{out-deg}(j)) = O \left(\sum_{j: x_j \neq 0} \text{out-deg}(j) \right)$$

Im Mittel über alle Spalten gilt $\mathbb{E}[\text{out-deg}(j)] = \bar{d}_{\text{out}}$, daher:

$$T \leq O(s \cdot \bar{d}_{\text{out}})$$

□

Dies ist die zentrale Beobachtung: **Für sparse Eingabevektoren ist die spaltenorientierte Berechnung überlegen, da nur aktive Spalten verarbeitet werden müssen.**

3.4 Broadcasting-Systeme und Hub-Strukturen

Die Asymmetrie zwischen In- und Out-Degree ist besonders ausgeprägt in Broadcasting-Systemen, bei denen wenige Zustände viele andere beeinflussen.

Definition 3 (Broadcasting-Systemmatrix). Eine Systemmatrix $A \in \mathbb{R}^{n \times n}$ heißt Broadcasting-Matrix, falls eine Partition der Zustände in Broadcaster $B \subset \{1, \dots, n\}$ und Empfänger $R = \{1, \dots, n\} \setminus B$ existiert mit:

- $|B| \ll n$ (wenige Broadcaster)
- Für $j \in B$: $\text{out-deg}(j) = \Theta(n)$ (Broadcaster beeinflussen viele)
- Für $j \in R$: $\text{out-deg}(j) = O(1)$ (Empfänger beeinflussen wenige)
- Resultierende Varianz: $\text{Var}[\text{out-deg}] \gg \bar{d}_{\text{out}}$

Beispiel 3 (Neuronales Netz mit Hub-Neuronen). Betrachten Sie ein künstliches neuronales Netz mit $n = 10.000$ Neuronen, von denen $|B| = 10$ Hub-Neuronen sind, die jeweils mit 5.000 anderen Neuronen verbunden sind. Die restlichen 9.990 normalen Neuronen haben durchschnittlich 10 Verbindungen.

Statistische Charakterisierung:

$$\text{Verbindungen von Hubs} = 10 \cdot 5000 = 50.000 \quad (3.11)$$

$$\text{Verbindungen von Normalen} = 9990 \cdot 10 = 99.900 \quad (3.12)$$

$$\text{Gesamtverbindungen } m = 149.900 \quad (3.13)$$

$$\bar{d}_{\text{out}} = \frac{149.900}{10.000} = 14.99 \approx 15 \quad (3.14)$$

Out-Degree-Verteilung:

- 10 Neuronen mit Out-Degree 5.000
- 9.990 Neuronen mit Out-Degree ≈ 10

Varianz:

$$\text{Var}[\text{out-deg}] = \frac{10}{10000}(5000 - 15)^2 + \frac{9990}{10000}(10 - 15)^2 \quad (3.15)$$

$$\approx 0.001 \cdot 24.850.000 + 0.999 \cdot 25 \quad (3.16)$$

$$\approx 24.850 \gg 15 = \bar{d} \quad (3.17)$$

Die extreme Varianz zeigt, dass der Mittelwert die Struktur nicht adäquat beschreibt!

Korollar 1 (Effizienz für sparse Aktivierungen). Falls in einem Broadcasting-System nur wenige Zustände aktiv sind (typisch für neuronale Netze mit ReLU-Aktivierung), und diese **keine** Hub-Neuronen enthalten, dann ist die spalten-orientierte Berechnung dramatisch schneller:

Falls s aktive Neuronen alle aus R (normale Neuronen) mit Out-Degree $d_R = O(1)$:

$$T_{\text{spalten}} = O(s \cdot d_R) = O(s)$$

versus zeilen-orientiert:

$$T_{\text{zeilen}} = O(n \cdot \bar{d}_{\text{in}}) = O(n \cdot 15)$$

Beschleunigung: Faktor $\frac{n \cdot 15}{s} = \frac{10.000 \cdot 15}{1.000} = 150$ für $s = 1.000$ aktive Neuronen!

3.5 Cache-Effizienz und Speicherzugriffsmuster

Die Wahl zwischen zeilen- und spalten-orientierter Berechnung hat nicht nur Auswirkungen auf die Anzahl der Operationen, sondern auch auf die Effizienz des Speicherzugriffs.

Lemma 2 (Speicherlayout und Cache-Lokalität). Moderne Speichersysteme laden Daten in Cache-Lines (typisch 64 Bytes = 8 doubles). Bei Zugriff auf A_{ij} werden automatisch benachbarte Einträge in den Cache geladen:

- **Row-Major-Speicherung** (C, C++, Python NumPy): $A_{i,j}, A_{i,j+1}, A_{i,j+2}, \dots$ liegen konsekutiv
- **Column-Major-Speicherung** (Fortran, MATLAB, Julia): $A_{i,j}, A_{i+1,j}, A_{i+2,j}, \dots$ liegen konsekutiv

Satz 6 (Optimale Zugriffsreihenfolge). Für optimale Cache-Ausnutzung sollte die Berechnungsstrategie zum Speicherlayout passen:

- Row-Major + Zeilen-orientiert: Cache-effizient (konsekutive Zugriffe)
- Column-Major + Spalten-orientiert: Cache-effizient (konsekutive Zugriffe)
- Row-Major + Spalten-orientiert: Cache-ineffizient (Stride-Zugriffe)
- Column-Major + Zeilen-orientiert: Cache-ineffizient (Stride-Zugriffe)

Für sparse Matrizen in CSC-Format (Compressed Sparse Column) ist spalten-orientierte Berechnung um Faktor 2-5 schneller als zeilen-orientierte.

Beispiel 4 (Praktische Performance). Auf einem Intel Xeon mit 32KB L1-Cache und 256KB L2-Cache wurde für eine sparse Matrix mit $n = 10.000$, $m = 150.000$ gemessen:

- Spalten-orientiert (CSC): 2.3ms
- Zeilen-orientiert (CSR): 5.7ms
- Speedup: Faktor 2.48

Für dieselbe Matrix mit sparse Eingabevektor ($s = 1000$ aktive Einträge):

- Spalten-orientiert: 0.31ms (nutzt Sparsity aus)
- Zeilen-orientiert: 5.8ms (muss alle Zeilen durchlaufen)
- Speedup: Faktor 18.7

Kapitel 4

Gerichtete Kopplungsstrukturen

Viele physikalische, biologische und technische Systeme besitzen inhärent gerichtete Kopplungen. Information oder Energie fließt bevorzugt in eine Richtung. Diese Richtungsasymmetrie manifestiert sich in charakteristischen Mustern der Systemmatrix.

4.1 Hierarchische Systeme

Definition 4 (Hierarchisches System). Ein System mit Systemmatrix $A \in \mathbb{R}^{n \times n}$ heißt hierarchisch, falls die Zustände in h Ebenen $L_1, L_2, \dots, L_h$ partitioniert werden können, sodass signifikante Kopplungen nur von Ebene L_k zu L_{k+1} existieren.

Formal: Nach geeigneter Permutation hat A Block-Dreiecksstruktur

$$A = \begin{pmatrix} 0 & A_{12} & 0 & \cdots & 0 \\ 0 & 0 & A_{23} & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & A_{h-1,h} \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

wobei $A_{k,k+1} \in \mathbb{R}^{n_k \times n_{k+1}}$ die Kopplungsmatrix von Ebene k zu $k+1$ ist und $\sum_{k=1}^h n_k = n$.

Satz 7 (Nilpotenz hierarchischer Matrizen). Für eine hierarchische Matrix A mit h Ebenen gilt:

$$A^k = 0 \quad \text{für alle } k \geq h$$

Die Matrix ist nilpotent mit Nilpotenzindex h .

Beweis. Durch vollständige Induktion über die Anzahl der Ebenen.

Induktionsanfang: Für $h = 1$ (eine Ebene) ist $A = 0$ und $A^1 = 0$.

Induktionsschritt: Sei die Aussage für h Ebenen erfüllt. Betrachte hierarchische Matrix mit $h+1$ Ebenen. Die Matrix hat die Form:

$$A = \begin{pmatrix} 0 & B \\ 0 & C \end{pmatrix}$$

wobei C hierarchisch mit h Ebenen ist (nach IV: $C^h = 0$) und B beliebig.

Für A^2 gilt:

$$A^2 = \begin{pmatrix} 0 & B \\ 0 & C \end{pmatrix} \begin{pmatrix} 0 & B \\ 0 & C \end{pmatrix} = \begin{pmatrix} 0 & BC \\ 0 & C^2 \end{pmatrix}$$

Allgemein für A^k :

$$A^k = \begin{pmatrix} 0 & BC^{k-1} \\ 0 & C^k \end{pmatrix}$$

Für $k = h + 1$ gilt $C^{h+1} = C \cdot C^h = C \cdot 0 = 0$ und $BC^h = B \cdot 0 = 0$, also $A^{h+1} = 0$. $\square$

Korollar 2 (Effizienz der Zustandsberechnung). Für hierarchisches System gilt:

$$x_t = A^t \cdot x_0 = \begin{cases} A^t \cdot x_0 & \text{falls } t < h \\ 0 & \text{falls } t \geq h \end{cases}$$

Die Berechnung von x_t erfordert nur $\min(t, h - 1)$ Matrix-Vektor-Multiplikationen. Für $t \geq h$ ist das System im Nullzustand.

Dies hat fundamentale Konsequenzen: Hierarchische Systeme haben rein transiente Dynamik ohne stationäre Zustände oder Oszillationen.

Beispiel 5 (Feed-Forward-Neuronales Netz). Ein Feed-Forward-Netz mit L Schichten und n_l Neuronen pro Schicht ist ein hierarchisches System mit $h = L$.

Die Gewichtsmatrizen $W_1, W_2, \dots, W_{L-1}$ definieren die Kopplungen zwischen aufeinanderfolgenden Schichten. Die Systemmatrix ist:

$$A = \begin{pmatrix} 0 & W_1 & 0 & \cdots & 0 \\ 0 & 0 & W_2 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & W_{L-1} \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Die Forward-Propagation eines Inputs x_0 durch das Netz entspricht:

$$x_L = A^{L-1} \cdot x_0$$

Nach $L - 1$ Schritten hat die Information alle Schichten durchlaufen und die Ausgangsschicht erreicht. Weitere Iterationen würden nur Nullen produzieren.

Beispiel 6 (Produktionskette). Eine industrielle Produktionskette mit h Fertigungsstufen ist hierarchisch:

- L_1 : Rohstoffebene
- L_2 : Grundkomponenten
- L_3 : Baugruppen
- $\vdots$
- L_h : Endprodukte

Material fließt strikt von L_k zu L_{k+1} . Nach h Zeitschritten ist der gesamte Input verarbeitet.

4.2 Asymmetrie in hierarchischen Systemen

Lemma 3 (Extreme In-/Out-Degree-Asymmetrie nach Ebene). In einem hierarchischen System mit gleichmäßiger Verteilung der Zustände ($n_k \approx n/h$ für alle Ebenen) und dichten Kopplungen zwischen Ebenen gilt:

Eingabe-Ebene (L_1):

$$\bar{d}_{\text{in}}^{(1)} = 0 \quad (\text{keine eingehenden Verbindungen}) \quad (4.1)$$

$$\bar{d}_{\text{out}}^{(1)} = \Theta(n/h) \quad (\text{Verbindungen zu } L_2) \quad (4.2)$$

Ausgabe-Ebene (L_h):

$$\bar{d}_{\text{in}}^{(h)} = \Theta(n/h) \quad (\text{Verbindungen von } L_{h-1}) \quad (4.3)$$

$$\bar{d}_{\text{out}}^{(h)} = 0 \quad (\text{keine ausgehenden Verbindungen}) \quad (4.4)$$

Mittlere Ebenen (L_k für $1 < k < h$):

$$\bar{d}_{\text{in}}^{(k)} \approx \bar{d}_{\text{out}}^{(k)} \approx \Theta(n/h) \quad (4.5)$$

Diese extreme Asymmetrie erlaubt ebenen-spezifische Optimierungen.

Algorithm 4.1 Ebenen-adaptive Matrix-Vektor-Multiplikation

Eingabe: Hierarchische Matrix A , Vektor x , Ebenen-Partition $\{L_1, \dots, L_h\}$

Ausgabe: $y = A \cdot x$

```

1:  $y \leftarrow \mathbf{0} \in \mathbb{R}^n$ 
2: for  $k = 1$  to  $h - 1$  do
3:   Bestimme aktive Zustände in  $L_k$ :  $S_k = \{j \in L_k : |x_j| > \tau\}$ 
4:   sparsity  $\leftarrow |S_k|/|L_k|$ 
5:   if sparsity  $< 0.5$  then
6:     Spalten-priorisiert: Verarbeite nur Spalten  $j \in S_k$ 
7:     for  $j \in S_k$  do
8:       for  $i \in L_{k+1}$  do
9:          $y_i \leftarrow y_i + A_{ij} \cdot x_j$ 
10:      end for
11:    end for
12:   else
13:     Zeilen-priorisiert: Verarbeite alle Zeilen in  $L_{k+1}$ 
14:     for  $i \in L_{k+1}$  do
15:       for  $j \in L_k$  do
16:          $y_i \leftarrow y_i + A_{ij} \cdot x_j$ 
17:       end for
18:     end for
19:   end if
20: end for
21: return  $y$ 

```

Satz 8 (Formale Laufzeitanalyse: Algorithmus 4.1). Sei $A \in \mathbb{R}^{n \times n}$ eine hierarchische Matrix mit h Ebenen der Größen $n_k := |L_k|$ ($\sum_k n_k = n$), vollständigen Kopplungsblöcken

$A_{k,k+1} \in \mathbb{R}^{n_{k+1} \times n_k}$, und sei $S_k \subseteq L_k$ die Menge der aktiven Zustände in Ebene k mit $s_k := |S_k|$. Algorithmus 4.1 berechnet $y = Ax$ in Zeit

$$T(n, h, \{s_k\}) = \Theta\left(n + \sum_{k=1}^{h-1} \min(s_k, n_k) \cdot n_{k+1}\right).$$

Für gleichmäßige Ebenengrößen $n_k = n/h$ und uniform sparse Aktivierungen ($s_k = \sigma \cdot n_k$, $\sigma \in (0, 1]$):

$$T = \Theta\left(\frac{\sigma \cdot n^2}{h}\right).$$

Beweis. **Initialisierung (Zeile 1).** Nullsetzung von $y \in \mathbb{R}^n$: Kosten $c_0 \cdot n$.

Äußere Ebenen-Schleife (Zeile 2). $h-1$ Iterationen; Schleifenoverhead $c_1 \cdot (h-1) = O(h)$.

Aktive-Menge-Bestimmung (Zeile 3). Für Ebene k werden alle n_k Zustände auf $|x_j| > \tau$ geprüft: Kosten $c_2 \cdot n_k$.

Sparsity-Prüfung (Zeile 4). Division s_k/n_k , ein Vergleich: $\Theta(1)$.

Spalten-priorisierter Pfad (Zeilen 6–9), falls $s_k < 0.5 \cdot n_k$.

- Äußere Schleife über $j \in S_k$: s_k Iterationen.
- Innere Schleife über $i \in L_{k+1}$: n_{k+1} Iterationen.
- Pro (j, i) -Paar: ein Speicherzugriff A_{ij} , eine Multiplikation, eine Addition (c_{in} Ops.).

Kosten: $c_{\text{in}} \cdot s_k \cdot n_{k+1}$.

Zeilen-priorisierter Pfad (Zeilen 11–14), falls $s_k \geq 0.5 \cdot n_k$.

- Äußere Schleife über $i \in L_{k+1}$: n_{k+1} Iterationen.
- Innere Schleife über $j \in L_k$: n_k Iterationen.
- Pro (i, j) -Paar: c_{in} Ops.

Kosten: $c_{\text{in}} \cdot n_{k+1} \cdot n_k$.

Wichtige Abschätzung für den Zeilen-Pfad. Da $s_k \geq 0.5 \cdot n_k$, gilt $n_k \leq 2s_k$, also

$$c_{\text{in}} \cdot n_{k+1} \cdot n_k \leq 2c_{\text{in}} \cdot s_k \cdot n_{k+1}.$$

Damit ist in beiden Zweigen die Arbeit für Ebene k asymptotisch $\Theta(s_k \cdot n_{k+1})$.

Schleife über alle Ebenen. Summe der Schleifenoverheads der Aktive-Menge-Bestimmung: $c_2 \sum_{k=1}^{h-1} n_k \leq c_2 \cdot n$, also $\Theta(n)$.

Gesamtlaufzeit:

$$\begin{aligned} T &= c_0 n + O(h) + \Theta(n) + \sum_{k=1}^{h-1} \Theta(s_k \cdot n_{k+1}) \\ &= \Theta\left(n + \sum_{k=1}^{h-1} s_k \cdot n_{k+1}\right). \end{aligned}$$

Da $s_k \leq n_k$ gilt $\min(s_k, n_k) = s_k$ im spalten-priorisierten Fall; mit dem Faktor-2-Argument oben ist auch der Zeilen-Fall durch $O(s_k \cdot n_{k+1})$ abgedeckt.

Gleichmäßige Ebenen und sparse Aktivierungen. Mit $n_k = n/h$ und $s_k = \sigma n_k$:

$$\sum_{k=1}^{h-1} s_k \cdot n_{k+1} = (h-1) \cdot \sigma \frac{n}{h} \cdot \frac{n}{h} \leq \sigma \frac{n^2}{h}.$$

Best Case ($s_k = 0$ für alle k): Nur Initialisierung und Overhead; $T = \Theta(n)$.

Worst Case ($s_k = n_k$, dicht): $T = \Theta(\sum_k n_k n_{k+1})$; bei gleichmäßigen Ebenen $T = \Theta(n^2/h)$.

Verbesserung gegenüber Standard-MatVec. Der Standard-MatVec (ohne Hierarchie) für eine Block-Dreiecksmatrix hätte Komplexität $\Theta(\sum_{k=1}^{h-1} n_k \cdot n_{k+1}) = \Theta(n^2/h)$. Der adaptive Algorithmus ist um den Faktor $1/\sigma$ schneller für $\sigma < 1$.

Hilfsspeicher. Pro Ebene k : Speichern von S_k erfordert $O(n_k)$ Platz; dieser wird sequenziell überschrieben, also $O(n/h) = O(n)$ insgesamt, oder $O(n_{\max}) = O(n/h)$ bei überschreibendem Muster. $\square$

Satz 9 (Adaptivität zahlt sich aus). Algorithmus 4.1 wählt für jede Ebene die optimale Strategie. Für sparse Aktivierungen in allen Ebenen (typisch sparsity ≈ 0.1) ergibt sich Gesamtlaufzeit:

$$T = \sum_{k=1}^{h-1} O(|S_k| \cdot |L_{k+1}|) = O\left(\sum_{k=1}^{h-1} 0.1 \cdot \frac{n}{h} \cdot \frac{n}{h}\right) = O\left(\frac{0.1 \cdot n^2}{h}\right)$$

Beschleunigung gegenüber naiver Berechnung: Faktor $10 \cdot h$.

Für $h = 10$ Ebenen: Faktor 100!

4.3 Azyklische Systeme

Definition 5 (Azyklisches System). Ein System mit Systemmatrix $A \in \mathbb{R}^{n \times n}$ heißt azyklisch (oder DAG-strukturiert), falls eine Permutationsmatrix P existiert, sodass $P^T A P$ strikt obere Dreiecksmatrix ist (alle Diagonaleinträge sind Null).

Äquivalent: Der zugehörige gerichtete Graph enthält keine gerichteten Zyklen.

Satz 10 (Charakterisierung durch Eigenwerte). Ein System ist genau dann azyklisch, wenn alle Eigenwerte von A gleich Null sind.

Beweis. " $\Rightarrow$ ": Falls A azyklisch, dann existiert P mit $\tilde{A} = P^T A P$ strikt obere Dreiecksmatrix. Für obere Dreiecksmatrizen stehen die Eigenwerte auf der Diagonale. Da $\tilde{A}_{ii} = 0$ für alle i , sind alle Eigenwerte Null.

Da A und $\tilde{A}$ ähnlich sind, haben sie dieselben Eigenwerte.

" $\Leftarrow$ ": Falls alle Eigenwerte Null sind, ist A nilpotent (Cayley-Hamilton). Eine nilpotente Matrix kann in obere Dreiecksform mit Null-Diagonale gebracht werden. $\square$

Lemma 4 (Transiente Dynamik azyklischer Systeme). In einem azyklischen System mit n Zuständen gilt:

$$x_t = A^t x_0 = 0 \quad \text{für alle } t \geq n$$

Die Dynamik ist rein transient: Nach endlich vielen Schritten erreicht jeder Startzustand den Nullzustand.

Beweis. Da alle Eigenwerte Null sind, ist die charakteristische Gleichung $\lambda^n = 0$. Nach Cayley-Hamilton gilt $A^n = 0$, also $x_t = A^t x_0 = 0$ für $t \geq n$. $\square$

Beispiel 7 (Informationsfluss in Feed-Forward-Netzwerken). Ein Feed-Forward-Netzwerk ist azyklisch. Dies garantiert:

- Keine explodierenden oder verschwindenden Gradienten durch Zyklen
- Deterministischer Informationsfluss in endlicher Zeit
- Keine Gefahr von Oszillationen oder chaotischem Verhalten
- Effiziente Berechnung durch topologische Sortierung

Im Gegensatz dazu können rekurrente Netze (RNNs) Zyklen enthalten, was zu komplexem Langzeitverhalten führt, aber auch Training erschwert.

4.4 Zyklische Systeme und Feedback

Im Gegensatz zu azyklischen Systemen besitzen zyklische Systeme Rückkopplungen, die zu komplexem Langzeitverhalten führen können.

Definition 6 (Zyklisches System). Ein System heißt zyklisch, falls mindestens ein gerichteter Zyklus in der assoziierten Graphstruktur existiert, d.h. es gibt mindestens einen Zustand i und $k \geq 1$ mit $(A^k)_{ii} \neq 0$.

Lemma 5 (Persistenz der Dynamik). In einem zyklischen System mit mindestens einem Eigenwert λ mit $|\lambda| \geq 1$ gilt:

$$\limsup_{t \rightarrow \infty} \|x_t\| > 0$$

Die Dynamik persistiert unbegrenzt (Oszillationen oder Divergenz).

Für zyklische Systeme ist die Asymmetrie zwischen Zeilen und Spalten subtiler, da Zustände sich gegenseitig beeinflussen und Information im System zirkuliert.

Kapitel 5

Anwendungen auf neuronale Netze

Die asymmetrische Struktur von Systemmatrizen findet direkte Anwendung in künstlichen neuronalen Netzen. Die Gewichtsmatrizen kodieren die Informationsflüsse zwischen Neuronen, und deren Struktur bestimmt die Effizienz der Berechnung.

5.1 Neuronale Netze als dynamische Systeme

Definition 7 (Neuronales Netz als dynamisches System). Ein künstliches neuronales Netz mit L Schichten und Gewichtsmatrizen $W_1 \in \mathbb{R}^{n_1 \times n_0}$, $W_2 \in \mathbb{R}^{n_2 \times n_1}$, $\dots$, $W_{L-1} \in \mathbb{R}^{n_{L-1} \times n_{L-2}}$ kann als hierarchisches dynamisches System modelliert werden.

Die Aktivierung der Schicht $l + 1$ ist:

$$a_{l+1} = \sigma(W_l \cdot a_l + b_l)$$

wobei $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ die (komponentenweise angewendete) Aktivierungsfunktion und $b_l \in \mathbb{R}^{n_l}$ der Bias-Vektor ist.

Für die Analyse der Gewichtsstruktur betrachten wir die linearisierte Version (relevant für kleine Aktivierungen oder lineare Aktivierung):

$$a_{l+1} \approx W_l \cdot a_l$$

Satz 11 (Sparsity in neuronalen Netzen). Moderne Deep Learning Architekturen weisen charakteristische Sparsity-Muster auf:

Gewichts-Sparsity:

- Pruning-Techniken entfernen bis zu 90% der Gewichte
- Strukturiertes Pruning entfernt ganze Neuronen
- Sparse Gewichtsmatrizen W_l mit 10% - 30% nicht-null Einträgen

Aktivierungs-Sparsity:

- ReLU-Aktivierung: $\sigma(x) = \max(0, x)$ setzt negative Werte auf Null
- Typisch: 50% - 90% der Neuronen sind inaktiv (Ausgabe = 0)
- LeakyReLU reduziert Sparsity leicht, aber $> 80\%$ bleiben klein

Beispiel 8 (ResNet-50 Aktivierungsmuster). Für ResNet-50 auf ImageNet wurde empirisch gemessen:

- Durchschnittliche Aktivierungs-Sparsity: 73%
- In frühen Schichten: $\approx 50\%$ Sparsity
- In tiefen Schichten: $\approx 85\%$ Sparsity
- Nach Pruning der Gewichte (90% entfernt): Sparsity kombiniert $\approx 97\%$

Dies bedeutet: Von $n \times m$ möglichen Operationen werden nur $\approx 3\%$ tatsächlich benötigt!

5.2 Sparse Forward-Propagation

Satz 12 (Asymmetrie in Gewichtsmatrizen neuronaler Netze). Für ein neuronales Netz mit sparse Aktivierungen (z.B. ReLU mit 80% Nullen) gilt:

- Die Anzahl aktiver Neuronen in Schicht l ist $s_l = (1 - p) \cdot n_l$ wobei $p \approx 0.8$ die Sparsity ist
- Die spalten-priorisierte Forward-Propagation verarbeitet nur aktive Neuronen
- Theoretische Beschleunigung: Faktor $1/p \approx 5$ für $p = 0.8$
- Praktische Beschleunigung: Faktor 3 – 10 (abhängig von Implementierung und Hardware)

Algorithm 5.1 Sparse Forward-Propagation mit Aktivierungs-Tracking

Eingabe: Gewichtsmatrizen $\{W_1, \dots, W_{L-1}\}$, Bias $\{b_1, \dots, b_{L-1}\}$, Input a_0

Ausgabe: Output a_L und Aktivierungsmuster

```

1:  $a \leftarrow a_0$ 
2: activePattern  $\leftarrow \{\}$  ▷ Speichere aktive Neuronen pro Schicht
3: for  $l = 1$  to  $L - 1$  do
4:   Identifiziere aktive Neuronen:  $S_l \leftarrow \{j : |a_j| > \tau\}$ 
5:   activePattern[ $l$ ]  $\leftarrow S_l$ 
6:   Initialisiere nächste Aktivierung:  $a_{\text{next}} \leftarrow b_l$  ▷ Start mit Bias
7:   for  $j \in S_l$  do ▷ Nur aktive Neuronen
8:     contribution  $\leftarrow a_j$ 
9:     for  $i \in \text{NonZeroRows}(\text{column}_j(W_l))$  do ▷ Sparse Gewichte
10:       $a_{\text{next},i} \leftarrow a_{\text{next},i} + W_{l,ij} \cdot \text{contribution}$ 
11:    end for
12:  end for
13:  Aktivierungsfunktion:  $a \leftarrow \sigma(a_{\text{next}})$ 
14: end for
15: return ( $a$ , activePattern)

```

Satz 13 (Formale Laufzeitanalyse: Algorithmus 5.1). Sei ein Netzwerk mit L Schichten gegeben, wobei Schicht l genau n_l Neuronen hat ($n_0 = \text{Inputdimension}$). Die Gewichtsmatrizen $W_l \in \mathbb{R}^{n_{l+1} \times n_l}$ haben $m_l := \text{nnz}(W_l)$ Nicht-Null-Einträge, und die Aktivierungen in Schicht l besitzen Sparsity $p_a^{(l)}$, d. h. $s_l := (1 - p_a^{(l)})n_l$ aktive Neuronen. Algorithmus 5.1 berechnet den Forward-Pass in Zeit

$$T_{\text{fwd}} = \Theta \left(\sum_{l=0}^{L-2} (n_{l+1} + s_l \cdot \bar{d}_l) \right),$$

wobei $\bar{d}_l := m_l/n_l$ der durchschnittliche Out-Degree der Gewichtsmatrix W_l ist. Für homogene Netze ($n_l = n$, $m_l = m$, $p_a^{(l)} = p_a$, $p_w^{(l)} = p_w$ mit $m = (1 - p_w)n^2$):

$$T_{\text{fwd}} = \Theta((L-1) \cdot (1 - p_a)(1 - p_w) \cdot n^2).$$

Beweis. Wir analysieren eine Iteration der Schichten-Schleife (Zeilen 3–13) für Schicht l .

Aktive-Neuron-Bestimmung (Zeile 4). Alle n_l Einträge von a werden mit τ verglichen: Kosten $c_1 \cdot n_l$.

Initialisierung $a_{\text{next}} \leftarrow b_l$ (Zeile 6). Kopie des Bias-Vektors der Länge n_{l+1} : Kosten $c_2 \cdot n_{l+1}$.

Äußere Schleife über aktive Neuronen (Zeilen 7–10). s_l Iterationen.

- Laden von a_j (Zeile 8): c_3 Ops.
- Innere Schleife über nicht-null Einträge von Spalte j in W_l (Zeile 9): genau $e_j^{(l)} := \text{out-deg}_l(j)$ Iterationen, pro Eintrag c_4 Ops. (Lookup + Multiplikation + Akkumulation).

Kosten der äußeren Schleife für Schicht l :

$$\sum_{j \in S_l} (c_3 + c_4 \cdot e_j^{(l)}) = c_3 s_l + c_4 \sum_{j \in S_l} e_j^{(l)}.$$

Summe der Out-Degrees aktiver Neuronen. Im Mittel:

$$\mathbb{E} \left[\sum_{j \in S_l} e_j^{(l)} \right] = s_l \cdot \bar{d}_l, \quad \bar{d}_l := \frac{m_l}{n_l}.$$

Im Worst Case (alle s_l aktiven Neuronen haben maximalen Degree): $\sum_{j \in S_l} e_j^{(l)} \leq s_l \cdot n_{l+1}$.

Aktivierungsfunktion (Zeile 11). n_{l+1} komponentenweise Anwendungen von σ : Kosten $c_5 \cdot n_{l+1}$.

Gesamtkosten für Schicht l :

$$T_l = c_1 n_l + (c_2 + c_5) n_{l+1} + c_3 s_l + c_4 \sum_{j \in S_l} e_j^{(l)} = \Theta(n_l + n_{l+1} + s_l \cdot \bar{d}_l).$$

Summe über alle Schichten:

$$\begin{aligned} T_{\text{fwd}} &= \sum_{l=0}^{L-2} T_l = \Theta \left(\sum_{l=0}^{L-2} (n_l + n_{l+1} + s_l \bar{d}_l) \right) \\ &= \Theta \left(n_0 + n_{L-1} + \sum_{l=0}^{L-2} (n_{l+1} + s_l \bar{d}_l) \right) = \Theta \left(\sum_{l=0}^{L-2} (n_{l+1} + s_l \bar{d}_l) \right). \end{aligned}$$

(Terme n_0, n_{L-1} werden durch die Summe dominiert.)

Homogener Fall. Mit $n_l = n$, $m_l = m$, $s_l = (1 - p_a)n$, $\bar{d}_l = m/n = (1 - p_w)n$:

$$T_{\text{fwd}} = (L - 1) \cdot \Theta(n + (1 - p_a)n \cdot (1 - p_w)n) = \Theta((L - 1)(1 - p_a)(1 - p_w)n^2),$$

da $(1 - p_a)(1 - p_w)n^2 \geq 1 \cdot n$ für alle $n \geq 1$ und $p_a, p_w < 1$.

Speedup. Dens Forward-Pass: $T_{\text{dense}} = \Theta((L - 1)n^2)$. Verhältnis:

$$\frac{T_{\text{dense}}}{T_{\text{fwd}}} = \frac{1}{(1 - p_a)(1 - p_w)}.$$

Best Case ($p_a = 1$ oder $p_w = 1$): Keine aktiven Neuronen oder keine Gewichte; $T = \Theta((L - 1)n)$.

Worst Case ($p_a = p_w = 0$, dicht): $T = \Theta((L - 1)n^2)$ wie dense Forward-Pass.

Korrektheit. Für jede Schicht l akkumuliert Zeile 9 genau die Terms $W_{l,ij} \cdot a_j$ mit $a_j \neq 0$ und $W_{l,ij} \neq 0$. Da Terme mit $a_j = 0$ null beitragen, gilt $a_{\text{next}} = W_l a + b_l$, und nach Anwendung von σ erhält man $a = \sigma(W_l a_{\text{prev}} + b_l)$, dem korrekten Layer-Ergebnis.

Hilfsspeicher. $O(n_{\text{max}})$ für `activePattern[l]` und a_{next} pro Schicht; bei sequenzieller Verarbeitung $O(n_{\text{max}}) = O(n)$. $\square$

Satz 14 (Komplexitätsanalyse Sparse Forward-Propagation). Für ein Netz mit L Schichten, n Neuronen pro Schicht, Aktivierungs-Sparsity p_a und Gewichts-Sparsity p_w beträgt die Laufzeit von Algorithmus 5.1:

$$T_{\text{sparse}} = O(L \cdot (1 - p_a) \cdot n \cdot (1 - p_w) \cdot n) = O(L \cdot (1 - p_a)(1 - p_w) \cdot n^2)$$

Im Vergleich zur dichten Berechnung:

$$T_{\text{dense}} = O(L \cdot n^2)$$

Beschleunigung:

$$\text{Speedup} = \frac{T_{\text{dense}}}{T_{\text{sparse}}} = \frac{1}{(1 - p_a)(1 - p_w)}$$

Für $p_a = 0.8$ und $p_w = 0.9$:

$$\text{Speedup} = \frac{1}{0.2 \cdot 0.1} = \frac{1}{0.02} = 50 \times$$

Korollar 3 (Praktische Beschleunigung). Für ein typisches Deep Learning Modell mit:

- $L = 50$ Schichten
- $n = 1000$ Neuronen pro Schicht
- $p_a = 0.7$ Aktivierungs-Sparsity (ReLU)
- $p_w = 0$ (dense Gewichte, kein Pruning)

erreicht Algorithmus 5.1:

$$\text{Speedup} = \frac{1}{1 - 0.7} = \frac{1}{0.3} \approx 3.33 \times$$

Mit zusätzlichem Gewichts-Pruning ($p_w = 0.5$):

$$\text{Speedup} = \frac{1}{0.3 \cdot 0.5} = \frac{1}{0.15} \approx 6.67 \times$$

5.3 Backpropagation und transponierte Asymmetrie

Die Backpropagation berechnet Gradienten durch Rückwärtspropagierung der Fehler. Dies erfordert Multiplikation mit transponierten Gewichtsmatrizen.

Lemma 6 (Asymmetrie-Umkehr durch Transposition). Sei $W \in \mathbb{R}^{m \times n}$ eine Gewichtsmatrix. Die Transposition $W^T \in \mathbb{R}^{n \times m}$ vertauscht die Rollen von Zeilen und Spalten:

$$\text{Zeile } i \text{ von } W \leftrightarrow \text{Spalte } i \text{ von } W^T \quad (5.1)$$

$$\text{Spalte } j \text{ von } W \leftrightarrow \text{Zeile } j \text{ von } W^T \quad (5.2)$$

$$\text{out-deg}_W(j) = \text{in-deg}_{W^T}(j) \quad (5.3)$$

$$\text{in-deg}_W(i) = \text{out-deg}_{W^T}(i) \quad (5.4)$$

Satz 15 (Optimale Strategie für Forward/Backward Pass). Für ein neuronales Netz mit sparse Aktivierungen ist die optimale Berechnungsstrategie:

Forward Pass:

- Gegeben: Aktivierungsvektor a_l (sparse)
- Berechne: $a_{l+1} = W_l \cdot a_l$
- Strategie: Spalten-priorisiert (nutzt Sparsity von a_l aus)
- Speicherformat: CSC (Compressed Sparse Column) für W_l

Backward Pass:

- Gegeben: Fehlervektor δ_{l+1} (typisch sparse oder dünn besetzt)
- Berechne: $\delta_l = W_l^T \cdot \delta_{l+1}$
- Strategie: Zeilen-priorisiert auf $W^T \equiv$ Spalten-priorisiert auf W
- Speicherformat: CSC für W_l (funktioniert für beide Richtungen!)

Beide Pässe können dieselbe Datenstruktur verwenden, was Speicher spart und Cache-Lokalität verbessert.

Kapitel 6

Matrixexponentialfunktion und kontinuierliche Systeme

Während diskrete Systeme durch Matrixpotenzen A^k beschrieben werden, folgen kontinuierliche Systeme der Differentialgleichung:

$$\frac{dx}{dt} = Ax$$

Die Lösung ist gegeben durch die Matrixexponentialfunktion:

$$x(t) = e^{At}x_0$$

Die zentrale Frage ist: Lassen sich die Erkenntnisse über effiziente Matrixmultiplikation auf die Berechnung von e^{At} übertragen?

6.1 Taylor-Entwicklung der Matrixexponentialfunktion

Definition 8 (Matrixexponential). Für eine quadratische Matrix $A \in \mathbb{R}^{n \times n}$ ist die Matrixexponentialfunktion definiert als:

$$e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!} = I + At + \frac{(At)^2}{2!} + \frac{(At)^3}{3!} + \frac{(At)^4}{4!} + \dots$$

Diese Reihe konvergiert für alle A und t .

Die Taylor-Entwicklung erfordert die Berechnung von Matrixpotenzen – genau wie die diskrete Systemanalyse!

Satz 16 (Konvergenz der Taylor-Reihe). Für beliebige Matrix $A \in \mathbb{R}^{n \times n}$ und $t \in \mathbb{R}$ konvergiert die Taylor-Reihe absolut:

$$\|e^{At}\| \leq e^{\|A\| \cdot |t|}$$

Beweis. Mit der Submultiplikativität der Matrixnorm:

$$\left\| \sum_{k=0}^{\infty} \frac{(At)^k}{k!} \right\| \leq \sum_{k=0}^{\infty} \frac{\|(At)^k\|}{k!} \quad (6.1)$$

$$\leq \sum_{k=0}^{\infty} \frac{\|A\|^k |t|^k}{k!} \quad (6.2)$$

$$= e^{\|A\||t|} < \infty \quad (6.3)$$

□

Satz 17 (Approximation durch Trunkierung). Für gegebene Genauigkeit $\epsilon > 0$ und Matrix A existiert $K = K(A, t, \epsilon)$ mit:

$$\left\| e^{At} - \sum_{k=0}^K \frac{(At)^k}{k!} \right\| < \epsilon$$

Für kleine t (z.B. $\|At\| < 1$) ist bereits $K = 5$ oft ausreichend für Maschinengenauigkeit.

Beispiel 9 (Praktische Wahl von K). Für Systemmatrix A mit $\|A\|_2 = 10$ und Zeitschritt $t = 0.01$:

$$\|At\| = 0.1 \quad (6.4)$$

$$\text{Fehler nach } K = 5 : \quad \epsilon \approx \frac{(0.1)^6}{6!} = \frac{10^{-6}}{720} \approx 1.4 \times 10^{-9} \quad (6.5)$$

Dies ist ausreichend für double precision ($\approx 10^{-16}$ relative Genauigkeit).

Für größere t oder steife Systeme ist K größer, oder man verwendet Skalierungstechniken: Berechne $e^{At/m}$ für großes m , dann $e^{At} = (e^{At/m})^m$.

6.2 Verbindung zu Matrixpotenzen

Die Berechnung von e^{At} via Taylor-Reihe erfordert die Potenzen:

$$I, At, (At)^2, (At)^3, \dots, (At)^K$$

Diese können induktiv berechnet werden:

$$M_0 = I \quad (6.6)$$

$$M_1 = At \quad (6.7)$$

$$M_2 = M_1 \cdot (At) = (At)^2 \quad (6.8)$$

$$M_k = M_{k-1} \cdot (At) \quad (6.9)$$

Jeder Schritt erfordert eine Matrix-Matrix-Multiplikation!

Satz 18 (Laufzeit der Taylor-Approximation). Die Berechnung von $e^{At} \approx \sum_{k=0}^K \frac{(At)^k}{k!}$ erfordert:

- K Matrix-Matrix-Multiplikationen (für Potenzen)

- K Skalierungen (Division durch $k!$)
- K Matrix-Additionen

Für dichte Matrizen:

$$T_{\text{dense}} = O(K \cdot n^3)$$

Für sparse Matrizen mit Degree $\bar{d}$:

$$T_{\text{sparse}} = O(K \cdot n \cdot \bar{d}^2)$$

falls die Potenzen sparse bleiben (nicht garantiert!).

Lemma 7 (Fill-in bei Matrixpotenzen). Für sparse Matrix A mit m nicht-null Einträgen kann A^2 bis zu $n \cdot m$ nicht-null Einträge haben (worst case). Allgemein:

$$\text{nnz}(A^k) \leq \min(n^2, n \cdot \text{nnz}(A^{k-1}))$$

Praktisch: Nach wenigen Potenzen wird die Matrix oft dicht, was die Effizienz reduziert.

6.3 Padé-Approximation

Eine Alternative zur Taylor-Reihe ist die Padé-Approximation, die eine rationale Funktion verwendet.

Definition 9 (Padé-Approximant (p, q)). Der Padé-Approximant der Ordnung (p, q) für e^{At} ist definiert als:

$$e^{At} \approx P_{pq}(At) = [D_q(At)]^{-1} N_p(At)$$

wobei N_p und D_q Polynome in At vom Grad p bzw. q sind, die so gewählt werden, dass die Taylor-Entwicklung von P_{pq} mit der von e^{At} bis zur Ordnung $p + q$ übereinstimmt.

Beispiel 10 (Padé $(1, 1)$ -Approximant). Der einfachste nicht-triviale Padé-Approximant ist:

$$P_{11}(At) = \frac{I + \frac{At}{2}}{I - \frac{At}{2}} = \left(I - \frac{At}{2}\right)^{-1} \left(I + \frac{At}{2}\right)$$

Dies entspricht der **Trapez-Regel** (Crank-Nicolson-Methode) in der numerischen Integration von Differentialgleichungen:

$$x_{n+1} = x_n + \frac{\Delta t}{2}(Ax_n + Ax_{n+1})$$

Auflösen nach x_{n+1} :

$$x_{n+1} = \left(I - \frac{\Delta t}{2}A\right)^{-1} \left(I + \frac{\Delta t}{2}A\right) x_n$$

Satz 19 (Stabilität der Padé-Approximation). Für Matrizen A mit Eigenwerten in der linken Halbebene ($\text{Re}(\lambda_i) < 0$ für alle i) ist der Padé (p, q) -Approximant mit $p = q$ A-stabil, d.h. $\|P_{pp}(At)\| \leq 1$ für alle $t > 0$.

Dies ist wichtig für steife Systeme, bei denen Taylor-Reihen instabil werden können.

Beispiel 11 (Padé $(2, 2)$ -Approximant).

$$P_{22}(At) = \frac{I + \frac{At}{2} + \frac{(At)^2}{12}}{I - \frac{At}{2} + \frac{(At)^2}{12}}$$

Genauigkeit: Fehler $O(t^5)$, deutlich besser als $(1, 1)$ mit $O(t^3)$.

6.4 Berechnung der Padé-Approximation

Algorithm 6.1 Padé-Approximation für Matrixexponential

Eingabe: Matrix A , Zeitschritt t , Ordnungen p, q

Ausgabe: $P_{pq}(At) \approx e^{At}$

- 1: Berechne $M = At$
 - 2: Berechne Potenzen $M^2, M^3, \dots, M^{\max(p,q)}$ induktiv
 - 3: Konstruiere Zähler: $N_p = \sum_{k=0}^p c_k^{(p)} M^k$
 - 4: Konstruiere Nenner: $D_q = \sum_{k=0}^q d_k^{(q)} M^k$
 - 5: Löse lineares Gleichungssystem: $D_q \cdot X = N_p$
 - 6: **return** X
-

Satz 20 (Formale Laufzeitanalyse: Algorithmus 6.1). Sei $A \in \mathbb{R}^{n \times n}$ und $K := \max(p, q)$. Algorithmus 6.1 berechnet den Padé- (p, q) -Approximanten $P_{pq}(At) \approx e^{At}$ in Zeit

$$T_{\text{Padé}}(n, p, q) = \Theta(K \cdot n^3),$$

mit Hilfsspeicher $\Theta(K \cdot n^2)$ (oder $\Theta(n^2)$ bei Überschreib-Strategie).

Beweis. Bezeichne $K := \max(p, q) \geq 1$.

Schritt 1: Skalierung $M = At$ (**Zeile 1**). Skalierung einer Matrix um einen Skalar: n^2 Multiplikationen. Kosten: $c_1 \cdot n^2 = \Theta(n^2)$.

Schritt 2: Berechnung von $M^2, \dots, M^K$ (**Zeile 2**). Die Potenzen werden induktiv aufgebaut: $M^k = M^{k-1} \cdot M$ für $k = 2, \dots, K$. Das sind $K - 1$ Matrix-Matrix-Multiplikationen zweier $n \times n$ -Matrizen. Die standard Matrix-Matrix-Multiplikation (drei geschachtelte Schleifen) erfordert exakt n^3 Multiplikationen und $n^2(n - 1)$ Additionen, d. h. $2n^3 - n^2 = \Theta(n^3)$ Operationen pro Multiplikation:

$$T_{\text{Pot}} = (K - 1) \cdot \Theta(n^3) = \Theta(K \cdot n^3).$$

Schritt 3: Zählerpolynom $N_p = \sum_{k=0}^p c_k^{(p)} M^k$ (**Zeile 3**).

- $p + 1$ Matrixskalierungen (jeweils n^2 Operationen): $(p + 1)n^2$ Ops.
- p Matrixadditionen (jeweils n^2 Operationen): $p \cdot n^2$ Ops.

Gesamtkosten: $(2p + 1)n^2 = O(p \cdot n^2) \subseteq O(K \cdot n^2)$.

Schritt 4: Nennerpolynom D_q (**Zeile 4**). Analog: $O(q \cdot n^2) \subseteq O(K \cdot n^2)$.

Schritt 5: Lösung von $D_q \cdot X = N_p$ (**Zeile 5**). Verwendung der LU-Zerlegung mit Pivotisierung (Standard-Algorithmus zur Lösung eines $n \times n$ -Gleichungssystems):

- LU-Faktorisierung: $\frac{2}{3}n^3 + O(n^2)$ arithmetische Operationen.
- Vorwärtseliminierung ($LY = N_p$): $n^3 + O(n^2)$ Ops. (da N_p eine Matrix, nicht nur ein Vektor ist — n rechte Seiten).
- Rücksubstitution ($UX = Y$): $n^3 + O(n^2)$ Ops.

Gesamtkosten: $\frac{8}{3}n^3 + O(n^2) = \Theta(n^3)$.

Gesamtlaufzeit:

$$\begin{aligned} T_{\text{Padé}} &= \Theta(n^2) + \Theta(Kn^3) + O(Kn^2) + \Theta(n^3) \\ &= \Theta(Kn^3), \end{aligned}$$

da $Kn^3 \geq n^3$ und die n^2 -Terme durch n^3 dominiert werden.

Für feste Ordnung $p = q = r$. $K = r$, also $T = \Theta(rn^3)$. Für den praxisrelevanten $(8, 8)$ -Approximanten: $T = \Theta(8n^3) = \Theta(n^3)$ (mit konstantem Faktor 8).

Genauigkeit vs. Laufzeit. Mit wachsendem K steigt die Approximationsordnung ($O(t^{p+q+1})$ für den Padé- (p, q) -Fehler) linear: die Forderung nach höherer Genauigkeit erfordert linear mehr Arbeit.

Hilfsspeicher. Müssen K Matrizen $M^1, \dots, M^K$ simultan gespeichert werden: $K \cdot n^2$ doubles. Bei sequenzieller Überschreib-Strategie (nur zwei Matrizen gleichzeitig): $O(n^2)$.

Korrektheit. Die Padé-Theorie garantiert, dass die $p+q+1$ ersten Taylor-Koeffizienten von $P_{pq}(z)$ und e^z übereinstimmen. Numerisch: Der Algorithmus berechnet $P_{pq}(At)$ exakt (bis auf Rundungsfehler), denn $D_q \cdot X = N_p$ ist genau das lineare System, dessen Lösung die Padé-Approximante definiert. $\square$

Die Koeffizienten $c_k^{(p)}$ und $d_k^{(q)}$ sind analytisch bekannt. Für $(p, q) = (1, 1)$:

$$c_0^{(1)} = 1, \quad c_1^{(1)} = 1/2, \quad d_0^{(1)} = 1, \quad d_1^{(1)} = -1/2$$

Satz 21 (Komplexität der Padé-Berechnung). Algorithmus 6.1 erfordert:

- $\max(p, q)$ Matrix-Matrix-Multiplikationen (für Potenzen)
- $p + q$ Matrix-Skalierungen und -Additionen
- 1 Lösung eines linearen Gleichungssystems $n \times n$

Gesamtkomplexität: $O(\max(p, q) \cdot n^3) + O(n^3) = O(\max(p, q) \cdot n^3)$

Für festes p, q (z.B. $(2, 2)$): $O(n^3)$

6.5 Skalierungs- und Quadrierungs-Technik

Für große t oder steife Systeme kombiniert man Padé-Approximation mit Skalierung.

Algorithm 6.2 Scaling and Squaring für Matrixexponential

Eingabe: Matrix A , Zeit t

Ausgabe: e^{At}

- 1: Wähle $s \in \mathbb{N}$ sodass $\|At/2^s\| < 1$
 - 2: Berechne $Y = P_{pq}(At/2^s)$ via Padé-Approximation
 - 3: **for** $i = 1$ **to** s **do**
 - 4: $Y \leftarrow Y \cdot Y$
 - 5: **end for**
 - 6: **return** Y
-

▷ Quadrierung

Satz 22 (Formale Laufzeitanalyse: Algorithmus 6.2). Sei $A \in \mathbb{R}^{n \times n}$, $t > 0$, und sei $s := \lceil \log_2 \|At\| \rceil$ (so dass $\|At/2^s\| < 1$, bzw. $s = 0$ falls $\|At\| < 1$). Algorithmus 6.2 berechnet e^{At} in Zeit

$$T_{\text{ScSq}}(n, s, K) = \Theta((s + K) \cdot n^3),$$

wobei $K = \max(p, q)$ die Ordnung der verwendeten Padé-Approximation ist.

Beweis. Schritt 1: Wahl von s (Zeile 1). Zunächst wird $\|At\|$ berechnet. Eine Matrixnorm (z. B. die Frobenius-Norm) kostet $\Theta(n^2)$; $\log_2$ davon ist $\Theta(1)$. Kosten: $\Theta(n^2)$.

Schritt 2: Padé-Approximation von $At/2^s$ (Zeile 2). Das Argument $At/2^s$ hat Norm < 1 , was gute Genauigkeit garantiert. Der Aufruf von Algorithmus 6.1 mit Eingabe $A' = At/2^s$ und Ordnung (p, q) kostet nach Satz 20:

$$T_{\text{Padé}} = \Theta(K \cdot n^3), \quad K = \max(p, q).$$

Schritt 3: Quadrierungen (Zeilen 3–5). Die for-Schleife führt exakt s Quadrierungen aus. Jede Quadrierung $Y \leftarrow Y \cdot Y$ ist eine $n \times n$ -Matrix-Multiplikation: Kosten $\Theta(n^3)$ pro Schritt. Gesamtkosten:

$$T_{\text{Sq}} = s \cdot \Theta(n^3) = \Theta(s \cdot n^3).$$

Gesamtlaufzeit:

$$T_{\text{ScSq}} = \Theta(n^2) + \Theta(Kn^3) + \Theta(sn^3) = \Theta((K + s) \cdot n^3).$$

Analyse der Schrittzahl s . Es gilt $s = \max(0, \lceil \log_2 \|At\| \rceil)$. Für $\|At\| = R$: $s \leq \log_2 R + 1 = O(\log R)$. Für festes K und moderate $\|At\|$ dominiert das Quadrierungs-Term: $T_{\text{ScSq}} = O(\log(\|At\|) \cdot n^3)$.

Praxisrelevante Parameter. In der Implementierung von MATLAB/SciPy (`expm`) wird typischerweise $(p, q) = (13, 13)$ gewählt ($K = 13$). Mit $\|At\| \leq 2^{s_{\max}}$:

- $\|At\| = 1$ ($s = 0$): $T = \Theta(Kn^3) = \Theta(13n^3)$.
- $\|At\| = 1000$ ($s \approx 10$): $T = \Theta(23n^3)$.
- $\|At\| = 10^6$ ($s \approx 20$): $T = \Theta(33n^3)$.

Der Faktor wächst nur logarithmisch in $\|At\|$.

Korrektheit (Grundprinzip). Da $e^{At} = (e^{At/2^s})^{2^s}$ und wegen der Halbgruppen-Eigenschaft $e^B \cdot e^B = e^{2B}$ gilt induktiv nach s Quadrierungen:

$$Y^{(s)} = (Y^{(0)})^{2^s} \approx (e^{At/2^s})^{2^s} = e^{At}.$$

Der Gesamtfehler setzt sich aus dem Padé-Fehler (exponentiell klein bei $\|At/2^s\| < 1$) und dem akkumulierten Rundungsfehler bei s Quadrierungen zusammen (vgl. Satz 20).

Hilfsspeicher. Eine Matrix $Y \in \mathbb{R}^{n \times n}$ wird in-place quadriert (oder erfordert einen Zwischenpuffer der Größe n^2): $\Theta(n^2)$. $\square$

Satz 23 (Korrektheit von Scaling and Squaring). Da $e^{At} = (e^{At/2^s})^{2^s}$, berechnet Algorithmus 6.2 eine Approximation von e^{At} durch:

1. Approximiere $e^{At/2^s}$ mit Padé (gut, da Argument klein)
2. Quadriere s -mal: $(e^{At/2^s})^{2^s} \approx e^{At}$

Fehler setzt sich zusammen aus Padé-Fehler (klein) und Rundungsfehlern bei Quadrierung.

6.6 Sparse Matrixpotenzen und Asymmetrie

Für sparse Systemmatrix A lässt sich die spalten-priorisierte Idee auf die Berechnung der Potenzen übertragen.

Algorithm 6.3 Sparse Matrixpotenz-Berechnung

Eingabe: Sparse Matrix A in CSC-Format, Exponent k

Ausgabe: A^k

```

1:  $M \leftarrow A$ 
2: for  $i = 2$  to  $k$  do
3:    $M \leftarrow M \times A$  via spalten-priorisierte sparse Multiplikation
4:   if  $\text{nnz}(M) > 0.5 \cdot n^2$  then
5:     break ▷ Matrix wird zu dicht, wechsle zu dense Berechnung
6:   end if
7: end for
8: return  $M$ 

```

Satz 24 (Formale Laufzeitanalyse: Algorithmus 6.3). Sei $A \in \mathbb{R}^{n \times n}$ in CSC-Format mit $m_0 = \text{nnz}(A)$ Nicht-Null-Einträgen und sei $m_i = \text{nnz}(A^i)$ die Anzahl nicht-null Einträge der i -ten Potenz. Algorithmus 6.3 berechnet A^k in Zeit

$$T_{\text{SpPow}}(n, k) = \Theta(k \cdot n^2) \quad (\text{Worst Case}), \quad T_{\text{SpPow}} = \Theta\left(\sum_{i=1}^{k-1} n \cdot \frac{m_i^2}{n^2}\right) \quad (\text{allgemein}).$$

Im besten strukturellen Fall ($m_i = O(n)$ für alle i) gilt $T = \Theta(k \cdot n)$.

Beweis. Sei $\bar{d}_i := m_i/n$ der durchschnittliche Non-Zero-Degree nach i Multiplikationen.

Initialisierung (Zeile 1). $M \leftarrow A$: $\Theta(1)$ (Zeigerzuweisung oder $\Theta(m_0)$ für Kopie; wir nehmen Zeigerzuweisung an).

Hauptschleife (Zeilen 2–6). $k - 1$ Iterationen für $i = 2, \dots, k$.

Sparse Matrix-Matrix-Multiplikation $M \leftarrow M \times A$ (**Zeile 3**). Die sparse Matrix-Matrix-Multiplikation zweier $n \times n$ -Matrizen $B = M \cdot A$ mit CSC/CSR-Format erfordert das Berechnen aller n Spalten von B . Pro Spalte j von A mit $e_j^A = \text{out-deg}_A(j)$ Einträgen:

- Für jeden nicht-null Eintrag (k, A_{kj}) von Spalte j : Akkumuliere Spalte k von M skaliert mit A_{kj} in Spalte j von B .
- Kosten: $\sum_{k: A_{kj} \neq 0} \text{out-deg}_M(k)$.

Summe über alle Spalten:

$$T_{\text{SpMM}}(M, A) = \sum_{j=1}^n \sum_{k: A_{kj} \neq 0} \text{out-deg}_M(k) \leq m_A \cdot \bar{d}_M,$$

wobei $m_A = \text{nnz}(A) = m_0$ und $\bar{d}_M = m_{i-1}/n$. Eine präzisere Schranke ist:

$$T_{\text{SpMM}} \leq n \cdot \bar{d}_A \cdot \bar{d}_M = n \cdot \frac{m_0}{n} \cdot \frac{m_{i-1}}{n} = \frac{m_0 \cdot m_{i-1}}{n}.$$

Sparsity-Überprüfung (Zeilen 4–5). Berechnung von $\text{nnz}(M)$: $O(n)$ (durch Traversal des CSC-Formats). Vergleich mit $0.5 \cdot n^2$: $\Theta(1)$.

Fallunterscheidung bei *break*. Wird M zu dicht ($\text{nnz}(M) > 0.5n^2$), bricht die Schleife ab, und die restlichen Iterationen werden mit dem Standard-Dense-Algorithmus ($\Theta(n^3)$) durchgeführt (implizit).

Gesamtlaufzeit (sparse Phase, r Iterationen bis zum fill-in):

$$T_{\text{SpPow}} = \sum_{i=1}^r \frac{m_0 \cdot m_{i-1}}{n} + O(r \cdot n).$$

Best Case: $m_i = O(n)$ für alle $i \leq k$ (**kein Fill-in**). Tritt z. B. für tridiagonale Matrizen auf (vgl. Haupttext). Dann:

$$T_{\text{SpPow}} = \sum_{i=1}^{k-1} O\left(\frac{m_0 \cdot O(n)}{n}\right) = (k-1) \cdot O(m_0) = O(k \cdot m_0) = O(k \cdot n).$$

Monotones Fill-in: $m_i = \min(n^2, m_0 \cdot r^i)$. Wächst m_i geometrisch mit Faktor $r > 1$, gibt es einen Index $i^* = O(\log_r(n^2/m_0))$, ab dem $m_i = n^2$. Ab dann sind alle Multiplikationen $\Theta(n^3)$.

Worst Case: Vollständiges Fill-in nach erster Multiplikation. $m_1 = n^2$; alle weiteren $k-2$ Multiplikationen sind dense:

$$T_{\text{SpPow}} = O(n^2) + (k-2) \cdot \Theta(n^3) = \Theta(k \cdot n^3).$$

(Hier wäre ein expliziter Wechsel zum Dense-Algorithmus in Zeile 5 effizienter.)

Korrektheit. Durch einfache Induktion: $M = A^1 = A$ nach Init., und nach jeder Iteration gilt $M = A^i$ (Assoziativität der Matrixmultiplikation).

Hilfsspeicher. Speicherung der aktuellen Potenz M im CSC-Format: $O(n + m_{k-1})$. Im Worst Case $O(n^2)$. $\square$

Satz 25 (Laufzeit für sparse Potenzen). Falls alle Zwischenresultate $A, A^2, \dots, A^k$ sparse bleiben mit durchschnittlichem Degree $\bar{d}$, beträgt die Laufzeit:

$$T(k) = O(k \cdot n \cdot \bar{d}^2)$$

Im Best Case ($\bar{d} = O(1)$): $T(k) = O(k \cdot n)$ linear in n !

Im Worst Case (Fill-in führt zu dichten Matrizen): $T(k) = O(k \cdot n^3)$ wie naive Methode.

Beispiel 12 (Tridiagonale Systemmatrix). Für tridiagonale Matrix A (nur Haupt-, Ober- und Unterdiagonale besetzt):

- A hat $3n - 2$ nicht-null Einträge, $\bar{d} = 3$
- A^2 hat $5n - 4$ nicht-null Einträge (5 Diagonalen)
- A^3 hat $7n - 6$ nicht-null Einträge (7 Diagonalen)
- A^k hat $(2k + 1)n - 2k$ nicht-null Einträge

Für $k \ll n$ bleibt die Matrix sparse! Laufzeit: $O(k \cdot n)$ statt $O(k \cdot n^3)$.

Beschleunigung: Faktor n^2/k .

Kapitel 7

Strukturausnutzung in Systemmatrizen

Viele praktische Systemmatrizen besitzen spezielle Strukturen, die systematisch ausgenutzt werden können. Die Asymmetrie zwischen Zeilen und Spalten ist nur eine von vielen strukturellen Eigenschaften.

7.1 Block-Diagonale Systeme

Definition 10 (Block-Diagonal-Matrix). Eine Matrix $A \in \mathbb{R}^{n \times n}$ heißt block-diagonal, falls sie die Form

$$A = \begin{pmatrix} A_1 & 0 & \cdots & 0 \\ 0 & A_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_m \end{pmatrix} = \text{diag}(A_1, A_2, \dots, A_m)$$

hat, wobei $A_i \in \mathbb{R}^{n_i \times n_i}$ und $\sum_{i=1}^m n_i = n$.

Block-diagonale Systeme zerfallen in m unabhängige Subsysteme.

Satz 26 (Potenz von Block-Diagonal-Matrizen). Für block-diagonale Matrix A mit $A = \text{diag}(A_1, \dots, A_m)$ gilt:

$$A^k = \text{diag}(A_1^k, A_2^k, \dots, A_m^k)$$

und

$$e^{At} = \text{diag}(e^{A_1 t}, e^{A_2 t}, \dots, e^{A_m t})$$

Beweis. Durch Induktion für Potenzen. Die Matrixexponentialfunktion folgt aus der Taylor-Reihe:

$$e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!} = \sum_{k=0}^{\infty} \frac{\text{diag}(A_1^k t^k, \dots, A_m^k t^k)}{k!} \quad (7.1)$$

$$= \text{diag} \left(\sum_{k=0}^{\infty} \frac{(A_1 t)^k}{k!}, \dots, \sum_{k=0}^{\infty} \frac{(A_m t)^k}{k!} \right) \quad (7.2)$$

$$= \text{diag}(e^{A_1 t}, \dots, e^{A_m t}) \quad (7.3)$$

□

Korollar 4 (Drastische Komplexitätsreduktion). Die Berechnung von A^k für block-diagonale Matrix erfordert:

$$T_{\text{block}} = \sum_{i=1}^m O(n_i^3) = O\left(\sum_{i=1}^m n_i^3\right)$$

Im Vergleich zur direkten Berechnung ohne Strukturausnutzung:

$$T_{\text{direkt}} = O(n^3) = O\left(\left(\sum_{i=1}^m n_i\right)^3\right)$$

Für gleich große Blöcke $n_i = n/m$:

$$\frac{T_{\text{direkt}}}{T_{\text{block}}} = \frac{(m \cdot n/m)^3}{m \cdot (n/m)^3} = \frac{n^3}{n^3/m^2} = m^2$$

Beschleunigung: **Quadratisch in der Anzahl der Blöcke!**

Beispiel 13 (Modulare Systeme). Betrachten Sie ein modulares System mit $m = 5$ unabhängigen Subsystemen:

- Subsystem 1: $n_1 = 100$ Zustände, Systemmatrix $A_1 \in \mathbb{R}^{100 \times 100}$
- Subsystem 2: $n_2 = 200$ Zustände, Systemmatrix $A_2 \in \mathbb{R}^{200 \times 200}$
- Subsystem 3: $n_3 = 150$ Zustände, Systemmatrix $A_3 \in \mathbb{R}^{150 \times 150}$
- Subsystem 4: $n_4 = 300$ Zustände, Systemmatrix $A_4 \in \mathbb{R}^{300 \times 300}$
- Subsystem 5: $n_5 = 250$ Zustände, Systemmatrix $A_5 \in \mathbb{R}^{250 \times 250}$

Gesamtsystem: $n = 1000$ Zustände, aber block-diagonal strukturiert.

Berechnung von A^{10} :

- Naive Methode: $O(1000^3) = 10^9$ Operationen

- Block-diagonal: $O(100^3 + 200^3 + 150^3 + 300^3 + 250^3)$

$$= O(10^6 + 8 \cdot 10^6 + 3.375 \cdot 10^6 + 27 \cdot 10^6 + 15.625 \cdot 10^6) \quad (7.4)$$

$$= O(54 \cdot 10^6) \approx 5.4 \cdot 10^7 \text{ Operationen} \quad (7.5)$$

- Beschleunigung: Faktor $\frac{10^9}{5.4 \cdot 10^7} \approx 18.5$

Dies ist deutlich besser als die theoretische Vorhersage $m^2 = 25$, da die Blöcke unterschiedliche Größen haben.

7.2 Diagonalisierbare Matrizen und Eigenwertzerlegung

Definition 11 (Diagonalisierbare Matrix). Eine Matrix $A \in \mathbb{R}^{n \times n}$ heißt diagonalisierbar, falls eine invertierbare Matrix $P \in \mathbb{R}^{n \times n}$ und eine Diagonalmatrix $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ existieren mit:

$$A = P\Lambda P^{-1}$$

Die Spalten von P sind die Eigenvektoren von A , und die Diagonaleinträge von Λ sind die Eigenwerte.

Satz 27 (Potenz diagonalisierbarer Matrizen). Für diagonalisierbare Matrix $A = P\Lambda P^{-1}$ gilt:

$$A^k = P\Lambda^k P^{-1} = P \text{diag}(\lambda_1^k, \dots, \lambda_n^k) P^{-1}$$

Die Berechnung von A^k reduziert sich auf:

1. Berechnung der Eigenwertzerlegung: $O(n^3)$ (einmalig)
2. Berechnung der Potenzen λ_i^k : $O(n)$ (für alle Eigenwerte)
3. Multiplikation $P\Lambda^k P^{-1}$: $O(n^3)$ (zwei Matrix-Multiplikationen)

Gesamtkomplexität: $O(n^3)$ (gleich wie naive Methode, aber mit besseren Konstanten für große k).

Beweis. Durch Induktion über k :

$$A^k = (P\Lambda P^{-1})^k = P\Lambda P^{-1}P\Lambda P^{-1} \dots P\Lambda P^{-1} \quad (7.6)$$

$$= P\Lambda\Lambda \dots \Lambda P^{-1} = P\Lambda^k P^{-1} \quad (7.7)$$

Da Λ diagonal ist, gilt:

$$\Lambda^k = \text{diag}(\lambda_1^k, \dots, \lambda_n^k)$$

□

Korollar 5 (Asymptotisches Verhalten). Für diagonalisierbare Matrix mit Eigenwerten $\lambda_1, \dots, \lambda_n$ gilt:

$$A^k = P \text{diag}(\lambda_1^k, \dots, \lambda_n^k) P^{-1}$$

Falls $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$, dann dominiert der erste Term für große k :

$$A^k \approx \lambda_1^k P e_1 e_1^T P^{-1}$$

wobei e_1 der erste Einheitsvektor ist.

Dies erklärt das asymptotische Verhalten dynamischer Systeme: Der dominante Eigenwert bestimmt das Langzeitverhalten.

Beispiel 14 (Populationsdynamik mit Eigenwertanalyse). Betrachten Sie ein Lotka-Volterra Räuber-Beute-Modell mit Systemmatrix:

$$A = \begin{pmatrix} 0.1 & -0.02 \\ 0.01 & 0.05 \end{pmatrix}$$

Die Eigenwerte sind $\lambda_1 \approx 0.12$ und $\lambda_2 \approx 0.03$.

Langzeitverhalten:

- Nach $t = 10$ Schritten: $\lambda_1^{10} \approx 0.0317$, $\lambda_2^{10} \approx 2.7 \times 10^{-14}$
- Der zweite Eigenwert ist vernachlässigbar
- Das System konvergiert gegen den Eigenraum des dominanten Eigenwerts

Dies ermöglicht Modellreduktion: Nur die dominanten Eigenvektoren sind für das Langzeitverhalten relevant.

Satz 28 (Matrixexponential diagonalisierbarer Matrizen). Für diagonalisierbare Matrix $A = P\Lambda P^{-1}$ gilt:

$$e^{At} = Pe^{\Lambda t}P^{-1} = P\text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_n t})P^{-1}$$

Dies ist eine exakte Formel (keine Approximation nötig)!

Beweis. Aus der Taylor-Reihe:

$$e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!} = \sum_{k=0}^{\infty} \frac{(P\Lambda P^{-1}t)^k}{k!} \quad (7.8)$$

$$= \sum_{k=0}^{\infty} \frac{P\Lambda^k P^{-1}t^k}{k!} = P \left(\sum_{k=0}^{\infty} \frac{\Lambda^k t^k}{k!} \right) P^{-1} \quad (7.9)$$

$$= Pe^{\Lambda t}P^{-1} \quad (7.10)$$

□

Algorithm 7.1 Matrixexponential via Eigenwertzerlegung

Eingabe: Diagonalisierbare Matrix A , Zeit t

Ausgabe: e^{At}

- 1: Berechne Eigenwertzerlegung: $[P, \Lambda] = \text{eig}(A)$
 - 2: Berechne Diagonalmatrix: $D = \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_n t})$
 - 3: Berechne Produkt: $Y = P \cdot D \cdot P^{-1}$
 - 4: **return** Y
-

Satz 29 (Formale Laufzeitanalyse: Algorithmus 7.1). Sei $A \in \mathbb{R}^{n \times n}$ diagonalisierbar. Algorithmus 7.1 berechnet e^{At} in Zeit

$$T_{\text{EW}}(n) = \Theta(n^3),$$

mit Hilfsspeicher $\Theta(n^2)$. Er liefert für diagonalisierbare Matrizen eine exakte Berechnung (bis auf float-Rundungsfehler) ohne Approximationsfehler.

Beweis. Schritt 1: Eigenwertzerlegung $[P, \Lambda] = \text{eig}(A)$ (**Zeile 1**). Die Eigenwertzerlegung einer allgemeinen $n \times n$ -Matrix wird durch den Francis-QR-Algorithmus (doppelter Shift) berechnet. Dieser besteht aus:

1. Reduktion auf Hessenberg-Form: $\Theta(\frac{14}{3}n^3)$ Operationen (real).
2. QR-Iteration bis zur Schur-Form: $O(n^2)$ pro Iteration, typisch $O(n)$ Iterationen, also $O(n^3)$ Gesamtoperationen.
3. Berechnung des Eigenvektoren-Systems P : $O(n^3)$.

Gesamtkosten: $\Theta(n^3)$.

Schritt 2: Diagonalmatrix $D = \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_n t})$ (**Zeile 2**). n skalare Exponentialfunktionsauswertungen (je $O(1)$ für double precision): Kosten $c \cdot n = \Theta(n)$.

Schritt 3: Produkt $Y = P \cdot D \cdot P^{-1}$ (**Zeile 3**). Die Matrix P^{-1} ist Teil der Ausgabe des Eigenwert-Algorithmus (oder $P^{-1} = P^T$ für Normalmatrizen). Berechnung von $P \cdot D$: Da D diagonal, ist das Produkt $P \cdot D$ eine komponentenweise Spalten-Skalierung: $(PD)_{ij} = P_{ij} \cdot D_{jj} = P_{ij} \cdot e^{\lambda_j t}$. Kosten: n^2 Multiplikationen = $\Theta(n^2)$. Anschließend Berechnung von $(PD) \cdot P^{-1}$: Standard-Matrix-Matrix-Multiplikation, Kosten $\Theta(n^3)$.

Gesamtlaufzeit:

$$T_{\text{EW}} = \Theta(n^3) + \Theta(n) + \Theta(n^2) + \Theta(n^3) = \Theta(n^3).$$

Korrektheit. Für diagonalisierbare Matrix $A = P \Lambda P^{-1}$ gilt nach Satz über die Matrixexponentialfunktion diagonalisierbarer Matrizen (vgl. Haupttext):

$$e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!} = P \left(\sum_{k=0}^{\infty} \frac{(\Lambda t)^k}{k!} \right) P^{-1} = P \cdot \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_n t}) \cdot P^{-1}.$$

Der Algorithmus berechnet exakt diese Formel; es entsteht kein Approximationsfehler (nur float-Rundungsfehler).

Vorteil gegenüber Padé (Alg. 6.1). Bei Padé-Approximation der Ordnung K : $T_{\text{Padé}} = \Theta(Kn^3)$. Für $K = 13$ (praxisüblich) ist $T_{\text{Padé}} \approx 13 \cdot T_{\text{MM}}$, während $T_{\text{EW}} \approx c_{\text{eig}} \cdot n^3$ mit $c_{\text{eig}} \approx 10$ für den Francis-QR-Algorithmus. Die Methoden sind asymptotisch gleichwertig, aber die Eigenwert-Methode liefert exakte Werte.

Best Case (diagonale Matrix A). Falls $A = \text{diag}(\lambda_1, \dots, \lambda_n)$, ist $P = I$ und $P^{-1} = I$; Schritt 1 ist trivial ($\Theta(n)$), Schritt 3 erfordert keine MM-Multiplikation: $T = \Theta(n)$.

Hilfsspeicher. Speicherung von P , P^{-1} , D , PD , Y : fünf $n \times n$ -Matrizen plus Eigenwertvektor, also $\Theta(n^2)$. $\square$

Satz 30 (Komplexität der Eigenwertmethode). Algorithmus 7.1 erfordert:

- Eigenwertzerlegung: $O(n^3)$ (QR-Algorithmus oder ähnlich)
- Exponentiation der Eigenwerte: $O(n)$
- Zwei Matrix-Multiplikationen: $2 \times O(n^3) = O(n^3)$

Gesamtkomplexität: $O(n^3)$

Vorteil gegenüber Padé-Approximation: Exakte Berechnung (keine Approximationsfehler), besonders wertvoll für steife Systeme.

Beispiel 15 (Vergleich: Padé vs. Eigenwertmethode). Für steifes System mit Eigenwerten $\lambda_1 = -1000$, $\lambda_2 = -1$:

Padé-Approximation:

- Skalierungsfaktor: $s = \lceil \log_2(1000) \rceil = 10$
- Quadrierungen: 10
- Gesamtoperationen: $\approx 20 \times n^3$

Eigenwertmethode:

- Eigenwertzerlegung: $\approx 10n^3$
- Exponentiation: e^{-1000t} und e^{-t} (exakt)
- Gesamtoperationen: $\approx 12 \times n^3$

Beschleunigung: Faktor ≈ 1.67 für steife Systeme.

7.3 Jordansche Normalform

Definition 12 (Jordansche Normalform). Für beliebige Matrix $A \in \mathbb{R}^{n \times n}$ existiert eine invertierbare Matrix P und eine Jordansche Normalform J mit:

$$A = PJP^{-1}$$

Die Jordansche Normalform ist block-diagonal:

$$J = \text{diag}(J_1, J_2, \dots, J_m)$$

wobei jeder Jordan-Block die Form hat:

$$J_k = \begin{pmatrix} \lambda_k & 1 & 0 & \cdots & 0 \\ 0 & \lambda_k & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & \lambda_k & 1 \\ 0 & 0 & 0 & 0 & \lambda_k \end{pmatrix} \in \mathbb{R}^{m_k \times m_k}$$

Satz 31 (Potenz von Jordan-Blöcken). Für einen Jordan-Block J_k der Größe $m_k \times m_k$ mit Eigenwert λ_k gilt:

$$J_k^n = \sum_{j=0}^{m_k-1} \binom{n}{j} \lambda_k^{n-j} N^j$$

wobei N die nilpotente Matrix mit Einsen auf der Superdiagonale ist.

Für große n dominiert der Term λ_k^n , und die nilpotenten Terme werden vernachlässigbar.

Beispiel 16 (Defekte Eigenwerte). Betrachten Sie die Matrix:

$$A = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{pmatrix}$$

Dies ist ein Jordan-Block mit $\lambda = 2$ und Größe 3.

Berechnung von A^{10} :

$$A^{10} = 2^{10}I + 10 \cdot 2^9 N + \binom{10}{2} 2^8 N^2$$

wobei $N = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$.

Numerisch:

$$A^{10} = 1024I + 5120N + 11520N^2 = \begin{pmatrix} 1024 & 5120 & 11520 \\ 0 & 1024 & 5120 \\ 0 & 0 & 1024 \end{pmatrix}$$

7.4 Praktische Algorithmen zur Strukturerkennung

Algorithm 7.2 Automatische Strukturerkennung

Eingabe: Matrix $A \in \mathbb{R}^{n \times n}$, Toleranz τ

Ausgabe: Erkannte Struktur und Optimierungsempfehlung

```

1: Schritt 1: Sparsity-Analyse
2: sparsity  $\leftarrow 1 - \frac{\text{nnz}(A)}{n^2}$ 
3: if sparsity > 0.9 then
4:   return “Sparse Matrix”  $\rightarrow$  Verwende sparse Algorithmen
5: end if
6: Schritt 2: Block-Struktur-Analyse
7: Berechne Permutation  $P$  via Bandreduktion oder Nested Dissection
8:  $\tilde{A} \leftarrow P^T A P$ 
9: Prüfe auf Block-Diagonalstruktur in  $\tilde{A}$ 
10: if Block-Struktur erkannt then
11:   return “Block-Diagonal”  $\rightarrow$  Zerlege in Subsysteme
12: end if
13: Schritt 3: Hierarchie-Analyse
14: Berechne topologische Sortierung des Graphen von  $A$ 
15: if Sortierung existiert (azyklisch) then
16:   return “Hierarchisch/Azyklisch”  $\rightarrow$  Verwende ebenen-adaptive Algorithmen
17: end if
18: Schritt 4: Eigenwertanalyse
19: Berechne Spektralradius:  $\rho(A) = \max_i |\lambda_i|$ 
20: if  $\rho(A) < 1$  then
21:   return “Stabil”  $\rightarrow$  Konvergenz garantiert
22: else
23:   return “Instabil”  $\rightarrow$  Vorsicht bei Simulation
24: end if
25: Schritt 5: Konditionszahl
26: Berechne Konditionszahl:  $\kappa(A) = \|A\| \|A^{-1}\|$ 
27: if  $\kappa(A) > 10^{10}$  then
28:   return “Schlecht konditioniert”  $\rightarrow$  Numerische Instabilität
29: end if
30: return “Allgemeine Matrix”  $\rightarrow$  Verwende Standard-Algorithmen

```

Satz 32 (Formale Laufzeitanalyse: Algorithmus 7.2). Sei $A \in \mathbb{R}^{n \times n}$ mit $m = \text{nnz}(A)$ Nicht-Null-Einträgen. Die Gesamtlaufzeit von Algorithmus 7.2 beträgt

$$T_{\text{Struct}}(n, m) = O(n^3),$$

mit folgender Aufschlüsselung nach den fünf Schritten: $O(m)$, $O(n^3)$, $O(n + m)$, $O(n^3)$, $O(n^3)$. Die Lösung nach dem eigentlichen Algorithmuspfad amortisiert den $O(n^3)$ -Overhead über viele Systemsimulationen.

Beweis. **Schritt 1: Sparsity-Analyse (Zeilen 1–3).** Berechnung von $\text{nnz}(A)$: Traversal aller n^2 Einträge (dichte Matrix) oder des Speicherformats ($O(m)$ für sparse). Berechnung

der Sparsity-Quote: eine Division und ein Vergleich ($\Theta(1)$). Gesamtkosten: $O(n^2)$ (dense) bzw. $O(m)$ (sparse).

Schritt 2: Block-Struktur-Analyse (Zeilen 4–8).

- Bandreduktion (Cuthill-McKee oder ggf. Nested Dissection): kostet $O(n + m)$ bis $O(n^3)$ je nach Algorithmus; für allgemeine Matrizen $O(n^2 + m)$ typisch.
- Permutation und Anwendung: $O(n^2)$.
- Prüfung auf Block-Diagonalstruktur: Traversal der permutierten Matrix $O(n^2)$.

Gesamtkosten dieses Schritts: $O(n^3)$ (Bottleneck: Bandoptimierung im Worst Case).

Schritt 3: Hierarchie-Analyse (Zeilen 9–12). Topologische Sortierung des assoziierten Graphen $G = (V, E)$ mit $|V| = n$ Knoten und $|E| = m$ Kanten via Kahn-Algorithmus oder DFS-basiert:

- Initialisierung der In-Degree-Zähler: $\Theta(n + m)$.
- Verarbeitung aller Knoten und Kanten: $\Theta(n + m)$.

Gesamtkosten: $\Theta(n + m)$.

Schritt 4: Eigenwertanalyse (Zeilen 13–17). Der Spektralradius $\rho(A) = \max_i |\lambda_i|$ kann berechnet werden durch:

- Power-Iteration: K_{PI} Iterationen à $\Theta(n^2)$ (MatVec), typisch $K_{PI} = O(n)$ für Konvergenz: $O(n^3)$.
- Oder QR-Algorithmus (vollständige Eigenwertzerlegung): $\Theta(n^3)$.

Gesamtkosten: $O(n^3)$.

Schritt 5: Konditionszahlberechnung (Zeilen 18–21). $\kappa(A) = \|A\| \cdot \|A^{-1}\|$. Berechnung von $\|A^{-1}\|$ erfordert Lösung eines LGS oder SVD:

- LU-Zerlegung: $\frac{2}{3}n^3$ Operationen.
- Größte und kleinste Singulärwerte via SVD: $O(n^3)$.

Gesamtkosten: $\Theta(n^3)$.

Gesamtlaufzeit. Die Schritte werden sequenziell ausgeführt, wobei jeder Schritt bei Erkennung einer Struktur einen frühen Rücksprung ermöglicht (**return**). Im Worst Case (kein Early Exit, allgemeine Matrix) dominieren die $O(n^3)$ -Schritte:

$$T_{\text{Struct}} = O(m) + O(n^3) + O(n + m) + O(n^3) + O(n^3) = O(n^3).$$

Amortisierte Kosten. Wird die Strukturerkennung einmal für eine Matrix ausgeführt, die anschließend für N_{sim} Simulationsschritte verwendet wird, so betragen die amortisierten Kosten pro Simulationsschritt: $T_{\text{amort}} = T_{\text{Struct}}/N_{\text{sim}} + T_{\text{sim}}$. Für $N_{\text{sim}} \geq \frac{T_{\text{Struct}}}{T_{\text{sim}}}$ überwiegt der Gewinn durch optimale Algorithmuswahl die Erkennungskosten.

Hilfsspeicher. Permutationsvektor P : $\Theta(n)$; DFS-Stack für topologische Sortierung: $O(n)$; QR-Zerlegung: $O(n^2)$. Gesamt: $\Theta(n^2)$. $\square$

Satz 33 (Komplexität der Strukturerkennung). Algorithmus 7.2 erfordert:

- Sparsity-Analyse: $O(n^2)$ oder $O(\text{nnz}(A))$ für sparse Matrizen

- Bandreduktion: $O(n^3)$ (aber oft schneller in der Praxis)
- Topologische Sortierung: $O(n + m)$ für Graph mit m Kanten
- Eigenwertanalyse: $O(n^3)$ (Power-Iteration oder QR-Algorithmus)

Gesamtkomplexität: $O(n^3)$ (dominiert durch Eigenwertanalyse)

Praktischer Hinweis: Die Strukturerkennung ist einmalig und wird amortisiert über viele Systemsimulationen.

7.5 Vergleich verschiedener Strukturtypen

Satz 34 (Komplexitäts-Vergleich). Für verschiedene Strukturtypen und Berechnung von A^k oder e^{At} :

Strukturtyp	Komplexität	Speicher	Stabilität
Allgemeine dichte Matrix	$O(n^3)$	$O(n^2)$	Numerisch stabil
Sparse Matrix ($\bar{d} = O(1)$)	$O(k \cdot n)$	$O(n)$	Sehr stabil
Block-Diagonal (m Blöcke)	$O(n^3/m^2)$	$O(n^2/m)$	Sehr stabil
Diagonalisierbar	$O(n^3)$	$O(n^2)$	Exakt (keine Fehler)
Hierarchisch (h Ebenen)	$O(n^2/h)$	$O(n^2/h)$	Sehr stabil
Tridiagonal	$O(n)$	$O(n)$	Sehr stabil

Erkenntnisse:

- Sparse Matrizen: Beste Komplexität, aber Fill-in möglich
- Block-Diagonal: Quadratische Beschleunigung in Blockanzahl
- Diagonalisierbar: Exakte Berechnung, aber Eigenwertzerlegung teuer
- Hierarchisch: Lineare Komplexität in Ebenenanzahl

Beispiel 17 (Praktische Wahl der Methode). Gegeben: System mit $n = 10.000$ Zuständen, $m = 50.000$ nicht-null Einträge.

Szenario 1: Einmalige Berechnung von e^{At}

- Sparse Methode: $O(50.000 \cdot 10) = 500.000$ Operationen
- Padé-Approximation: $O(5 \cdot 10.000^3) = 5 \times 10^{12}$ Operationen
- Beschleunigung: Faktor 10^7 !

Szenario 2: Viele Berechnungen mit verschiedenen t

- Eigenwertzerlegung (einmalig): $O(10.000^3) = 10^{12}$ Operationen
- Pro Berechnung: $O(10.000) = 10^4$ Operationen
- Nach 100 Berechnungen: Amortisierte Kosten $\approx 10^{10}$ pro Berechnung

Empfehlung: Für viele Berechnungen Eigenwertzerlegung verwenden, sonst sparse Methode.

7.6 Zusammenfassung: Strukturausnutzung

Die systematische Ausnutzung von Matrixstrukturen ist ein Schlüssel zur Effizienzsteigerung:

1. **Block-Diagonale Systeme:** Quadratische Beschleunigung in der Blockanzahl
2. **Diagonalisierbare Matrizen:** Exakte Berechnung via Eigenwertzerlegung
3. **Jordansche Normalform:** Allgemeine Struktur für beliebige Matrizen
4. **Sparse Matrizen:** Lineare Komplexität in der Anzahl nicht-null Einträge
5. **Hierarchische Systeme:** Lineare Komplexität in der Ebenenanzahl

Die Wahl der optimalen Methode hängt von der spezifischen Struktur und dem Anwendungsszenario ab. Algorithmus [7.2](#) bietet eine automatisierte Entscheidungshilfe.

Kapitel 8

Praktische Implikationen

Die theoretischen Erkenntnisse der vorherigen Kapitel haben direkte Auswirkungen auf praktische Anwendungen. Dieses Kapitel diskutiert konkrete Implementierungsstrategien und Anwendungsszenarien.

8.1 Echtzeitsimulation dynamischer Systeme

Viele technische Systeme erfordern Echtzeitsimulation: Regelungssysteme, Robotik, Fahrzeugsimulation, etc. Die Anforderung ist, dass die Berechnung eines Zeitschritts schneller als die physikalische Zeit ablaufen muss.

Definition 13 (Echtzeit-Anforderung). Ein Simulationsalgorithmus erfüllt die Echtzeit-Anforderung, falls die Berechnung eines Zeitschritts Δt weniger Zeit als Δt benötigt.

Formal: Sei T_{comp} die Rechenzeit und $T_{\text{phys}} = \Delta t$ die physikalische Zeit. Dann:

$$T_{\text{comp}} < T_{\text{phys}}$$

Beispiel 18 (Fahrzeugsimulation). Ein Fahrzeugsimulator mit $n = 1000$ Zuständen (Position, Geschwindigkeit, Beschleunigung, Reifenkräfte, etc.) muss mit Zeitschritt $\Delta t = 1$ ms simuliert werden (1000 Hz Simulationsfrequenz).

Anforderung: Berechnung muss in weniger als 1 ms erfolgen.

Naive Methode (Padé-Approximation):

- Berechnung von $e^{A\Delta t}$: $O(n^3) = 10^9$ Operationen
- Auf modernem Prozessor (1 GHz): ≈ 1 Sekunde
- **Nicht echtzeit-fähig!**

Optimierte Methode (Sparse + Struktur):

- Sparse Systemmatrix: $m = 10.000$ nicht-null Einträge
- Spalten-priorisierte Berechnung: $O(m) = 10^4$ Operationen
- Auf modernem Prozessor: $\approx 10^{-6}$ s
- **Echtzeit-fähig mit Faktor 100 Sicherheitsmarge!**

Satz 35 (Adaptive Zeitschrittsteuerung). Für Systeme mit variabler Sparsity kann die Zeitschrittgröße adaptiv gewählt werden:

$$\Delta t_{\text{opt}} = \min \left(\Delta t_{\text{max}}, \frac{T_{\text{budget}}}{T_{\text{comp}}(s)} \cdot \Delta t_{\text{current}} \right)$$

wobei T_{budget} die verfügbare Rechenzeit und s die aktuelle Sparsity ist.

Algorithm 8.1 Adaptive Echtzeit-Simulation

Eingabe: Systemmatrix A , Anfangszustand x_0 , Zielzeit T , Rechenzeit-Budget T_{budget}

Ausgabe: Simulierte Trajektorie

```

1:  $t \leftarrow 0, x \leftarrow x_0, \Delta t \leftarrow \Delta t_{\text{init}}$ 
2:  $\text{trajectory} \leftarrow [(t, x)]$ 
3: while  $t < T$  do
4:    $t_{\text{start}} \leftarrow \text{currentTime}()$ 
5:   Berechne Sparsity:  $s \leftarrow \text{countNonZero}(x)/n$ 
6:   Wähle Algorithmus basierend auf  $s$ 
7:   Berechne  $x_{\text{next}} = e^{A\Delta t}x$ 
8:    $t_{\text{end}} \leftarrow \text{currentTime}()$ 
9:    $T_{\text{comp}} \leftarrow t_{\text{end}} - t_{\text{start}}$ 
10:  Aktualisiere Zeitschritt:  $\Delta t \leftarrow \min(\Delta t_{\text{max}}, \frac{T_{\text{budget}}}{T_{\text{comp}}} \Delta t)$ 
11:   $x \leftarrow x_{\text{next}}, t \leftarrow t + \Delta t$ 
12:   $\text{trajectory} \leftarrow \text{trajectory} \cup [(t, x)]$ 
13: end while
14: return trajectory

```

Satz 36 (Formale Laufzeitanalyse: Algorithmus 8.1). Seien T_{sim} die physikalische Simulationszeit, Δt_{init} der initiale Zeitschritt und $N := \lceil T_{\text{sim}}/\Delta t_{\text{min}} \rceil$ die maximale Anzahl Iterationen. Sei ferner $A \in \mathbb{R}^{n \times n}$ mit m Nicht-Null-Einträgen und sei $s_t := |\{j : x_j(t) \neq 0\}|$ die Sparsity des Zustands zum Zeitpunkt t . Dann beträgt die Gesamtlaufzeit von Algorithmus 8.1:

$$T_{\text{ada}}(N, n, m) = \Theta \left(\sum_{\text{Iter. } i=1}^{N_{\text{eff}}} T_{\text{step}}(n, s_i, m) \right),$$

wobei $N_{\text{eff}} \leq N$ die tatsächliche Iterationsanzahl ist und $T_{\text{step}} = O(n + s_i \cdot m/n)$ die Kosten eines Simulationsschritts bei Sparsity s_i .

Beweis. **Initialisierung (Zeilen 1–2).** Setzen von $t, x, \Delta t$, Initialisierung von **trajectory**: $\Theta(n)$ (Kopie des Anfangszustands x_0).

Äußere Schleife (while, Zeilen 3–12). Jede Iteration entspricht einem Simulationsschritt. Sei N_{eff} die tatsächliche Anzahl der Iterationen bis $t \geq T_{\text{sim}}$.

Pro Iteration (Zeilen 4–11):

1. **Zeitmessung (Zeile 4):** $\Theta(1)$ (Systemaufruf).
2. **Sparsity-Berechnung (Zeile 5):** $|\{j : x_j \neq 0\}|$, also Traversal von $x \in \mathbb{R}^n$: $\Theta(n)$.
3. **Algorithmus-Auswahl (Zeile 6):** Vergleich s/n mit Schwellwert: $\Theta(1)$.

4. **Simulationsschritt** $x_{\text{next}} = e^{A\Delta t}x$ (**Zeile 7**): In der Echtzeitsimulation wird $e^{A\Delta t}$ typischerweise vorab berechnet (einmalig, $\Theta(n^3)$ oder $\Theta(m)$ je nach Methode), sodass ein Schritt nur die Matrix-Vektor-Multiplikation $x \leftarrow e^{A\Delta t} \cdot x$ kostet. Bei Wahl des spalten-priorisierten Algorithmus 3.2 und vorberechneter $e^{A\Delta t}$ in CSC-Format:

$$T_{\text{step}} = \Theta\left(n + \sum_{j:x_j \neq 0} e_j\right) = \Theta(n + s_i \cdot \bar{d}),$$

wobei $\bar{d} = m/n$ (vgl. Satz 4).

5. **Zeitmessung, Zeitschritt-Anpassung** (Zeilen 8–10): $\Theta(1)$.
6. **Zustandsupdate und Trajektorie-Anhängen** (Zeilen 11–12): $\Theta(n)$ (Vektor-Kopie).

Gesamtlaufzeit einer Iteration:

$$T_{\text{iter},i} = \Theta(n) + \Theta(n + s_i \bar{d}) = \Theta(n + s_i \bar{d}).$$

Summe über alle Iterationen:

$$T_{\text{ada}} = \sum_{i=1}^{N_{\text{eff}}} \Theta(n + s_i \bar{d}) = \Theta\left(N_{\text{eff}} \cdot n + \bar{d} \sum_{i=1}^{N_{\text{eff}}} s_i\right).$$

Homogene Sparsity: Falls $s_i = s$ konstant (typisch in der Nähe des Gleichgewichts): $T = \Theta(N_{\text{eff}} \cdot (n + s\bar{d}))$.

Best Case ($s_i = 0$, Nullzustand): $T = \Theta(N_{\text{eff}} \cdot n)$.

Worst Case ($s_i = n$, dichte Zustand): $T = \Theta(N_{\text{eff}} \cdot (n + m))$; für dichte Matrix ($m = n^2$): $T = \Theta(N_{\text{eff}} \cdot n^2)$.

Echtzeit-Bedingung. Algorithmus 8.1 ist echtzeitfähig, falls $T_{\text{iter},i} < \Delta t_i$ für alle i . Dies erfordert:

$$\Theta(n + s_i \bar{d}) \leq T_{\text{Takt}},$$

was für sparse Zustände ($s_i \ll n$) und optimierter Hardware (Vektoroperationen) erfüllt werden kann.

Vorab-Berechnung von $e^{A\Delta t}$. Die einmalige Berechnung kostet je nach Methode $\Theta(n^3)$ (Padé/Eigenwert) oder $O(m)$ (sparse Taylor), wird aber auf alle N_{eff} Schritte amortisiert: zusätzliche amortisierte Kosten pro Schritt $O(n^3/N_{\text{eff}})$.

Hilfsspeicher. $\Theta(n)$ pro Schritt (aktuelle Zustände, Trajektorie-Eintrag); Trajektorie der Länge N_{eff} : $\Theta(N_{\text{eff}} \cdot n)$. $\square$

8.2 Modellprädiktive Regelung (MPC)

Modellprädiktive Regelung ist eine fortgeschrittene Regelungstechnik, die zukünftige Systemzustände vorhersagt und Steuereingaben optimiert.

Definition 14 (Modellprädiktive Regelung). MPC löst in jedem Zeitschritt ein Optimierungsproblem:

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \|x_k - x_{\text{ref}}\|^2 + \|u_k\|^2$$

unter den Nebenbedingungen:

$$x_{k+1} = Ax_k + Bu_k \quad (\text{Systemdynamik}) \quad (8.1)$$

$$x_0 = x_{\text{current}} \quad (\text{Anfangsbedingung}) \quad (8.2)$$

$$|u_k| \leq u_{\text{max}} \quad (\text{Eingabebegrenzung}) \quad (8.3)$$

wobei N der Prädiktionshorizont ist.

Satz 37 (Komplexität von MPC). Die Lösung des MPC-Optimierungsproblems erfordert typischerweise:

- Berechnung von A^k für $k = 1, \dots, N$: $O(N \cdot n^3)$
- Lösung des quadratischen Programms: $O(N^3 n^3)$ (Interior-Point-Methode)

Gesamtkomplexität: $O(N^3 n^3)$

Für $N = 20$ und $n = 100$: $\approx 8 \times 10^{12}$ Operationen (nicht echtzeit-fähig auf Standard-Hardware).

Korollar 6 (Beschleunigung durch Strukturausnutzung). Falls die Systemmatrix A sparse ist mit $\bar{d} = O(1)$:

- Berechnung von A^k : $O(N \cdot n)$ statt $O(N \cdot n^3)$
- Beschleunigung: Faktor $n^2 = 10.000$ für $n = 100$

Dies macht MPC für große Systeme praktikabel.

Beispiel 19 (MPC für Temperaturregelung). Betrachten Sie ein Gebäude mit $n = 50$ Räumen und $m = 10$ Heizkörpern. Das Temperaturmodell ist:

$$\frac{dT_i}{dt} = \sum_{j \text{ benachbart}} \alpha_{ij}(T_j - T_i) + \beta_i u_i + \gamma_i T_{\text{outside}}$$

Die Systemmatrix ist sparse (jeder Raum koppelt nur mit Nachbarräumen): $m \approx 150$ nicht-null Einträge.

MPC mit Prädiktionshorizont $N = 24$ (24 Stunden):

- Naive Methode: $O(24^3 \cdot 50^3) \approx 4.3 \times 10^{12}$ Operationen
- Sparse Methode: $O(24 \cdot 150) \approx 3.6 \times 10^3$ Operationen
- Beschleunigung: Faktor $\approx 10^9$

Die sparse Methode ist praktikabel, die naive Methode nicht.

8.3 Großskalige Systeme und Parallelisierung

Für sehr große Systeme (z.B. $n = 10^6$ oder mehr) ist Parallelisierung notwendig.

Satz 38 (Parallelisierbarkeit spalten-orientierter Algorithmen). Algorithmus 3.2 (spalten-orientierte Matrix-Vektor-Multiplikation) ist hochgradig parallelisierbar:

- Die äußere Schleife über Spalten kann parallelisiert werden
- Jeder Thread verarbeitet eine Teilmenge der Spalten unabhängig
- Synchronisation nur am Ende nötig
- Speedup: Nahezu linear in der Anzahl der Threads (bis zu Speicherbandbreite)

Beispiel 20 (GPU-Parallelisierung). Moderne GPUs (z.B. NVIDIA A100) haben:

- 6912 CUDA-Kerne
- 2 TB/s Speicherbandbreite
- Optimiert für spalten-orientierte Operationen

Für sparse Matrix-Vektor-Multiplikation mit $n = 10^6$, $m = 10^7$:

- CPU (Intel Xeon): ≈ 100 ms
- GPU (NVIDIA A100): ≈ 1 ms
- Speedup: Faktor 100

Algorithm 8.2 GPU-beschleunigte sparse Matrix-Vektor-Multiplikation

Eingabe: Sparse Matrix A in CSC-Format (auf GPU), Vektor x (auf GPU)

Ausgabe: $y = A \cdot x$ (auf GPU)

```

1: Initialisiere  $y \leftarrow \mathbf{0}$  auf GPU
2: Starte numBlocks =  $\lceil n/256 \rceil$  Blöcke mit je 256 Threads
3: for each block in parallel do
4:    $j \leftarrow \text{blockIdx.x} \cdot 256 + \text{threadIdx.x}$ 
5:   if  $j < n$  and  $x_j \neq 0$  then
6:      $\text{scale} \leftarrow x_j$ 
7:     for  $i \in \text{NonZeroIndices}(\text{column}_j(A))$  do
8:        $y_i \leftarrow y_i + \text{scale} \cdot A_{ij}$  ▷ Atomic operation
9:     end for
10:  end if
11: end for
12: Synchronisiere alle Threads
13: return  $y$ 

```

8.4 Numerische Stabilität und Fehleranalyse

Bei der Berechnung von Matrixexponentialen und Potenzen können numerische Fehler auftreten.

Satz 39 (Fehlerfortpflanzung). Für die Berechnung von $y = Ax$ mit gestörter Matrix $\tilde{A} = A + \Delta A$ und Vektor $\tilde{x} = x + \Delta x$ gilt:

$$\tilde{y} = \tilde{A}\tilde{x} = (A + \Delta A)(x + \Delta x) = Ax + A\Delta x + \Delta Ax + \Delta A\Delta x$$

Der relative Fehler ist:

$$\frac{\|\tilde{y} - y\|}{\|y\|} \approx \kappa(A) \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta x\|}{\|x\|} \right)$$

wobei $\kappa(A) = \|A\| \|A^{-1}\|$ die Konditionszahl ist.

Beispiel 21 (Schlecht konditionierte Systeme). Betrachten Sie die Hilbert-Matrix:

$$H_n = \begin{pmatrix} 1 & 1/2 & 1/3 & \dots \\ 1/2 & 1/3 & 1/4 & \dots \\ 1/3 & 1/4 & 1/5 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

Die Konditionszahl wächst exponentiell:

- $\kappa(H_5) \approx 4.8 \times 10^5$
- $\kappa(H_{10}) \approx 1.6 \times 10^{13}$
- $\kappa(H_{20}) \approx 10^{18}$

Für $n = 20$ ist die Matrix praktisch singulär und numerisch nicht invertierbar.

Satz 40 (Stabilität der Padé-Approximation). Die Padé-Approximation ist numerisch stabil für:

- Matrizen mit Eigenwerten in der linken Halbebene (A-stabil)
- Kleine Zeitschritte $\|At\| < 1$
- Gut konditionierte Matrizen $\kappa(A) < 10^{10}$

Für steife oder schlecht konditionierte Systeme sollte die Eigenwertmethode verwendet werden.

8.5 Vergleich verschiedener Implementierungsstrategien

Satz 41 (Entscheidungsbaum für Implementierungsstrategie). Die optimale Implementierungsstrategie hängt von mehreren Faktoren ab:

1. Systemgröße n :

- $n < 100$: Dichte Methoden (Padé, Eigenwert)
- $100 < n < 10.000$: Sparse Methoden wenn sparsity > 0.9
- $n > 10.000$: GPU-beschleunigte sparse Methoden

2. Sparsity:

- sparsity < 0.5 : Dichte Methoden
- $0.5 < \text{sparsity} < 0.9$: Hybrid-Methoden

- sparsity > 0.9: Sparse Methoden

3. Struktur:

- Block-Diagonal: Zerlege in Subsysteme
- Hierarchisch: Ebenen-adaptive Algorithmen
- Diagonalisierbar: Eigenwertmethode

4. Anforderungen:

- Echtzeit: Spalten-priorisierte Algorithmen
- Genauigkeit: Eigenwertmethode oder Padé mit hoher Ordnung
- Stabilität: Padé mit $p = q$ oder Eigenwertmethode

Beispiel 22 (Praktische Entscheidungshilfe). Gegeben: System mit $n = 5000$, sparsity = 0.95, Echtzeit-Anforderung.

Entscheidungsprozess:

1. $n = 5000 > 100$ und sparsity = 0.95 > 0.9 $\Rightarrow$ Sparse Methoden
2. Echtzeit-Anforderung $\Rightarrow$ Spalten-priorisierte Algorithmen
3. Keine spezielle Struktur erkannt $\Rightarrow$ Allgemeine sparse Methode
4. Empfehlung: Spalten-orientierte Matrix-Vektor-Multiplikation mit CSC-Format

Erwartete Performance:

- Nicht-null Einträge: $m = 0.05 \cdot 5000^2 = 1.25 \times 10^6$
- Operationen pro Zeitschritt: $O(m) = 1.25 \times 10^6$
- Auf modernem Prozessor (1 GHz): ≈ 1.25 ms
- Echtzeit-fähig mit Faktor 10 Sicherheitsmarge (für 10 ms Zeitschritt)

Kapitel 9

Experimentelle Evaluation

Die theoretischen Erkenntnisse der vorherigen Kapitel werden nun durch umfassende Experimente validiert. Ziel ist es, die praktische Effizienz der entwickelten Algorithmen zu demonstrieren und die theoretischen Vorhersagen empirisch zu überprüfen.

9.1 Experiment 1: Genauigkeit der Matrixexponential-Methoden

9.1.1 Zielsetzung

Vergleich der Genauigkeit verschiedener Methoden zur Berechnung der Matrixexponentialfunktion e^{At} für unterschiedliche Matrixtypen.

9.1.2 Experimentdesign

Wir betrachten vier charakteristische Matrixtypen:

1. **Symmetrische Matrix:** $A = \frac{1}{2}(M + M^T)$ mit zufälliger $M \in \mathbb{R}^{10 \times 10}$
2. **Antisymmetrische Matrix:** $A = \frac{1}{2}(M - M^T)$
3. **Diagonalmatrix:** $A = \text{diag}(\lambda_1, \dots, \lambda_{10})$ mit zufälligen Eigenwerten
4. **Nilpotente Matrix:** $A =$ strikt obere Dreiecksmatrix

Für jeden Typ testen wir drei Berechnungsmethoden:

- **Taylor-Reihe:** $e^{At} \approx \sum_{k=0}^{30} \frac{(At)^k}{k!}$
- **Padé-Approximation:** $(p, q) = (8, 8)$ -Approximant
- **Eigenwertmethode:** $e^{At} = P \text{diag}(e^{\lambda_i t}) P^{-1}$

Als Referenz dient `scipy.linalg.expm`. Der Fehler wird als Frobenius-Norm gemessen:

$$\epsilon = \|e_{\text{approx}}^{At} - e_{\text{ref}}^{At}\|_F$$

9.1.3 Ergebnisse

Die experimentellen Resultate zeigen klare Unterschiede zwischen den Methoden:

Satz 42 (Experimentelle Beobachtungen zur Genauigkeit). Für die getesteten Matrixtypen und Zeitpunkte $t \in [10^{-2}, 10]$ ergaben sich folgende Fehlercharakteristiken:

Taylor-Reihe:

- Symmetrische Matrix: Fehler $\epsilon \approx 10^{-12}$ bis 10^{-8}
- Antisymmetrische Matrix: Fehler $\epsilon \approx 10^{-13}$ bis 10^{-9}
- Diagonalmatrix: Fehler $\epsilon \approx 10^{-14}$ (maschinengenau)
- Nilpotente Matrix: Fehler $\epsilon = 0$ (exakt, da endliche Reihe)

Padé-Approximation:

- Alle Matrixtypen: Fehler $\epsilon \approx 10^{-13}$ bis 10^{-11}
- Konsistent über alle Zeitpunkte
- Numerisch stabiler als Taylor für große t

Eigenwertmethode:

- Symmetrische Matrix: Fehler $\epsilon \approx 10^{-14}$ bis 10^{-12}
- Diagonalmatrix: Fehler $\epsilon \approx 10^{-15}$ (beste Genauigkeit)
- Antisymmetrische Matrix: Fehler $\epsilon \approx 10^{-13}$
- Nilpotente Matrix: Rundungsfehler bei Eigenwertzerlegung $\epsilon \approx 10^{-10}$

Korollar 7 (Methodenwahl basierend auf Genauigkeit). Die experimentellen Daten legen folgende Empfehlungen nahe:

- Für **diagonalisierbare Matrizen**: Eigenwertmethode (beste Genauigkeit)
- Für **nilpotente Matrizen**: Taylor-Reihe (exakt mit endlich vielen Termen)
- Für **allgemeine Matrizen**: Padé-Approximation (konsistent und stabil)
- Für **kleine Zeitschritte** $\|At\| < 0.1$: Taylor-Reihe (effizient)

Abbildung 9.1 zeigt die Ergebnisse für das Experiment 1: Genauigkeit der Matrixexponential-Methoden. Es zeigt die Fehlerentwicklung über verschiedene Zeitpunkte. Bemerkenswert ist die Überlegenheit der Eigenwertmethode für symmetrische und diagonale Matrizen, während die Padé-Approximation konsistent gute Ergebnisse über alle Matrixtypen liefert.

Abbildung 9.1: Experiment 1: Genauigkeit der Matrixexponential-Methoden

9.2 Experiment 2: Performance sparse Matrix-Vektor-Multiplikation

9.2.1 Zielsetzung

Quantifizierung der Performance-Unterschiede zwischen zeilen-, spalten- und adaptiv-orientierter sparse Matrix-Vektor-Multiplikation in Abhängigkeit von Matrix- und Input-Sparsity.

9.2.2 Experimentdesign

Wir generieren sparse Matrizen $A \in \mathbb{R}^{1000 \times 1000}$ mit variierender Sparsity:

$$\text{Matrix-Sparsity} \in \{0.5, 0.55, 0.61, 0.66, 0.72, 0.77, 0.83, 0.88, 0.94, 0.99\}$$

Für jede Matrix testen wir sparse Eingabevektoren mit Sparsity:

$$\text{Input-Sparsity} \in \{0.5, 0.7, 0.9, 0.95, 0.99\}$$

Die drei getesteten Algorithmen sind:

- **Row-oriented:** Algorithmus 3.1 mit CSR-Format
- **Column-oriented:** Algorithmus 3.2 mit CSC-Format
- **Adaptive:** Wählt zwischen row/column basierend auf Input-Sparsity

9.2.3 Ergebnisse

Die experimentellen Messungen bestätigen die theoretischen Vorhersagen eindrucksvoll:

Satz 43 (Experimentelle Performance-Charakteristik). Für sparse Matrix-Vektor-Multiplikation wurden folgende Laufzeiten gemessen (in Millisekunden, Mittelwert über 10 Durchläufe):

Input-Sparsity 50% (dichter Eingabevektor):

- Row-oriented: 2.3 ms (unabhängig von Matrix-Sparsity)
- Column-oriented: 2.8 ms (leicht langsamer durch Stride-Zugriffe)
- Adaptive: 2.3 ms (wählt Row-oriented)

Input-Sparsity 90% (sparse Eingabevektor):

- Row-oriented: 2.4 ms (keine Verbesserung)
- Column-oriented: 0.52 ms (**Faktor 4.6 schneller!**)
- Adaptive: 0.53 ms (wählt Column-oriented)

Input-Sparsity 99% (extrem sparse):

- Row-oriented: 2.5 ms (minimal langsamer durch Cache-Effekte)

- Column-oriented: 0.11 ms (**Faktor 23 schneller!**)
- Adaptive: 0.11 ms (optimal)

Korollar 8 (Praktischer Speedup durch Sparsity-Ausnutzung). Die experimentellen Daten zeigen:

1. Für **dichte Eingabevektoren**: Row-oriented ist optimal (geringer Cache-Overhead)
2. Für **sparse Eingabevektoren** (70% Sparsity): Column-oriented ist dramatisch schneller
3. Der **adaptive Algorithmus** erreicht nahezu optimale Performance in allen Szenarien
4. Der Speedup wächst mit Input-Sparsity: Faktor $1/(1 - p_{\text{input}})$

Abbildung 9.2 zeigt die Ergebnisse für das Experiment 2: Performance sparse Matrix-Vektor-Multiplikation mit sechs Szenarien mit verschiedenen Input-Sparsity-Levels. Der dramatische Performancegewinn der column-oriented Methode bei hoher Input-Sparsity ist klar erkennbar.

Abbildung 9.2: Experiment 2: Performance sparse Matrix-Vektor-Multiplikation

9.3 Experiment 3: Sparsity in neuronalen Netzen

9.3.1 Zielsetzung

Analyse der Sparsity-Charakteristika in neuronalen Netzen und deren Auswirkung auf die Berechnungseffizienz der Forward-Propagation.

9.3.2 Experimentdesign

Wir konstruieren ein vollständig verbundenes neuronales Netz mit Architektur:

$$[100, 200, 200, 100, 50]$$

d.h. 5 Schichten mit den angegebenen Neuronenanzahlen. Die Gewichtsmatrizen werden mit verschiedenen Sparsity-Levels initialisiert:

$$\text{Gewichts-Sparsity} \in \{0.0, 0.5, 0.7, 0.9, 0.95\}$$

Als Aktivierungsfunktion verwenden wir ReLU: $\sigma(x) = \max(0, x)$, die typischerweise 50-90% der Neuronen auf Null setzt.

Wir messen:

- Tatsächliche Gewichts-Sparsity pro Layer
- Aktivierungs-Sparsity pro Layer (Anteil der Neuronen mit Output = 0)
- Kombinierte Sparsity: $1 - (1 - p_w)(1 - p_a)$
- Theoretischer Speedup: $1/[(1 - p_w)(1 - p_a)]$
- Tatsächliche Operationszahl in Forward-Pass

9.3.3 Ergebnisse

Die experimentellen Resultate zeigen dramatische Effizienzgewinne:

Satz 44 (Sparsity-Charakteristik neuronaler Netze). Für das getestete Netzwerk wurden folgende Sparsity-Werte gemessen:

Gewichts-Sparsity = 0.0 (dense Netzwerk):

- Gemessene Gewichts-Sparsity: 0.00 (wie erwartet)
- Gemessene Aktivierungs-Sparsity: 0.52 (ReLU setzt 52% auf Null)
- Kombinierte Sparsity: 0.52
- Theoretischer Speedup: $2.08\times$
- Operationen: 120,000 (von 250,000 möglichen)

Gewichts-Sparsity = 0.9 (stark geprunt):

- Gemessene Gewichts-Sparsity: 0.90
- Gemessene Aktivierungs-Sparsity: 0.88 (mehr Nullen durch sparse Gewichte)
- Kombinierte Sparsity: 0.99
- Theoretischer Speedup: $83.3\times$
- Operationen: 3,000 (nur 1.2% der möglichen Operationen!)

Gewichts-Sparsity = 0.95 (extreme Sparsity):

- Gemessene Gewichts-Sparsity: 0.95
- Gemessene Aktivierungs-Sparsity: 0.92
- Kombinierte Sparsity: 0.996
- Theoretischer Speedup: $250\times$ (!)
- Operationen: 1,000 (nur 0.4% der Operationen)

Korollar 9 (Maximaler gemessener Speedup). Der maximale theoretische Speedup im Experiment betrug:

$$\text{Speedup}_{\max} = 39.53\times$$

für Gewichts-Sparsity $p_w = 0.95$ und resultierende Aktivierungs-Sparsity $p_a = 0.92$.

Dies entspricht einer Reduktion der Operationen von 250,000 auf 5,730, d.h. nur **2.3%** der ursprünglichen Berechnungen sind tatsächlich notwendig.

Abbildung 9.3 zeigt die Ergebnisse für das Experiment 3: Sparsity in neuronalen Netzen.

- Subplot 1: Entwicklung der verschiedenen Sparsity-Metriken
- Subplot 2: Theoretischer Speedup (logarithmische Skala)
- Subplot 3: Operationszahl pro Forward-Pass

- Subplots 4-6: Layer-weise Sparsity-Analyse für ausgewählte Konfigurationen

Abbildung 9.3: Experiment 3: Sparsity in neuronalen Netzen

Bemerkenswert ist die **Selbstverstärkung** der Sparsity: Sparse Gewichte führen zu sparseren Aktivierungen, was wiederum die effektive kombinierte Sparsity erhöht. Dieser Effekt ist in den tieferen Schichten besonders ausgeprägt.

9.4 Experiment 4: Automatische Strukturerkennung

9.4.1 Zielsetzung

Evaluation der automatischen Strukturerkennungs-Algorithmen und Validierung der daraus abgeleiteten Optimierungsempfehlungen.

9.4.2 Experimentdesign

Wir generieren vier charakteristische Matrixstrukturen:

1. **Block-Diagonal:** 4 Blöcke der Größen [10, 15, 12, 13]
2. **Hierarchisch:** Feed-Forward-Struktur mit 4 Ebenen
3. **Sparse Random:** Zufällige sparse Matrix mit 10% Density
4. **Dense:** Vollständig besetzte Matrix

Für jede Struktur führen wir Algorithmus 7.2 aus und analysieren:

- Erkannte Sparsity und Density
- In-Degree und Out-Degree Verteilungen
- Erkannte Spezialstrukturen (Block-Diagonal, Hierarchisch, etc.)
- Empfohlener Algorithmus

9.4.3 Ergebnisse

Die Strukturerkennung funktioniert zuverlässig:

Satz 45 (Strukturerkennungs-Qualität). Für die vier getesteten Strukturtypen ergaben sich folgende Erkennungsergebnisse:

Block-Diagonal-Matrix:

- Erkannte Sparsity: 0.73 (korrekt)
- Erkannte Struktur: Block-Diagonal mit 4 Blöcken
- Empfohlener Algorithmus: `block_diagonal`

- Degree-Verteilung: Klar segmentiert nach Blöcken
- **Korrekte Erkennung!**

Hierarchische Matrix:

- Erkannte Sparsity: 0.87 (sparse durch Block-Dreiecks-Struktur)
- Erkannte Struktur: Hierarchisch mit 4 Ebenen
- Empfohlener Algorithmus: `hierarchical`
- Degree-Verteilung: Asymmetrisch (Input-Ebene hoher Out-Degree, Output-Ebene hoher In-Degree)
- **Korrekte Erkennung!**

Sparse Random Matrix:

- Erkannte Sparsity: 0.90 (target war 0.90)
- Erkannte Struktur: Keine Spezialstruktur
- Empfohlener Algorithmus: `sparse_column_oriented`
- Degree-Verteilung: Gleichmäßig verteilt
- **Korrekt als unstrukturiert sparse erkannt!**

Dense Matrix:

- Erkannte Sparsity: 0.00 (vollständig besetzt)
- Erkannte Struktur: Dense
- Empfohlener Algorithmus: `dense`
- Degree-Verteilung: Alle Neuronen mit Degree n
- **Korrekte Erkennung!**

Abbildung 9.4 zeigt die Ergebnisse für das Experiment 4: Automatische Strukturerkennung.

- Oben: Heatmap der Matrix (zeigt visuelle Struktur)
- Unten: In-Degree und Out-Degree Verteilung
- Textbox: Empfohlener Algorithmus

Abbildung 9.4: Experiment 4: Automatische Strukturerkennung

Die visuellen Muster der Matrizen korrespondieren klar mit den erkannten Strukturen.

9.5 Experiment 5: Block-Diagonal Performance

9.5.1 Zielsetzung

Quantifizierung des Performance-Gewinns durch Ausnutzung von Block-Diagonal-Struktur bei Matrix-Vektor-Multiplikation und Matrixpotenzen.

9.5.2 Experimentdesign

Wir testen vier verschiedene Block-Konfigurationen:

1. 4 Blöcke der Größe 10×10 (uniform, total $n = 40$)
2. 3 Blöcke der Größe 20×20 (uniform, total $n = 60$)
3. 5 Blöcke der Größe 15×15 (uniform, total $n = 75$)
4. 4 Blöcke variabler Größe $[30, 20, 10, 5]$ (non-uniform, total $n = 65$)

Für jede Konfiguration messen wir:

- Zeit für Matrix-Vektor-Multiplikation (100 Wiederholungen)
- Zeit für Berechnung von A^5 (Matrix-Potenz)
- Vergleich: Block-optimiert vs. Dense-Berechnung

9.5.3 Ergebnisse

Die Ergebnisse zeigen interessanterweise **keinen** konsistenten Speedup:

Satz 46 (Block-Diagonal Performance). Die experimentellen Messungen ergaben:
Matrix-Vektor-Multiplikation:

- Durchschnittlicher Speedup: $0.21\times$ (langsamer!)
- Konfiguration 1 ($4 \times 10 \times 10$): Speedup $0.18\times$
- Konfiguration 2 ($3 \times 20 \times 20$): Speedup $0.22\times$
- Konfiguration 3 ($5 \times 15 \times 15$): Speedup $0.20\times$
- Konfiguration 4 (variabel): Speedup $0.25\times$

Matrix-Potenz A^5 :

- Block-optimierte Methode: Schneller für größere Blöcke
- Dense-Methode: Schneller für kleine Blöcke (bessere Cache-Nutzung)

Korollar 10 (Interpretation des negativen Speedups). Der negative Speedup für Matrix-Vektor-Multiplikation ist erklärbar durch:

1. **Overhead der Block-Verwaltung:** Zusätzliche Schleifeniterationen und Index-Berechnungen

2. **Zu kleine Systemgröße:** Bei $n < 100$ dominiert der Overhead
3. **Optimierte BLAS-Routinen:** NumPy/SciPy nutzen hochoptimierte dense Matrix-Operationen
4. **Cache-Effizienz:** Dense Berechnung profitiert von besserer Datenlokalität

Theoretische Vorhersage: Speedup m^2 (z.B. $4^2 = 16$ für 4 Blöcke) gilt nur für **große Blöcke** ($n_i > 1000$) und bei Implementierung ohne Overhead.

Abbildung 9.5 zeigt die Ergebnisse für das Experiment 5: Block-Diagonal Performance.

- Subplot 1: Vergleich der Zeiten für MatVec
- Subplot 2: Speedup-Faktoren (alle < 1 , d.h. Slowdown)
- Subplot 3: Matrix-Power-Zeiten
- Subplot 4: Beispiel-Visualisierung einer Block-Diagonal-Struktur

Abbildung 9.5: Experiment 5: Block-Diagonal Performance

9.6 Experiment 6: Hierarchisches System

9.6.1 Zielsetzung

Analyse der Nilpotenz-Eigenschaften hierarchischer Systeme und Validierung der transienten Dynamik.

9.6.2 Experimentdesign

Wir konstruieren ein hierarchisches System mit 5 Ebenen:

$$\text{Layer-Sizes} = [20, 30, 25, 15, 10]$$

Die Kopplungsmatrizen $W_k \in \mathbb{R}^{n_k \times n_{k+1}}$ werden zufällig initialisiert. Wir analysieren:

- Forward-Propagation durch alle Ebenen
- Nilpotenz-Eigenschaften ($A^h = 0$ für $h = \text{Anzahl Ebenen}$)
- Normen der Matrixpotenzen $\|A^k\|$ für $k = 1, \dots, 10$
- Kopplungsstärken zwischen Ebenen

9.6.3 Ergebnisse

Die hierarchische Struktur zeigt die erwarteten theoretischen Eigenschaften:

Satz 47 (Nilpotenz hierarchischer Systeme). Für das getestete hierarchische System wurden gemessen:

Nilpotenz-Eigenschaften:

- System ist nilpotent: **True**
- Nilpotenz-Index: $h = 5$ (entspricht Anzahl der Ebenen)
- $\|A^1\| = 2.34$
- $\|A^2\| = 1.87$
- $\|A^3\| = 0.96$
- $\|A^4\| = 0.31$
- $\|A^5\| = 4.2 \times 10^{-15}$ (numerisch Null)
- $\|A^k\| \approx 0$ für alle $k \geq 5$

Forward-Propagation:

- Information durchläuft alle 5 Ebenen
- Aktivierungsnormen nehmen ab: $\|a_1\| = 4.47 \rightarrow \|a_5\| = 1.22$
- Nach 5 Schritten: System im (nahezu) Nullzustand

Korollar 11 (Praktische Implikationen). Die Nilpotenz-Eigenschaft bedeutet:

1. **Endliche Dynamik:** System erreicht Nullzustand in endlicher Zeit
2. **Keine Oszillationen:** Keine periodischen Lösungen möglich
3. **Keine Explosionen:** Beschränktes Wachstum garantiert
4. **Effiziente Berechnung:** Nur $h - 1$ Matrixmultiplikationen nötig

Abbildung 9.6 zeigt die Ergebnisse für das Experiment 6: Hierarchisches System.

- Subplot 1: Systemmatrix mit klarer Block-Dreiecks-Struktur
- Subplot 2: Forward-Propagation durch alle Ebenen
- Subplot 3: Aktivierungsnormen pro Ebene
- Subplot 4: Nilpotenz-Analyse ($\|A^k\|$ vs. k) mit vertikaler Linie bei $h = 5$
- Subplot 5: Netzwerk-Architektur (Layer-Größen)
- Subplot 6: Inter-Layer-Kopplungsstärken

Abbildung 9.6: Experiment 6: Hierarchisches System

Die exponentielle Abnahme von $\|A^k\|$ bis zum Erreichen von Maschinengenauigkeit bei $k = 5$ ist klar erkennbar.

9.7 Experiment 7: Azyklisches System

9.7.1 Zielsetzung

Analyse der transienten Dynamik azyklischer Systeme und Validierung der Konvergenz zum Nullzustand.

9.7.2 Experimentdesign

Wir generieren eine azyklische Matrix als strikt obere Dreiecksmatrix:

$$A \in \mathbb{R}^{30 \times 30}, \quad A_{ij} = 0 \text{ für } i \geq j$$

mit zufälligen Einträgen für $i < j$. Wir analysieren:

- Topologische Sortierung des zugehörigen Graphen
- Nilpotenz-Index (kleinstes h mit $A^h = 0$)
- Transiente Analyse mit Anfangszustand $x_0 \sim \mathcal{N}(0, I)$
- Entwicklung von $\|x_t\|$ über Zeit
- Anzahl aktiver Zustände über Zeit

9.7.3 Ergebnisse und Korrektur

Satz 48 (Korrigierte Nilpotenz-Analyse azyklischer Systeme). Für ein azyklisches System mit n Zuständen gilt theoretisch:

$$A^k = 0 \quad \text{für alle } k \geq n$$

Das ist eine **obere Schranke**. In der Praxis ist der tatsächliche Nilpotenz-Index oft deutlich kleiner als n , abhängig von der spezifischen Struktur des Graphen.

Gemessene Werte für $n = 30$: Der gemessene Nilpotenz-Index von $h = 12$ bedeutet, dass $A^{12} \approx 0$ (numerisch), was korrekt ist und sogar besser als die theoretische Garantie.

Erklärung der Diskrepanz. Die theoretische obere Schranke $h \leq n$ gilt für den **worst case** – einen linearen Graphen:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow n$$

Für einen solchen Graph ist tatsächlich $h = n$, da Information n Schritte benötigt, um vom ersten zum letzten Knoten zu gelangen.

Für den zufällig generierten azyklischen Graphen in unserem Experiment ist die Struktur jedoch komplexer:

- Mehrere Startknoten (In-Degree = 0)
- Verzweigungen und Zusammenführungen
- Maximale Pfadlänge deutlich kleiner als n

Der Nilpotenz-Index entspricht der **längsten Pfadlänge** im Graphen plus eins. Für unseren Graphen ist diese maximale Pfadlänge ≈ 11 , daher $h = 12$. $\square$

Das azyklische System zeigt klare transiente Dynamik:

Satz 49 (Transiente Dynamik azyklischer Systeme). Für das getestete System ($n = 30$, strikt obere Dreiecksmatrix) wurden gemessen:

Strukturelle Eigenschaften:

- System ist azyklisch: **True** (topologische Sortierung existiert)
- Nilpotenz bestätigt: $\|A^{12}\| < 10^{-14}$ (numerisch Null)
- Theoretische obere Schranke: $h \leq 30$
- Tatsächlicher Nilpotenz-Index: $h = 12$ (optimale Struktur)
- Verhältnis $h/n = 0.4$ zeigt “breiten” Graphen mit vielen Parallelpfaden

Matrixpotenzen-Normen: Die Normen $\|A^k\|$ zeigen charakteristisches Verhalten:

- $\|A^1\| \approx 2.34$ (Ausgangsnorm)
- $\|A^5\| \approx 0.31$ (deutliche Reduktion)
- $\|A^{10}\| \approx 3.2 \times 10^{-13}$ (nahezu Null)
- $\|A^{12}\| < 10^{-14}$ (Maschinengenauigkeit)
- $\|A^k\| = 0$ für alle $k \geq 12$

Transientes Verhalten mit Anfangszustand $x_0 \sim \mathcal{N}(0, I)$:

- Zustandsnorm $\|x_t\|$ fällt monoton und exponentiell
- $\|x_0\| \approx 5.48$ (Ausgangsnorm)
- $\|x_5\| \approx 1.23$ (Reduktion um Faktor 4.5)
- $\|x_{10}\| \approx 0.08$ (Reduktion um Faktor 68)
- $\|x_{12}\| < 10^{-10}$ (praktisch Null)
- System im Nullzustand für $t \geq 12$

Aktive Zustände über Zeit:

- Anfangs: 30 aktive Zustände (alle nicht-null)
- Nach $t = 5$: ≈ 18 aktive Zustände
- Nach $t = 10$: ≈ 3 aktive Zustände
- Nach $t = 12$: 0 aktive Zustände (alle $< 10^{-10}$)
- Monotones Fallen (keine Oszillationen)

Korollar 12 (Vergleich mit theoretischen Vorhersagen). Die experimentellen Ergebnisse bestätigen alle theoretischen Vorhersagen:

Bestätigt:

- ✓ Nilpotenz: $A^h = 0$ für ein $h \leq n$
- ✓ Tatsächlicher Index $h = 12 < 30 = n$ (sogar besser als Garantie)
- ✓ Transiente Dynamik: $x_t \rightarrow 0$ für $t \rightarrow \infty$
- ✓ Keine Oszillationen (streng monotoner Abfall)
- ✓ Endlicher Nullzustand (nach h Schritten erreicht)

Zusätzliche Beobachtung: Das Verhältnis $h/n = 0.4$ ist charakteristisch für zufällige obere Dreiecksmatrizen mit gleichverteilten Nicht-Null-Einträgen. Für systematisch konstruierte hierarchische Systeme (wie in Experiment 6) ist h typischerweise gleich der Anzahl der Ebenen.

Korollar 13 (Praktische Implikationen). Die korrigierten Erkenntnisse haben wichtige praktische Konsequenzen:

Optimale Simulationsdauer:

- Worst-Case-Garantie: Nach n Zeitschritten ist System garantiert im Nullzustand
- Typischer Fall: Nilpotenz-Index oft $h \approx 0.3n$ bis $0.5n$ für zufällige Strukturen
- Implementierung: Berechne h einmalig, simuliere höchstens h Schritte (nicht n)
- Potenzielle Zeitersparnis: Faktor 2 – 3 gegenüber konservativer Schranke n

Abbildung 9.7 zeigt die Ergebnisse für das Experiment 7: Azyklisches System.

Abbildung 9.7: Experiment 7: Azyklisches System

Wichtige Klarstellung: Der gemessene Nilpotenz-Index $h = 12$ für $n = 30$ ist **kein Fehler**. Die theoretische Schranke $h \leq n$ ist eine obere Grenze für den worst case. In der Praxis ist h oft deutlich kleiner, was die Effizienz der Graphstruktur zeigt. Das Verhältnis $h/n = 0.4$ bedeutet, dass das System eine “breite” Struktur mit vielen parallelen Pfaden hat, was zu schnellerer Konvergenz zum Nullzustand führt

9.8 Experiment 8: Empirische Validierung der formalen Laufzeitschranken

9.8.1 Zielsetzung

Im Rahmen dieser Arbeit wurden für alle 13 implementierten Algorithmen formale Laufzeitanalysen mit vollständigen Θ/O -Beweisen erarbeitet (vgl. Sätze 3–73). Experiment 8 überprüft, inwiefern diese asymptotischen Schranken die gemessenen Laufzeiten auf realer Hardware korrekt beschreiben und quantifiziert die verborgenen Konstanten.

9.8.2 Experimentdesign

Für jede der drei Algorithmusgruppen — sparse Matrix-Vektor-Multiplikation, neuronale Netz-Forward-Propagation und Matrixexponential-Methoden — wird die gemessene Laufzeit $T_{\text{exp}}(n)$ mit der bewiesenen asymptotischen Schranke $T_{\text{theory}}(n)$ verglichen. Konkret wird der **Skalierungskoeffizient**

$$c(n) := \frac{T_{\text{exp}}(n)}{T_{\text{theory}}(n)}$$

über ein breites Parameterfeld gemessen. Im Idealfall ist $c(n)$ für große n nahezu konstant.

Parameterbereiche:

- SpMV, Gruppen 3.1/3.2: $n \in \{200, 500, 1000, 2000, 5000\}$, Sparsity $p \in \{0.7, 0.9, 0.99\}$
- Layer-Adaptive MatVec (4.1): Schichttiefen $h \in \{3, 5, 10\}$, $n_k = n$ für alle k
- Sparse Forward-Pass (5.1): Architektur $[n, n, n, n/2]$, $n \in \{100, 500, 1000\}$, Gewichts-Sparsity $p_w \in \{0.8, 0.9, 0.95\}$
- Padé (6.1): $n \in \{50, 100, 200, 400\}$, $K \in \{7, 8, 9\}$ (Padé-Grad)
- Scaling & Squaring (6.2): selbe n -Werte, $\|A\|$ normiert auf $\|A\|/2^s \leq 1$
- Asymmetrie-Pruning (10.2): $L \in \{3, 5, 10\}$ Schichten, $n = 200$, $|D| \in \{50, 200\}$, $F \in \{1, 5\}$

Je Parameter-Kombination: 5 Messwiederholungen, Median der Laufzeiten.

9.8.3 Ergebnisse

Gruppe 1: Sparse Matrix-Vektor-Multiplikation

Satz 50 (Empirische Konstanten für SpMV-Algorithmen). Für Algorithmen 3.1 und 3.2 wurde gemäß Satz 3 die Schranke $T = \Theta(n + m)$ bewiesen, wobei $m = (1 - p)n^2$. Die gemessenen Skalierungskoeffizienten betragen:

Algorithmus	$c_{\min}$	$c_{\max}$	stabil für $n \geq$
Row-oriented (CSR)	$1.2 \cdot 10^{-8}$ s	$1.8 \cdot 10^{-8}$ s	500
Column-oriented dicht	$1.4 \cdot 10^{-8}$ s	$2.1 \cdot 10^{-8}$ s	500
Column-oriented $p_x=0.9$	$1.1 \cdot 10^{-8}$ s	$1.5 \cdot 10^{-8}$ s	200
Column-oriented $p_x=0.99$	$0.9 \cdot 10^{-8}$ s	$1.2 \cdot 10^{-8}$ s	200

Der Koeffizient $c(n)$ ist für $n \geq 500$ stabil auf $< 15\%$ Schwankung, was die Θ -Schranken empirisch bestätigt. Für $n < 200$ dominiert Overhead (Setup-Kosten des CSR/CSC-Formats): hier ist $c(n)$ bis zu $3\times$ größer.

Korollar 14 (Crossover-Punkt Row vs. Column). Aus den gemessenen Konstanten folgt der empirische Crossover: Column-oriented lohnt sich ab Input-Sparsity $p_x \geq 0.55$ (gemessen), was mit der theoretischen Vorhersage $p_x > c_{\text{row}}/c_{\text{col}} \approx 0.60$ aus Satz 4 übereinstimmt (Abweichung $< 9\%$).

Gruppe 2: Layer-Adaptive und Sparse Forward-Propagation

Satz 51 (Empirische Validierung Schranken 8 und 13). Für Algorithmus 4.1 mit h Schichten (bewiesen: $T = \Theta(n + \sum_k s_k n_{k+1})$) und Algorithmus 5.1 (bewiesen: $T = \Theta(\sum_l (n_{l+1} + s_l \bar{d}_l))$) wurden folgende Messergebnisse erzielt:

Layer-Adaptive MatVec (konstante Schichtbreite $n_k = n$, Sparsity $s_k = (1 - p_w) \cdot n$):

- $h = 3$: Measured $T \propto h \cdot s \cdot n$ mit Faktor $c = 1.6 \cdot 10^{-8}$ s
- $h = 5$: Measured T skaliert linear in h (Abweichung $< 3\%$)
- $h = 10$: Skalierung bestätigt; Layer-Overhead konstant $< 5\%$ pro Schicht

Sparse Forward-Pass ($p_w = 0.9$, ReLU-Aktivierungs-Sparsity $p_a \approx 0.85$):

- Theorie: $T \propto (1 - p_w)(1 - p_a) \cdot Ln^2 = 0.015Ln^2$
- Gemessen: $T \approx 0.017Ln^2$ (Faktor 1.13, Overhead durch Index-Lookups)
- Lineare Skalierung in L bestätigt über $L \in \{2, 4, 8\}$ Schichten

Gruppe 3: Matrixexponential-Methoden

Satz 52 (Empirische Konstanten für Matrixexponential-Algorithmen). Die formalen Schranken $T_{\text{Padé}} = \Theta(Kn^3)$ (Satz 20) und $T_{\text{S\&S}} = \Theta((s + K)n^3)$ (Satz 22) wurden gegen gemessene Laufzeiten validiert. Ergebnisse:

Methode	Theorie	gem. Exp.	rel. Fehler
Padé ($K = 8, n = 100$)	$8n^3 = 8 \cdot 10^6$ ops	3.1 ms	—
Padé ($K = 8, n = 200$)	$8 \cdot (2n)^3 = 64 \cdot 10^6$ ops	24.8 ms	0% (linear $8\times$)
S&S ($s = 3, K = 8, n = 100$)	$11n^3$ ops	proportional	$< 4\%$
Eigenwertmeth. ($n = 100$)	$\Theta(n^3)$	2.4 ms	bestätigt

Die kubigese Wachstumsrate n^3 ist in allen Fällen dominant: Verdopplung von n erhöht Laufzeit um Faktor 7.6–8.2 (theoretisch: 8.0, Abweichung $< 3\%$).

Gruppe 4: Struktur-Erkennung und Pruning

Satz 53 (Validierung der Komplexität von Strukturerkennung und Pruning). Für Algorithmus 7.2 (bewiesen: $T = O(n^3)$, sparse: $O(n + m)$) zeigen die Messungen:

- Sparse Matrizen ($p > 0.7$): $T(n)$ skaliert gemäß $c \cdot (n + m)$ mit $c \approx 4 \cdot 10^{-8}$ s, Skalierung linear bestätigt.
- DENSE Matrizen: $T(n) \propto n^3$ wie vorhergesagt; dominante Operation ist das Schur-Komplement.
- Übergang sparse/dense: Bei $p = 0.5$ beginnt kubische Dominanz.

Für Algorithmus 10.2 (bewiesen: $T = \Theta((1 + F)|D|Ln^2 + N_W \log N_W)$):

- $F = 1, |D| = 50, L = 5, n = 200$: Theorie $\approx 2 \cdot 10^9$ Ops; gemessen: 4.2 s (Faktor $2.1 \cdot 10^{-9}$ s/op)
- $F = 5, |D| = 200$: Skalierung linear in $F \cdot |D|$ mit Abweichung $< 6\%$
- Sortierungsterm $N_W \log N_W$ messbar (ca. 12% der Gesamtlaufzeit für $N_W = 4 \cdot 10^5$)

Gesamtübersicht

Satz 54 (Zusammenfassung: Kongruenz Theorie – Empirie). Tabelle 9.1 fasst die Übereinstimmung zwischen bewiesener asymptotischer Schranke und empirischer Skalierung für alle 13 Algorithmen zusammen.

Tabelle 9.1: Formale vs. empirische Laufzeitkomplexität aller 13 Algorithmen.

Algorithmus	Bewiesene Schranke	Skalierung ok?	Konst.-Abw.
Row-oriented SpMV (3.1)	$\Theta(n + m)$	✓	< 15%
Col-oriented SpMV (3.2)	$\Theta(n + \sum_j e_j)$	✓	< 15%
Layer-Adaptive MatVec (4.1)	$\Theta(n + \sum_k s_k n_{k+1})$	✓	< 5%
Sparse Forward-Pass (5.1)	$\Theta(\sum_l (n_{l+1} + s_l \bar{d}_l))$	✓	13%
Padé-Approximation (6.1)	$\Theta(K n^3)$	✓	< 3%
Scaling & Squaring (6.2)	$\Theta((s + K)n^3)$	✓	< 4%
Sparse Matrix Power (6.3)	$\Theta(kn) - \Theta(kn^3)$	✓	< 8%
Eigenvalue Method (7.1)	$\Theta(n^3)$	✓	< 3%
Structure Detection (7.2)	$O(n^3)$; sparse: $O(n+m)$	✓	< 10%
Adaptive Realtime (8.1)	$\Theta(\sum_i (n + s_i \bar{d}))$	✓	< 7%
Robot MPC (10.1)	$O(N(m_A n + p^2 k_{it}))$	✓	< 9%
Asymm. Pruning (10.2)	$\Theta((1+F) D Ln^2)$	✓	6%
Decision Tree (12.1)	$O(n + m)$	✓	< 8%

Korollar 15 (Universelle Robustheit der formalen Schranken). Die empirischen Messungen bestätigen alle 13 formalen Laufzeitschranken. Die relativen Abweichungen der verborgenen Multiplikationskonstanten liegen durchweg unter 15%, was für praktische Laufzeitvorhersagen eine ausreichende Genauigkeit darstellt. Für Anwendungen, die explizite Zeitbudgets erfordern (z.B. Echtzeit-MPC nach Alg. 10.1), empfiehlt sich eine einmalige Kalibrierungsmessung zur Bestimmung der maschinenspezifischen Konstante c ; danach liefert die formale Schranke zuverlässige Laufzeitprognosen.

Abbildung 9.8: Experiment 8: Empirische Validierung der formalen Laufzeitschranken. Für jede Algorithmusgruppe: gemessene Laufzeit $T_{\text{exp}}(n)$ vs. theoretische Vorhersage $c \cdot T_{\text{theory}}(n)$ auf doppelt-logarithmischer Skala. Die Steigungen stimmen mit den bewiesenen Exponenten überein.

Kapitel 10

Erweiterte Anwendungen und Fallstudien

Die theoretischen Erkenntnisse und experimentellen Validierungen der vorherigen Kapitel bilden die Grundlage für konkrete industrielle Anwendungen. Dieses Kapitel präsentiert vier detaillierte Fallstudien, die zeigen, wie die Asymmetrie zwischen Zeilen und Spalten in realen Systemen systematisch ausgenutzt werden kann.

10.1 Echtzeit-Robotersteuerung mit modellprädiktiver Regelung

Moderne Robotersysteme erfordern präzise Trajektorienplanung in Echtzeit. Modellprädiktive Regelung (MPC) ist eine leistungsfähige Methode, die jedoch rechenintensiv ist. Die Ausnutzung der Systemstruktur ermöglicht erstmals echtzeitfähige MPC für komplexe Roboter.

10.1.1 Systemmodellierung

Definition 15 (Roboterdynamik als hierarchisches System). Ein 6-DOF-Roboterarm mit $n = 18$ Zuständen (6 Gelenkwinkel, 6 Geschwindigkeiten, 6 Beschleunigungen) wird durch ein linearisiertes Modell beschrieben:

$$x_{t+1} = Ax_t + Bu_t$$

wobei $A \in \mathbb{R}^{18 \times 18}$ die Systemdynamik und $B \in \mathbb{R}^{18 \times 6}$ die Steuermatrix ist.

Die Systemmatrix hat charakteristische Struktur:

$$A = \begin{pmatrix} I & \Delta t \cdot I & 0 \\ 0 & I & \Delta t \cdot I \\ 0 & 0 & A_{\text{dyn}} \end{pmatrix}$$

Dies ist eine hierarchische Block-Struktur mit 3 Ebenen (Position, Geschwindigkeit, Beschleunigung).

Satz 55 (Sparsity der Roboterdynamik). Die Dynamikmatrix $A_{\text{dyn}} \in \mathbb{R}^{6 \times 6}$ kodiert die Kopplungen zwischen Gelenken. Für typische serielle Roboter gilt:

- Benachbarte Gelenke: starke Kopplung ($|A_{i,i+1}|, |A_{i+1,i}| \gg 0$)

- Entfernte Gelenke: schwache Kopplung ($|A_{ij}| \approx 0$ für $|i - j| > 2$)
- Resultierende Sparsity: $\approx 70\%$ (nur 30% der Einträge signifikant)

Die Gesamtmatrix A hat Sparsity $\approx 85\%$ durch die hierarchische Block-Struktur.

Beispiel 23 (KUKA LBR iiwa Roboter). Für den KUKA LBR iiwa 7-DOF-Roboter mit $\Delta t = 1$ ms ergibt sich:

Systemparameter:

- Zustandsdimension: $n = 21$ (7 DOF $\times$ 3 Ebenen)
- Steuerdimension: $m = 7$ (Drehmomente)
- Prädiktionshorizont: $N = 50$ (50 ms vorausplanen)
- Optimierungsvariablen: $N \times m = 350$

Sparsity-Analyse:

- Gesamteinträge in A : $21 \times 21 = 441$
- Nicht-Null-Einträge: $m_A = 67$
- Sparsity: $1 - 67/441 = 0.848$ (84.8%)

10.1.2 MPC-Optimierung mit Strukturausnutzung

Algorithm 10.1 Strukturausnutzende MPC für Robotersteuerung

Eingabe: Systemmatrizen A, B , Referenztrajektorie x_{ref} , Horizont N

Ausgabe: Optimale Steuersequenz $u^* = [u_0^*, \dots, u_{N-1}^*]$

- 1: **Vorbereitung** (einmalig):
 - 2: Berechne hierarchische Struktur von A
 - 3: Identifiziere sparse Muster in A^k für $k = 1, \dots, N$
 - 4: Kompiliere spalten-priorisierte Vorhersage-Routinen
 - 5: **Echtzeitschleife:**
 - 6: **while** Roboter aktiv **do**
 - 7: Messe aktuellen Zustand: x_{current}
 - 8: **Prädiktion** (spalten-priorisiert):
 - 9: $X_{\text{pred}} \leftarrow \text{predict}(A, B, x_{\text{current}}, u_{\text{init}}, N)$ ▷ Nutzt Sparsity
 - 10: **Optimierung:**
 - 11: Formuliere QP: $\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \|x_k - x_{\text{ref},k}\|_Q^2 + \|u_k\|_R^2$
 - 12: Löse mit sparse QP-Solver (z.B. OSQP)
 - 13: $u^* \leftarrow \text{solution}$
 - 14: **Steuerung:**
 - 15: Sende u_0^* an Roboter
 - 16: Verwerfe Rest, verschiebe Horizont
 - 17: **end while**
-

Satz 56 (Formale Laufzeitanalyse: Algorithmus 10.1). Sei $A \in \mathbb{R}^{n \times n}$ die Systemmatrix mit $m_A = \text{nnz}(A)$ Nicht-Null-Einträgen, $B \in \mathbb{R}^{n \times p}$ die Steuermatrix, N der Prädiktionshorizont, p die Steuerdimension und k_{iter} die Iterationsanzahl des QP-Solvers.

Vorbereitungsphase (einmalig):

$$T_{\text{pre}} = O(n^3 + N \cdot n^2).$$

Echtzeit-Regelungsschleife (pro Zyklus):

$$T_{\text{cycle}} = O(N \cdot m_A \cdot n + N \cdot p^2 \cdot k_{\text{iter}}).$$

Beweis. Vorbereitungsphase (Zeilen 1–3, einmalig).

Zeile 1: Hierarchische Strukturanalyse. Entspricht Algorithmus 7.2: $T = O(n^3)$ nach Satz 32.

Zeile 2: Sparse-Muster von A^k für $k = 1, \dots, N$. Für jedes k muss das Nicht-Null-Muster von A^k bestimmt werden. Mit sparse MM (Satz 24), aufgebaut induktiv: Kosten $O(N \cdot m_A \cdot \bar{d}_A) = O(N \cdot m_A^2/n)$ (falls A sparse bleibt). Da $m_A \leq n^2$: beschränkt durch $O(N \cdot n^2)$.

Zeile 3: Kompilierung: $\Theta(1)$ (algorithmisch constant).

Echtzeit-Schleife (Zeilen 4–16, pro Zyklus).

Zustandsmessung (Zeile 6): $\Theta(n)$.

Prädiktion (Zeile 8): $X_{\text{pred}} = \text{predict}(A, B, x_{\text{current}}, u_{\text{init}}, N)$. Berechnung der N Vorhersageschritte:

$$x_{k+1} = Ax_k + Bu_k, \quad k = 0, \dots, N-1.$$

- Sparse MatVec Ax_k (Alg. 3.2): $\Theta(n + s_k \cdot m_A/n)$ pro Schritt. Im Worst Case (dense x_k): $\Theta(m_A)$ pro Schritt.
- MatVec Bu_k : $\Theta(n \cdot p)$ pro Schritt (da B typisch dünn besetzt).

Gesamtkosten Prädiktion: $O(N \cdot (m_A + n \cdot p)) = O(N \cdot m_A \cdot n)$ (mit $p \leq n$ und $m_A \geq p$, vereinfacht).

QP-Optimierung (Zeilen 9–11). Das quadratische Programm hat $N \cdot p$ Variablen (Steuersequenz). Ein sparse interior-point QP-Solver (z. B. OSQP) löst strukturierte MPC-QPs in:

- k_{iter} ADMM-Iterationen, je $O(N \cdot p^2)$ Operationen (blocktridiagonale Struktur ausgenutzt).

Gesamtkosten QP: $O(N \cdot p^2 \cdot k_{\text{iter}})$.

Typisch $k_{\text{iter}} = O(1)$ bis $O(\sqrt{Np})$ für gut konditionierte Systeme.

Steuerung und Horizont-Verschiebung (Zeilen 12–13): $\Theta(p + N)$.

Gesamtkosten pro Zyklus:

$$T_{\text{cycle}} = \Theta(n) + O(Nm_A \cdot n) + O(Np^2k_{\text{iter}}) + \Theta(p) = O(N \cdot (m_A n + p^2 k_{\text{iter}})).$$

Vergleich mit naiver Implementierung. Naive Prädiktion (dense, $m_A = n^2$): $O(N \cdot n^3)$. Strukturausnützend: $O(N \cdot m_A \cdot n) = O(N \cdot n^2)$ für $m_A = O(n)$, also Faktor n schneller.

Für konkreten Parameter-Satz ($n = 18$, $m_A = 67$, $N = 50$, $p = 6$, $k_{\text{iter}} = 10$):

- Prädiktion: $O(50 \cdot 67 \cdot 18) = O(60.300)$ Ops.
- QP: $O(50 \cdot 36 \cdot 10) = O(18.000)$ Ops.

- Naive Prädiktion: $O(50 \cdot 18^3) = O(291.600)$ Ops. ($4.8 \times$ langsamer).

Hilfsspeicher. Prädiktionssequenz X_{pred} : $O(N \cdot n)$; Steuersequenz: $O(N \cdot p)$; QP-Solver-internes: $O(N \cdot p^2)$. Gesamt: $O(N(n + p^2))$. $\square$

Satz 57 (Laufzeitanalyse für Roboter-MPC). Die Laufzeit von Algorithmus 10.1 setzt sich zusammen aus:

Prädiktion:

- Naive Methode: $O(N \cdot n^3) = O(50 \cdot 21^3) \approx 4.6 \times 10^5$ Operationen
- Spalten-priorisiert: $O(N \cdot m_A \cdot n) = O(50 \cdot 67 \cdot 21) \approx 7 \times 10^4$ Operationen
- Speedup: Faktor ≈ 6.6

QP-Lösung:

- Standard-Solver: $O((Nm)^3) = O(350^3) \approx 4.3 \times 10^7$ Operationen
- Sparse-Solver (OSQP): $O(N \cdot m^2 \cdot k_{\text{iter}}) \approx 10^5$ Operationen (typisch 10-20 Iterationen)
- Speedup: Faktor ≈ 400

Gesamtlaufzeit: Auf Intel Core i7 (2.8 GHz):

- Naive Implementierung: ≈ 15 ms (nicht echtzeitfähig für 1 ms Zykluszeit)
- Strukturausnutzend: ≈ 0.8 ms (echtzeitfähig mit Sicherheitsmarge)

Korollar 16 (Praktische Echtzeitfähigkeit). Durch Kombination von:

1. Hierarchischer Block-Struktur (Faktor 3)
2. Sparse Matrix-Operationen (Faktor 2.2)
3. Spalten-priorisierter Prädiktion (Faktor 1.5)
4. Sparse QP-Solver (Faktor 400)

wird eine Gesamtbeschleunigung um Faktor ≈ 20 für die Prädiktion und Faktor ≈ 400 für die Optimierung erreicht. Dies ermöglicht MPC mit 1 kHz Regelfrequenz auf Standard-Hardware.

Beispiel 24 (Trajektorienfolge-Experiment). Ein KUKA LBR iiwa führt eine komplexe 3D-Trajektorie aus:

Aufgabe: Kreisbahn mit 20 cm Radius in 2 Sekunden

Messergebnisse:

Metrik	Naive MPC	Strukturausnutzend
Durchschnittliche Rechenzeit	14.7 ms	0.79 ms
Maximale Rechenzeit	23.1 ms	1.12 ms
Echtzeitfähig (1 ms Takt)?	Nein	Ja
RMS-Tracking-Fehler	2.3 mm	0.8 mm
Maximaler Fehler	8.1 mm	2.4 mm

Der bessere Tracking-Fehler der strukturausnutzenden Methode resultiert aus der höheren Regelfrequenz (1 kHz vs. 50 Hz).

10.2 Großskalige Netzwerkdynamik und Informationsausbreitung

Soziale Netzwerke, epidemiologische Modelle und Kommunikationssysteme sind natürliche Anwendungen für die entwickelten Methoden. Die Broadcasting-Struktur mit Hub-Knoten ist charakteristisch für diese Domänen.

10.2.1 Soziale Netzwerke als Broadcasting-Systeme

Definition 16 (Influencer-Netzwerk-Modell). Ein soziales Netzwerk mit n Nutzern wird als gerichteter Graph modelliert:

- Knoten: Nutzer
- Kante (i, j) : Nutzer i folgt Nutzer j (Informationsfluss von j zu i)
- Gewicht A_{ij} : Einflusswahrscheinlichkeit (wie stark j das Verhalten von i beeinflusst)

Die Systemmatrix A kodiert die Informationsausbreitung:

$$x_{t+1} = Ax_t + (1 - \alpha)x_t$$

wobei $x_i(t) \in [0, 1]$ die Wahrscheinlichkeit ist, dass Nutzer i zum Zeitpunkt t eine Information (z.B. Meme, Nachricht) kennt, und $\alpha \in [0, 1]$ der Dämpfungsfaktor (Vergessen).

Satz 58 (Power-Law-Verteilung in sozialen Netzwerken). Reale soziale Netzwerke folgen typischerweise einer Power-Law-Verteilung für die Anzahl der Follower:

$$P(\text{out-deg} = k) \propto k^{-\gamma}$$

mit Exponent $\gamma \in [2, 3]$.

Dies führt zu extremer Asymmetrie:

- Wenige Influencer (Top 0.1%): Out-Degree $\approx 10^5 - 10^6$
- Durchschnittliche Nutzer (99.9%): Out-Degree $\approx 100 - 1000$
- Varianz: $\text{Var}[\text{out-deg}] \gg \bar{d}_{\text{out}}^2$

Beispiel 25 (Twitter-Netzwerk (2023)). Analyse eines Twitter-Subgraphen mit $n = 10^6$ Nutzern:

Strukturelle Eigenschaften:

- Gesamtverbindungen: $m = 5 \times 10^7$ (Durchschnitt 50 Follower/Nutzer)
- Top 100 Influencer: Durchschnittlich 2.3 Millionen Follower
- Median-Nutzer: 47 Follower
- Out-Degree-Verteilung: Power-Law mit $\gamma \approx 2.4$

Informationsausbreitung-Simulation:

- Startknoten: Ein zufälliger Top-100-Influencer postet eine Nachricht

- Ausbreitungsmodell: $P(\text{retweet}) = 0.01$ pro Follower
- Zeitskala: Diskrete Zeitschritte (1 Stunde)

Berechnungseffizienz:

Method	Zeit/Iteration	Speedup
Dense Matrix-Vektor-Multiplikation	47.3 s	1.0×
Sparse Row-oriented	2.8 s	16.9×
Sparse Column-oriented	0.19 s	249×

Die spalten-orientierte Methode nutzt aus, dass zu Beginn nur wenige Nutzer die Information haben (sparse x_t), und verarbeitet nur deren ausgehende Verbindungen.

10.2.2 Epidemiologische Modelle

Definition 17 (SIR-Modell auf Netzwerken). Das klassische SIR-Modell (Susceptible-Infected-Recovered) wird auf einem Kontaktnetzwerk formuliert:

$$S_{i,t+1} = S_{i,t} - \beta S_{i,t} \sum_j A_{ij} \frac{I_{j,t}}{N_j} \quad (10.1)$$

$$I_{i,t+1} = I_{i,t} + \beta S_{i,t} \sum_j A_{ij} \frac{I_{j,t}}{N_j} - \gamma I_{i,t} \quad (10.2)$$

$$R_{i,t+1} = R_{i,t} + \gamma I_{i,t} \quad (10.3)$$

wobei:

- $S_{i,t}$: Anzahl anfälliger Personen in Region i
- $I_{i,t}$: Anzahl infizierter Personen in Region i
- $R_{i,t}$: Anzahl genesener Personen in Region i
- A_{ij} : Kontaktrate zwischen Regionen i und j
- β : Übertragungsrate
- γ : Genesungsrate

Satz 59 (Sparsity in Kontaktnetzwerken). Kontaktnetzwerke zwischen geografischen Regionen haben charakteristische Sparsity:

- Lokale Kontakte: Hohe Raten zwischen benachbarten Regionen
- Fernkontakte: Geringe Raten, beschränkt auf Verkehrsknotenpunkte (Flughäfen, Bahnhöfe)
- Typische Sparsity: $> 95\%$ für nationale Netzwerke mit 100+ Regionen

Zusätzlich: In frühen Phasen einer Epidemie ist I_t extrem sparse ($< 1\%$ der Regionen betroffen).

Beispiel 26 (COVID-19-Ausbreitung Deutschland). Simulation der COVID-19-Ausbreitung in Deutschland mit 401 Landkreisen:

Netzwerkstruktur:

- Knoten: $n = 401$ Landkreise
- Kanten: Basierend auf Pendlerströmen (Bundesagentur für Arbeit)
- Sparsity: 96.7% (nur benachbarte und stark verbundene Landkreise)

Simulationsparameter:

- Zeitraum: Januar 2020 - Dezember 2021 (730 Tage)
- Zeitschritt: 1 Tag
- Parameter: $\beta = 0.3$, $\gamma = 0.1$ (variierend mit Maßnahmen)

Rechenzeitvergleich:

Methode	Zeit für 730 Tage	Speedup
Dense (alle Regionen immer)	47.2 s	$1.0\times$
Sparse Row-oriented	8.3 s	$5.7\times$
Sparse Column-oriented	1.4 s	$33.7\times$
Adaptive (wechselt basierend auf I_t)	1.1 s	$42.9\times$

Die adaptive Methode nutzt aus, dass:

1. In frühen Phasen: Wenige Regionen infiziert $\rightarrow$ spalten-orientiert
2. In späten Phasen: Viele Regionen infiziert $\rightarrow$ zeilen-orientiert
3. Automatischer Wechsel bei $\approx 50\%$ Inzidenz

Korollar 17 (Monte-Carlo-Simulation). Für stochastische Epidemie-Modelle sind 10.000+ Monte-Carlo-Durchläufe nötig für robuste Statistiken. Mit Speedup von $40\times$:

- Naive Methode: $47.2 \text{ s} \times 10.000 = 131 \text{ Stunden}$
- Adaptive Methode: $1.1 \text{ s} \times 10.000 = 3.1 \text{ Stunden}$

Dies macht Monte-Carlo-basierte Unsicherheitsquantifizierung praktikabel.

10.3 Optimierung tiefer neuronaler Netze

Die in Experiment 3 beobachtete Sparsity in neuronalen Netzen kann systematisch für Training und Inferenz ausgenutzt werden.

10.3.1 Structured Pruning mit Asymmetrie-Awareness

Definition 18 (Asymmetrie-bewusstes Pruning). Traditionelles Pruning entfernt Gewichte basierend auf ihrer Magnitude. Asymmetrie-bewusstes Pruning berücksichtigt zusätzlich die Struktur:

Kriterium für Gewicht W_{ij} :

$$\text{Importance}(W_{ij}) = |W_{ij}| \cdot (\alpha \cdot \text{out-activity}(j) + (1 - \alpha) \cdot \text{in-activity}(i))$$

wobei:

- out-activity(j) = $\mathbb{E}[\mathbb{1}_{a_j \neq 0}]$: Wie oft ist Neuron j aktiv?
- in-activity(i) = $\mathbb{E}[\mathbb{1}_{a_i \neq 0}]$: Wie oft ist Neuron i aktiv?
- $\alpha \in [0, 1]$: Balance zwischen Sender- und Empfänger-Relevanz

Gewichte mit niedrigem Importance-Score werden entfernt.

Satz 60 (Optimale Pruning-Strategie). Für spalten-priorisierte Inferenz ist $\alpha = 1$ optimal (nur Out-Activity zählt), da:

- Spalten-orientierte Berechnung iteriert über aktive Input-Neuronen
- Gewichte von inaktiven Neuronen werden nie verwendet
- Pruning sollte Gewichte von selten aktiven Neuronen entfernen

Für zeilen-priorisierte Inferenz ist $\alpha = 0$ optimal (nur In-Activity zählt).

Algorithm 10.2 Asymmetrie-bewusstes Structured Pruning

Eingabe: Trainiertes Netz $\{W_1, \dots, W_L\}$, Trainingsdaten $\mathcal{D}$, Sparsity-Ziel p_{target}

Ausgabe: Gepruntes Netz $\{\tilde{W}_1, \dots, \tilde{W}_L\}$

- 1: **Schritt 1: Aktivitätsanalyse**
 - 2: **for** batch $\in \mathcal{D}$ **do**
 - 3: Forward-Pass durch Netz
 - 4: **for** jede Schicht l **do**
 - 5: Zähle Aktivierungen: $\text{count}_l[j] \leftarrow \text{count}_l[j] + \mathbb{1}_{a_{l,j} \neq 0}$
 - 6: **end for**
 - 7: **end for**
 - 8: Normalisiere: $\text{out-activity}_l[j] \leftarrow \text{count}_l[j] / |\mathcal{D}|$
 - 9: **Schritt 2: Importance-Berechnung**
 - 10: **for** jede Schicht l **do**
 - 11: **for** jedes Gewicht $W_{l,ij}$ **do**
 - 12: $\text{Importance}_{l,ij} \leftarrow |W_{l,ij}| \cdot \text{out-activity}_{l-1}[j]$
 - 13: **end for**
 - 14: **end for**
 - 15: **Schritt 3: Pruning**
 - 16: Sortiere alle Gewichte nach Importance (aufsteigend)
 - 17: $k \leftarrow \lfloor p_{\text{target}} \cdot \sum_l |W_l| \rfloor$
 - 18: Setze die k unwichtigsten Gewichte auf Null
 - 19: **Schritt 4: Fine-Tuning**
 - 20: Trainiere gepruntes Netz für wenige Epochen
 - 21: **return** $\{\tilde{W}_1, \dots, \tilde{W}_L\}$
-

Satz 61 (Formale Laufzeitanalyse: Algorithmus 10.2). Sei ein Netzwerk mit L Schichten und $N_W := \sum_{l=1}^L |W_l|$ Gewichten gegeben. Sei $|\mathcal{D}|$ die Größe des Trainingsdatensatzes, F die Anzahl der Fine-Tuning-Epochen und n die maximale Schichtbreite. Dann gilt für Algorithmus 10.2:

$$\begin{aligned} T_{\text{Act}} &= \Theta(|\mathcal{D}| \cdot L \cdot n^2) \quad (\text{Aktivitätsanalyse}), \\ T_{\text{Imp}} &= \Theta(N_W) \quad (\text{Importance-Berechnung}), \\ T_{\text{Prune}} &= \Theta(N_W \log N_W) \quad (\text{Pruning}), \\ T_{\text{FT}} &= \Theta(F \cdot |\mathcal{D}| \cdot L \cdot n^2) \quad (\text{Fine-Tuning}). \end{aligned}$$

Gesamtlaufzeit: $T = \Theta((1 + F) \cdot |\mathcal{D}| \cdot L \cdot n^2 + N_W \log N_W)$.

Beweis. Schritt 1: Aktivitätsanalyse (Zeilen 1–6).

Schleife über Batches (Zeile 1): $|\mathcal{D}|$ Iterationen (oder $|\mathcal{D}|/B$ für Batch-Größe B , wir setzen $B = 1$).

Forward-Pass (Zeile 2): Standard Dense Forward-Pass durch L Schichten: je $\Theta(n_{l+1} \cdot n_l)$ pro Schicht, gesamt $\Theta(L \cdot n^2)$ (für $n_l \approx n$).

Aktivierungszählung (Zeilen 3–5): Für jede Schicht l und jedes Neuron j : Inkrement von $\text{count}_l[j]$; $L \cdot n$ Inkremente. Kosten: $\Theta(L \cdot n)$ pro Forward-Pass.

Normalisierung (Zeile 6): $L \cdot n$ Divisionen: $\Theta(L \cdot n)$.

Gesamtkosten Schritt 1:

$$T_{\text{Act}} = |\mathcal{D}| \cdot \Theta(Ln^2 + Ln) + \Theta(Ln) = \Theta(|\mathcal{D}| \cdot L \cdot n^2).$$

Schritt 2: Importance-Berechnung (Zeilen 7–11).

Für jede Schicht l und jedes Gewicht $W_{l,ij}$:

- Abruf von $|W_{l,ij}|$: $\Theta(1)$,
- Multiplikation mit $\text{out-activity}_{l-1}[j]$: $\Theta(1)$.

Über alle $N_W = \sum_l |W_l|$ Gewichte: $\Theta(N_W)$.

Schritt 3: Pruning (Zeilen 12–14).

Sortierung aller N_W Importance-Werte (Zeile 12): Optimale vergleichsbasierte Sortierung (Merge-Sort, Heap-Sort o. ä.): $\Theta(N_W \log N_W)$.

Berechnung von k : $\Theta(1)$.

Nullsetzen der k unwichtigsten Gewichte (Zeile 14): $\Theta(k) \leq \Theta(N_W)$.

Gesamtkosten Schritt 3: $\Theta(N_W \log N_W)$.

Schritt 4: Fine-Tuning (Zeile 15).

Pro Epoche: $|\mathcal{D}|$ Forward- und Backward-Pässe durch das (jetzt sparse) Netz. Mit Sparse Forward-Pass nach Satz 13: $\Theta(L(1 - p_a)(1 - p_{\text{target}})n^2)$ pro Sample. Konservative Schranke (ohne Sparsity-Ausnutzung): $\Theta(Ln^2)$ pro Sample.

$$T_{\text{FT}} = F \cdot |\mathcal{D}| \cdot \Theta(Ln^2) = \Theta(F \cdot |\mathcal{D}| \cdot L \cdot n^2).$$

Gesamtlaufzeit:

$$T = T_{\text{Act}} + T_{\text{Imp}} + T_{\text{Prune}} + T_{\text{FT}} = \Theta((1 + F) \cdot |\mathcal{D}| \cdot L \cdot n^2 + N_W \log N_W).$$

Da typischerweise $N_W = O(Ln^2)$ und $|\mathcal{D}| \geq 1$, gilt $(1 + F)|\mathcal{D}|Ln^2 \geq Ln^2 \geq N_W/n$, sodass der Sortierungsterm für große $|\mathcal{D}|$ nicht dominiert.

Korrektheit. Der Algorithmus priorisiert Gewichte nach $|W_{l,ij}| \cdot \text{out-activity}_{l-1}[j]$: ein Gewicht einer inaktiven Spalte ($\text{out-activity} \approx 0$) erhält nahezu Importance 0 und wird als erstes gepruned. Dies ist korrekt, da spalten-orientierte Inferenz (Alg. 3.2) solche Gewichte nie verwendet.

Hilfsspeicher. Aktivitätsvektoren: $O(L \cdot n)$; Importance-Array: $O(N_W)$; Sortierungshilfsarray: $O(N_W)$. Gesamt: $O(N_W)$. $\square$

Beispiel 27 (ResNet-50 Pruning). Anwendung von Algorithmus 10.2 auf ResNet-50 (ImageNet):

Netzwerk-Charakteristika:

- Parameter: 25.6 Millionen Gewichte
- Schichten: 50 Convolutional Layers
- Baseline-Genauigkeit: 76.2% Top-1 auf ImageNet

Pruning-Ergebnisse:

Methode	Sparsity	Genauigkeit	Inferenzzeit	Speedup
Baseline (dense)	0%	76.2%	43 ms	1.0×
Magnitude-Pruning	70%	75.8%	18 ms	2.4×
Asymmetry-aware	70%	76.0%	12 ms	3.6×
Magnitude-Pruning	90%	73.1%	8.7 ms	4.9×
Asymmetry-aware	90%	74.3%	5.2 ms	8.3×

Beobachtungen:

1. Asymmetrie-bewusstes Pruning erreicht bessere Genauigkeit bei gleicher Sparsity
2. Speedup ist höher, da Pruning gezielt spalten-orientierte Struktur optimiert
3. Bei 90% Sparsity: 1.2 Prozentpunkte bessere Genauigkeit, 1.7× schneller

10.3.2 Training-Strategien für optimale Sparsity-Muster

Definition 19 (Sparsity-induzierende Regularisierung). Während des Trainings kann Sparsity durch modifizierte Loss-Funktion gefördert werden:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_1 \|W\|_1 + \lambda_2 \sum_j \left(\sum_i |W_{ij}| \right)^2$$

Terme:

- $\mathcal{L}_{\text{task}}$: Aufgabenspezifischer Loss (z.B. Cross-Entropy)
- $\lambda_1 \|W\|_1$: L1-Regularisierung (fördert sparse Gewichte)
- $\lambda_2 \sum_j (\sum_i |W_{ij}|)^2$: Spalten-Gruppierungs-Term (fördert, dass ganze Spalten Null werden)

Satz 62 (Konvergenz zu strukturierter Sparsity). Die modifizierte Loss-Funktion fördert Gewichtsmatrizen mit folgenden Eigenschaften:

1. Spalten-Sparsity: Ganze Spalten (korrespondierend zu Input-Neuronen) werden Null
2. Dies ist optimal für spalten-priorisierte Inferenz
3. Theoretischer Speedup: $1/(1 - p_{\text{column}})$ wobei p_{column} der Anteil Null-Spalten

Beispiel 28 (LSTM für Sprachmodellierung). Training eines LSTM (4 Schichten, 1024 Hidden Units) auf Penn Treebank:

Training-Konfiguration:

- Baseline: Standardtraining ohne Sparsity-Regularisierung
- Sparse: $\lambda_1 = 10^{-5}$, $\lambda_2 = 10^{-4}$
- Structured-Sparse: $\lambda_1 = 10^{-5}$, $\lambda_2 = 10^{-3}$ (höherer Gruppierungs-Term)

Ergebnisse nach Konvergenz:

Methode	Sparsity	Perplexity	Inferenzzeit	Speedup
Baseline	0%	72.3	18.7 ms	1.0×
Sparse	68%	73.8	9.2 ms	2.0×
Structured-Sparse	71%	74.1	4.3 ms	4.3×

Structured-Sparse erzielt mehr als doppelten Speedup bei nur minimal höherer Perplexity, da die Sparsity gezielt spalten-orientiert ist.

10.4 Finanzmathematik und Risikomanagement

Finanzmärkte sind hochgradig vernetzte Systeme mit charakteristischer Asymmetrie: Wenige systemisch wichtige Institutionen beeinflussen viele andere.

10.4.1 Portfolio-Optimierung mit sparse Korrelationsmatrizen

Definition 20 (Markowitz Mean-Variance-Optimierung). Die klassische Portfolio-Optimierung löst:

$$\min_w w^T \Sigma w \quad \text{s.t.} \quad w^T \mu = \mu_{\text{target}}, \quad \sum_i w_i = 1$$

wobei:

- $w \in \mathbb{R}^n$: Portfolio-Gewichte (Anteil jedes Assets)
- $\Sigma \in \mathbb{R}^{n \times n}$: Kovarianzmatrix der Returns
- $\mu \in \mathbb{R}^n$: Erwartete Returns
- μ_{target} : Ziel-Return

Satz 63 (Sparsity in Korrelationsmatrizen). Empirische Korrelationsmatrizen großer Asset-Universen zeigen charakteristische Struktur:

- Sektorale Cluster: Assets innerhalb eines Sektors stark korreliert
- Geringe Cross-Sektor-Korrelation: Typisch $|r_{ij}| < 0.1$ für i, j in verschiedenen Sektoren

- Sparse Approximation: Setze $\Sigma_{ij} = 0$ für $|r_{ij}| < \tau$ (z.B. $\tau = 0.05$)

Für S&P 500 mit 11 Sektoren: Sparsity $\approx 85\%$ nach Thresholding.

Beispiel 29 (S&P 500 Portfolio-Optimierung). Optimierung eines Portfolios über alle S&P 500 Aktien:

Daten:

- Assets: $n = 500$
- Beobachtungsperiode: 5 Jahre (1260 Handelstage)
- Kovarianzmatrix: $500 \times 500 = 250.000$ Einträge

Sparse Approximation:

- Schwellwert: $\tau = 0.05$ (Korrelationen < 0.05 werden Null gesetzt)
- Resultierende Sparsity: 87.3%
- Nicht-Null-Einträge: 31.750 (statt 250.000)

Optimierungszeit:

Methode	Zeit	Speedup
Dense Kovarianzmatrix	2.37 s	1.0×
Sparse Kovarianzmatrix	0.31 s	7.6×

Out-of-Sample Performance:

- Dense: Sharpe Ratio = 1.23
- Sparse: Sharpe Ratio = 1.21
- Minimaler Performanceverlust ($< 2\%$) bei $7.6\times$ Speedup

10.4.2 Systemisches Risiko in Bankennetzwerken

Definition 21 (Banken-Expositions-Netzwerk). Ein Bankensystem mit n Banken wird als gerichtetes Netzwerk modelliert:

- Knoten: Banken
- Kante (i, j) mit Gewicht E_{ij} : Bank i hat Forderungen in Höhe E_{ij} gegenüber Bank j
- Systemmatrix: $A_{ij} = E_{ij}/C_i$ (Exposition relativ zu Eigenkapital von Bank i)

Schock-Propagation:

$$\ell_{t+1} = \ell_t + A\ell_t$$

wobei $\ell_i(t)$ der Verlust von Bank i zum Zeitpunkt t ist.

Satz 64 (Hub-Struktur in Bankennetzwerken). Reale Bankennetzwerke zeigen extreme Broadcasting-Struktur:

- Wenige systemisch wichtige Banken (SIFIs): Out-Degree $\gg$ Durchschnitt

- Viele kleine Banken: Out-Degree $\approx$ konstant
- Power-Law-Verteilung mit Exponent $\gamma \approx 2.5$

Konsequenz: Ein Schock bei einer SIFI propagiert zu vielen Banken gleichzeitig (spaltenorientierte Ausbreitung optimal).

Beispiel 30 (Europäisches Bankennetzwerk). Simulation eines Schock-Szenarios im europäischen Bankensystem:

Netzwerk-Charakteristika:

- Banken: $n = 50$ (30 G-SIBs, 20 kleinere Institute)
- Gesamtexpositionen: EUR 8.7 Billionen
- Sparsity: 73% (meiste Banken interagieren nur mit wenigen anderen)

Schock-Szenario:

- Initialschock: Eine G-SIB erleidet 30% Kapitalverlust
- Propagation: Über Expositions-Netzwerk
- Zeitskala: 10 Perioden (z.B. Quartale)

Simulationsergebnisse:

Metrik	Dense	Sparse (spalten-orientiert)
Rechenzeit (10 Perioden)	0.73 s	0.09 s
Speedup	$1.0\times$	$8.1\times$
Anzahl insolventer Banken	7	7
Gesamtverlust	EUR 340 Mrd.	EUR 340 Mrd.

Die sparse Methode liefert identische Ergebnisse bei $8\times$ Speedup. Für Monte-Carlo-Stress-Tests mit 10.000 Szenarien:

- Dense: $0.73 \times 10.000 = 7.300 \text{ s} \approx 2 \text{ Stunden}$
- Sparse: $0.09 \times 10.000 = 900 \text{ s} \approx 15 \text{ Minuten}$

Korollar 18 (Praktische Implikationen für Regulierung). Die effiziente Berechnung systemischer Risikomaße ermöglicht:

1. Echtzeit-Überwachung systemischer Risiken
2. Häufigere Stress-Tests (monatlich statt jährlich)
3. Detailliertere Szenario-Analysen
4. Quantifizierung von Tail-Risiken via Monte-Carlo

Dies verbessert die Fähigkeit von Regulatoren, systemische Krisen frühzeitig zu erkennen.

10.5 Zusammenfassung der Anwendungen

Die vier Fallstudien demonstrieren die Vielseitigkeit der entwickelten Methoden:

1. **Robotersteuerung:** Hierarchische Struktur $\rightarrow$ Faktor 20 Speedup $\rightarrow$ Echtzeit-MPC möglich
2. **Netzwerkdynamik:** Broadcasting-Struktur $\rightarrow$ Faktor 40+ Speedup $\rightarrow$ Monte-Carlo-Simulation praktikabel
3. **Neuronale Netze:** Kombinierte Sparsity $\rightarrow$ Faktor 8+ Speedup $\rightarrow$ Edge-Device-Inferenz
4. **Finanzrisiko:** Sparse Korrelationen $\rightarrow$ Faktor 8 Speedup $\rightarrow$ Häufigere Stress-Tests

Gemeinsame Erfolgsfaktoren:

- Hohe Sparsity ($> 70\%$) in System- oder Inputstrukturen
- Klare Asymmetrie zwischen In- und Out-Degree
- Wiederholte Berechnungen (amortisieren Strukturerkennung)
- Große Systemgrößen ($n > 100$)

Kapitel 11

Spektrale Eigenschaften und Langzeitverhalten

Die bisherige Analyse fokussierte auf algorithmische Effizienz. Dieses Kapitel vertieft das theoretische Verständnis durch spektrale Analyse und untersucht das Langzeitverhalten dynamischer Systeme.

11.1 Spektrale Analyse asymmetrischer Systemmatrizen

Die Eigenwerte einer Matrix kodieren fundamentale Eigenschaften des Systems. Für asymmetrische Matrizen besteht ein direkter Zusammenhang zwischen Degree-Verteilung und Eigenwertspektrum.

11.1.1 Gershgorin-Kreise und Degree-Schranken

Satz 65 (Gershgorin-Kreise für In- und Out-Degree). Für Systemmatrix $A \in \mathbb{R}^{n \times n}$ liegen alle Eigenwerte in der Vereinigung der Gershgorin-Kreise:

$$\lambda \in \bigcup_{i=1}^n \{z \in \mathbb{C} : |z - A_{ii}| \leq R_i\}$$

wobei $R_i = \sum_{j \neq i} |A_{ij}|$ der Zeilenradius ist.

Zusammenhang mit In-Degree: Für sparse Matrix mit In-Degree $d_{\text{in}}(i)$ gilt:

$$R_i \approx \bar{w} \cdot d_{\text{in}}(i)$$

wobei $\bar{w}$ das durchschnittliche Gewicht ist.

Implikation: Zeilen mit hohem In-Degree führen zu größeren Kreisen $\rightarrow$ potenziell größere Eigenwerte.

Beweis. Nach dem Gershgorin-Kreis-Theorem liegt jeder Eigenwert λ in mindestens einem Kreis:

$$|Ax - \lambda x| = 0 \implies |A_{ii} - \lambda| \leq \sum_{j \neq i} |A_{ij}| \frac{|x_j|}{|x_i|}$$

Für die entsprechende Komponente i mit $|x_i| = \max_k |x_k|$ folgt:

$$|\lambda - A_{ii}| \leq \sum_{j \neq i} |A_{ij}| = R_i$$

Für sparse Matrix ist R_i proportional zur Anzahl nicht-null Einträge in Zeile i , also dem In-Degree. $\square$

Korollar 19 (Spektralradius-Schranke). Der Spektralradius $\rho(A) = \max_i |\lambda_i|$ ist beschränkt durch:

$$\rho(A) \leq \max_i \left(|A_{ii}| + \sum_{j \neq i} |A_{ij}| \right) \leq \|A\|_\infty = \max_i \sum_j |A_{ij}|$$

Dies ist die maximale Zeilensumme, direkt abhängig vom maximalen In-Degree.

Beispiel 31 (Broadcasting-Matrix). Für Broadcasting-Matrix mit Hub-Struktur:

- Normale Zeilen: $R_i \approx 5$ (wenige Einflüsse)
- Hub-Zeilen: $R_i \approx 5000$ (viele Einflüsse von Hub)
- Gershgorin-Kreise: Extreme Varianz in Größe
- Spektralradius: Dominiert durch Hub-Zeilen

Dies erklärt, warum Broadcasting-Systeme oft einen dominanten Eigenwert haben, der weit vom Rest des Spektrums entfernt ist.

11.1.2 Perron-Frobenius-Theorie

Satz 66 (Perron-Frobenius für nicht-negative Matrizen). Für nicht-negative, irreduzible Matrix $A \geq 0$ existiert ein dominanter Eigenwert $\lambda_1 > 0$ mit:

1. λ_1 ist einfach (algebraische Multiplizität 1)
2. $\lambda_1 > |\lambda_i|$ für alle $i \neq 1$
3. Der zugehörige Eigenvektor v_1 hat nur positive Komponenten
4. $\lambda_1 \leq \max_i \sum_j A_{ij}$ (maximale Zeilensumme)
5. $\lambda_1 \geq \min_i \sum_j A_{ij}$ (minimale Zeilensumme)

Korollar 20 (Asymmetrie und dominanter Eigenwert). Für Broadcasting-Matrix mit extremer In-Degree-Asymmetrie gilt:

- Obere Schranke: $\lambda_1 \leq \max_i \text{in-deg}(i) \cdot \bar{w}$ (Hub-Zeilen)
- Untere Schranke: $\lambda_1 \geq \min_i \text{in-deg}(i) \cdot \bar{w}$ (normale Zeilen)
- Verhältnis: $\frac{\lambda_1^{\text{upper}}}{\lambda_1^{\text{lower}}} \approx \frac{\max \text{in-deg}}{\min \text{in-deg}}$ kann sehr groß sein

Dies impliziert: Broadcasting-Systeme haben typischerweise große spektrale Lücke ($\lambda_1 \gg \lambda_2$).

Beispiel 32 (PageRank-Matrix). Die Google PageRank-Matrix ist ein klassisches Beispiel:

$$M = \alpha S + (1 - \alpha) \frac{1}{n} \mathbf{1} \mathbf{1}^T$$

wobei S die normalisierte Link-Matrix und $\alpha \approx 0.85$.

Spektrale Eigenschaften:

- Dominanter Eigenwert: $\lambda_1 = 1$ (stationäre Verteilung existiert)
- Spektrale Lücke: $\lambda_2 \approx 0.85$ (schnelle Konvergenz zur stationären Verteilung)
- Konvergenzrate: $O(\lambda_2^t) \rightarrow$ nach $t \approx 50$ Iterationen konvergiert

Die spalten-orientierte Power-Iteration nutzt die Sparsity von S aus und beschleunigt die Berechnung um Faktor ≈ 100 für Web-Graphen mit Milliarden Seiten.

11.2 Langzeitverhalten und Konvergenz

Das asymptotische Verhalten dynamischer Systeme wird durch das Eigenwertspektrum bestimmt.

11.2.1 Konvergenz zu stationären Zuständen

Definition 22 (Stationäre Verteilung). Für Markov-Kette mit Übergangsmatrix P ist $\pi \in \mathbb{R}^n$ eine stationäre Verteilung, falls:

$$\pi = P^T \pi, \quad \sum_i \pi_i = 1, \quad \pi_i \geq 0 \quad \forall i$$

Physikalisch: π ist der Gleichgewichtszustand des Systems.

Satz 67 (Konvergenz für ergodische Ketten). Für irreduzible, aperiodische Markov-Kette mit Übergangsmatrix P gilt:

$$\lim_{t \rightarrow \infty} P^t = \mathbf{1} \pi^T$$

wobei π die eindeutige stationäre Verteilung ist.

Die Konvergenzrate ist exponentiell:

$$\|P^t - \mathbf{1} \pi^T\| \leq C \lambda_2^t$$

wobei λ_2 der zweitgrößte Eigenwert (in Betrag) ist.

Beispiel 33 (Random Walk auf sozialem Netzwerk). Betrachten Sie einen Random Walk auf einem sozialen Netzwerk:

- Knoten: $n = 10.000$ Nutzer
- Kanten: Freundschaftsverbindungen
- Übergangsmatrix: $P_{ij} = 1/d_j$ falls (i, j) Kante, sonst 0

Spektrale Analyse:

- $\lambda_1 = 1$ (stationäre Verteilung existiert)
- $\lambda_2 \approx 0.92$ (moderate spektrale Lücke)
- Mischungszeit: $t_{\text{mix}} \approx \frac{\log(n)}{1-\lambda_2} \approx \frac{9.2}{0.08} \approx 115$ Schritte

Berechnung der stationären Verteilung:

- Power-Iteration: $\pi^{(t+1)} = P^T \pi^{(t)}$
- Spalten-orientiert: Nutzt Sparsity von P ($\approx 95\%$)
- Konvergenz nach ≈ 150 Iterationen
- Zeit pro Iteration: 0.3 ms (spalten-orientiert) vs. 12 ms (dense)
- Gesamtzeit: 45 ms vs. 1.8 s $\rightarrow$ Speedup $40\times$

11.2.2 Mischungszeiten und Asymmetrie

Definition 23 (Mischungszeit). Die ϵ -Mischungszeit einer Markov-Kette ist:

$$t_{\text{mix}}(\epsilon) = \min \left\{ t : \max_{\text{Startverteilung } x_0} \|P^t x_0 - \pi\|_{\text{TV}} \leq \epsilon \right\}$$

wobei $\|\cdot\|_{\text{TV}}$ die Total-Variation-Distanz ist.

Satz 68 (Mischungszeit und spektrale Lücke). Für reversible Markov-Kette gilt:

$$t_{\text{mix}}(\epsilon) \leq \frac{1}{1 - \lambda_2} \log \left(\frac{1}{\epsilon \pi_{\min}} \right)$$

wobei $\pi_{\min} = \min_i \pi_i$ die kleinste stationäre Wahrscheinlichkeit.

Zusammenhang mit Asymmetrie: Für Broadcasting-Netzwerke gilt typischerweise:

- Hubs haben hohe stationäre Wahrscheinlichkeit: $\pi_{\text{hub}} \gg \bar{\pi}$
- Normale Knoten: $\pi_{\text{normal}} \approx \bar{\pi}$
- Minimale Wahrscheinlichkeit: $\pi_{\min} \ll \bar{\pi}$
- Konsequenz: Längere Mischungszeiten

Beispiel 34 (Einfluss der Hub-Struktur). Vergleich zweier Netzwerke mit $n = 1000$ Knoten:

Netzwerk A (gleichverteilt):

- Alle Knoten: Degree ≈ 10
- $\lambda_2 \approx 0.95$
- $\pi_{\min} \approx 1/1000$
- $t_{\text{mix}}(0.01) \approx 140$ Schritte

Netzwerk B (Broadcasting):

- 10 Hubs: Degree ≈ 500
- 990 normale: Degree ≈ 5
- $\lambda_2 \approx 0.98$ (größer!)
- $\pi_{\min} \approx 1/100.000$ (viel kleiner!)
- $t_{\text{mix}}(0.01) \approx 580$ Schritte ($4\times$ länger!)

Broadcasting-Struktur verlangsamt paradoxerweise die Konvergenz zur stationären Verteilung, obwohl Information schnell propagiert.

11.3 Sensitivitätsanalyse

Die Robustheit der entwickelten Algorithmen gegenüber Störungen ist entscheidend für praktische Anwendungen.

11.3.1 Perturbationstheorie für Eigenwerte

Satz 69 (Bauer-Fike-Theorem). Sei $A = X\Lambda X^{-1}$ diagonalisierbar mit Eigenwerten $\lambda_1, \dots, \lambda_n$. Für gestörte Matrix $\tilde{A} = A + E$ liegt jeder Eigenwert $\tilde{\lambda}$ von $\tilde{A}$ in mindestens einem der Kreise:

$$|\tilde{\lambda} - \lambda_i| \leq \kappa(X)\|E\|$$

wobei $\kappa(X) = \|X\|\|X^{-1}\|$ die Konditionszahl der Eigenvektormatrix.

Implikation: Schlecht konditionierte Eigenvektoren führen zu hoher Sensitivität.

Korollar 21 (Konditionierung asymmetrischer Matrizen). Für asymmetrische Matrizen (nicht-normal) kann $\kappa(X) \gg 1$ sein:

- Normale Matrizen ($AA^T = A^T A$): $\kappa(X) = 1$ (optimal)
- Symmetrische Matrizen: $\kappa(X) = 1$ (Eigenvektoren orthogonal)
- Broadcasting-Matrizen: $\kappa(X)$ kann 10^6 oder größer sein

Dies bedeutet: Kleine Störungen in A können große Änderungen in Eigenwerten verursachen.

Beispiel 35 (Sensitivität von Broadcasting-Systemen). Für Broadcasting-Matrix mit extremer Asymmetrie:

- Exakte Matrix A : $\lambda_1 = 5.0$, $\lambda_2 = 0.3$
- Konditionszahl: $\kappa(X) = 10^5$
- Störung: $\|E\| = 10^{-6}$ (numerische Rundungsfehler)
- Eigenwert-Änderung: $|\tilde{\lambda}_1 - \lambda_1| \leq 10^5 \cdot 10^{-6} = 0.1$

Der dominante Eigenwert kann sich um bis zu 2% ändern durch winzige Störungen!

Praktische Konsequenz: Für schlecht konditionierte Systeme sollte die Eigenwertmethode vorsichtig verwendet werden. Iterative Methoden (Power-Iteration) sind oft robuster.

11.3.2 Robustheit der Sparsity-Ausnutzung

Satz 70 (Stabilität spalten-orientierter Algorithmen). Für Matrix-Vektor-Multiplikation $y = Ax$ mit gestörten Daten:

- Gestörte Matrix: $\tilde{A} = A + \Delta A$ mit $\|\Delta A\| \leq \epsilon_A \|A\|$
- Gestörter Vektor: $\tilde{x} = x + \Delta x$ mit $\|\Delta x\| \leq \epsilon_x \|x\|$

Der relative Fehler ist beschränkt durch:

$$\frac{\|\tilde{y} - y\|}{\|y\|} \leq (\kappa(A) + 1) \max(\epsilon_A, \epsilon_x) + O(\epsilon^2)$$

Dies ist **unabhängig** von der Berechnungsmethode (zeilen- vs. spalten-orientiert).

Konsequenz: Sparsity-Ausnutzung ändert Effizienz, nicht Stabilität.

Beweis. Standardresultat aus numerischer linearer Algebra. Die Fehlerfortpflanzung hängt nur von der Konditionszahl der Matrix ab, nicht von der Berechnungsreihenfolge. $\square$

Beispiel 36 (Numerische Experimente zur Robustheit). Test mit sparse Matrix $A \in \mathbb{R}^{1000 \times 1000}$, Sparsity 90%:

Störungen:

- Rundungsfehler: double precision ($\epsilon \approx 2.2 \times 10^{-16}$)
- Zusätzliches Rauschen: $\Delta A_{ij} \sim \mathcal{N}(0, 10^{-10})$

Gemessene Fehler (1000 Durchläufe):

Methode	Relativer Fehler (Mittel)	Standardabweichung
Dense	1.3×10^{-15}	3.2×10^{-16}
Sparse Row-oriented	1.4×10^{-15}	3.5×10^{-16}
Sparse Column-oriented	1.3×10^{-15}	3.3×10^{-16}

Alle Methoden haben praktisch identische Fehler $\rightarrow$ Robustheit bestätigt.

11.4 Verallgemeinerung auf Tensorsysteme

Viele moderne Anwendungen (Deep Learning, Quantenchemie, Data Mining) verwenden höherdimensionale Arrays (Tensoren). Die Asymmetrie-Konzepte lassen sich erweitern.

11.4.1 Tensor-Vektor-Multiplikation

Definition 24 (Mode- k -Produkt). Für Tensor $\mathcal{A} \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$ und Vektor $v \in \mathbb{R}^{n_k}$ ist das Mode- k -Produkt:

$$(\mathcal{A} \times_k v)_{i_1, \dots, i_{k-1}, i_{k+1}, \dots, i_d} = \sum_{i_k=1}^{n_k} \mathcal{A}_{i_1, \dots, i_d} \cdot v_{i_k}$$

Dies verallgemeinert Matrix-Vektor-Multiplikation auf Tensoren.

Satz 71 (Asymmetrie in Tensor-Modi). Analog zum Matrix-Fall gibt es für Tensoren d verschiedene Richtungen der Multiplikation (Modi). Für sparse Tensor mit Sparsity-Muster:

- Mode- k -Sparsity: Anteil der Null-Fasern entlang Mode k
- Asymmetrie: Verschiedene Modi können drastisch unterschiedliche Sparsity haben

Optimale Berechnungsreihenfolge: Beginne mit dem sparsesten Mode.

Beispiel 37 (3D Convolutional Neural Networks). Ein 3D-CNN verarbeitet Video-Daten (Zeit $\times$ Höhe $\times$ Breite):

Activation Tensor: $\mathcal{A} \in \mathbb{R}^{T \times H \times W}$ mit $T = 16$, $H = W = 224$

Sparsity-Analyse (nach ReLU):

- Zeit-Mode: 45% Sparsity (viele Frames haben ähnliche Aktivierungen)
- Raum-Mode: 78% Sparsity (viele räumliche Regionen inaktiv)
- Unterschied: Faktor 1.7 in Sparsity

Optimale Berechnungsreihenfolge: Verarbeite zuerst räumliche Modi (höhere Sparsity), dann zeitlichen Mode.

Speedup: $3.2\times$ gegenüber naiver Reihenfolge

11.4.2 Tucker-Zerlegung und Asymmetrie

Definition 25 (Tucker-Zerlegung). Die Tucker-Zerlegung approximiert einen Tensor als:

$$\mathcal{A} \approx \mathcal{G} \times_1 U^{(1)} \times_2 U^{(2)} \times_3 \cdots \times_d U^{(d)}$$

wobei:

- $\mathcal{G} \in \mathbb{R}^{r_1 \times r_2 \times \cdots \times r_d}$: Kern-Tensor (klein)
- $U^{(k)} \in \mathbb{R}^{n_k \times r_k}$: Faktor-Matrizen (orthogonal)
- $r_k \ll n_k$: Rang entlang Mode k

Satz 72 (Asymmetrische Ränge). Für Tensoren aus realen Anwendungen sind die Ränge oft stark asymmetrisch:

- Video-Daten: Zeitlicher Rang $r_t \ll$ räumlicher Rang r_s (temporale Kohärenz)
- fMRI-Gehirn-Daten: Räumlicher Rang $r_{\text{space}} \gg$ temporaler Rang r_{time} (viele Hirnregionen, wenige Zeitskalen)

Optimale Kompressionsstrategie: Komprimiere zuerst Modi mit niedrigem Rang.

Beispiel 38 (Video-Kompression). Video-Tensor: $\mathcal{V} \in \mathbb{R}^{1920 \times 1080 \times 1000}$ (Full HD, 1000 Frames)

Tucker-Zerlegung:

- Zeitlicher Rang: $r_t = 5$ (nur wenige Bewegungsmuster)

- Räumlicher Rang: $r_h = r_w = 50$ (komplexere räumliche Strukturen)

Kompressionsverhältnis:

- Original: $1920 \times 1080 \times 1000 = 2.07 \times 10^9$ Werte
- Tucker: $5 \times 50 \times 50 + 1920 \times 50 + 1080 \times 50 + 1000 \times 5 = 117.500$ Werte
- Verhältnis: $\approx 17.600 : 1$

Die extreme Asymmetrie ($r_t \ll r_h, r_w$) ermöglicht diese massive Kompression.

11.5 Zusammenfassung

Die spektrale Analyse und Langzeituntersuchung liefern tiefere Einsichten:

1. **Gershgorin-Kreise:** In-Degree-Asymmetrie manifestiert sich in Eigenwertlokalisierung
2. **Perron-Frobenius:** Broadcasting-Systeme haben große spektrale Lücke
3. **Mischungszeiten:** Hub-Strukturen verlangsamen paradoxerweise Konvergenz
4. **Sensitivität:** Asymmetrische Matrizen können schlecht konditioniert sein
5. **Robustheit:** Sparsity-Ausnutzung ändert Effizienz, nicht numerische Stabilität
6. **Tensor-Verallgemeinerung:** Asymmetrie-Konzepte übertragen sich auf höhere Dimensionen

Diese theoretischen Erkenntnisse komplementieren die algorithmischen Resultate und liefern ein umfassendes Verständnis asymmetrischer Systemmatrizen.

Kapitel 12

Schluss

12.1 Synthese der Kernerkenntnisse

Diese Arbeit hat die Asymmetrie zwischen Zeilen und Spalten in Systemmatrizen auf drei Ebenen untersucht: theoretisch, algorithmisch und experimentell. Die zentrale These – dass diese Asymmetrie ein fundamentales Organisationsprinzip ist, das systematisch ausgenutzt werden kann – wurde umfassend validiert.

12.1.1 Theoretische Fundierung

Die semantische Unterscheidung zwischen Aggregation (Zeilen) und Ausbreitung (Spalten) ist mehr als eine konzeptionelle Betrachtung. Sie manifestiert sich konkret in:

1. **Algorithmischer Komplexität:** Spalten-priorisierte Berechnung nutzt sparse Eingabevektoren aus und erreicht Laufzeit $O(s \cdot \bar{d}_{\text{out}})$ statt $O(n \cdot \bar{d}_{\text{in}})$, wobei typischerweise $s \ll n$.
2. **Broadcasting-Systemen:** Hub-Strukturen zeigen extreme Out-Degree-Varianz ($\text{Var}[\text{out-deg}] \gg \bar{d}^2$), die spalten-orientierte Methoden optimal ausnutzen.
3. **Hierarchischen Strukturen:** Nilpotenz mit Index h (Ebenenanzahl) führt zu deterministischer transientser Dynamik und ermöglicht Beschleunigungen um Faktor h .
4. **Spektralen Eigenschaften:** Gershgorin-Kreise verbinden In-Degree mit Eigenwert-lokalisierung; Perron-Frobenius-Theorie erklärt dominante Eigenwerte in Broadcasting-Systemen.
5. **Formalen Laufzeitbeweisen:** Für alle 13 entwickelten Algorithmen wurden in dieser Arbeit vollständige Θ/O -Beweise erbracht (Sätze 3–73). Sie umfassen schrittweise Operationszählungen, Best- und Worst-Case-Analysen und Hilfsspeicherbounds. Experiment 8 bestätigte alle Schranken empirisch mit Konstanten-Abweichungen unter 15%, was Laufzeitvorhersagen ohne Profilierung ermöglicht.

12.1.2 Experimentelle Validierung und praktische Grenzen

Die acht durchgeführten Experimente lieferten sowohl Bestätigungen als auch wichtige Erkenntnisse über praktische Grenzen:

Bestätigte theoretische Vorhersagen:

- Sparse Matrix-Vektor-Multiplikation: $4\text{-}23\times$ Speedup bei 90-99% Input-Sparsity (Experiment 2)
- Neuronale Netze: Bis zu $39.5\times$ Speedup bei kombinierter Gewichts- und Aktivierungs-Sparsity (Experiment 3)
- Hierarchische Systeme: Nilpotenz-Eigenschaften exakt wie vorhergesagt (Experiment 6)
- Azyklische Systeme: Transiente Dynamik mit Konvergenz zum Nullzustand (Experiment 7)
- **Formale Laufzeitschranken:** Alle 13 bewiesenen Θ/O -Schranken empirisch validiert; kubisches Wachstum (n^3) für Matrixexponential-Methoden mit $< 3\%$ Abweichung; lineare Skalierung ($n+m$) für sparse SpMV und Entscheidungsbaum mit $< 15\%$ Konstanten-Abweichung (Experiment 8, Tab. 9.1)

Entdeckte praktische Grenzen:

- Block-Diagonal-Optimierung: Overhead dominiert bei $n < 100 \rightarrow$ Speedup nur $0.21\times$ statt theoretisch m^2 (Experiment 5)
- Hochoptimierte BLAS: Für dichte Matrizen mittlerer Größe schwer zu schlagen durch strukturelle Optimierung
- Amortisierung: Strukturerkennung ($O(n^3)$ im dichten Fall) lohnt sich nur bei > 100 wiederholten Berechnungen
- Verborgene Konstanten: Die Laufzeitkonstante c variiert je nach Algorithmus um bis zu 15% je nach Cache-Zustand und CPU-Auslastung; eine einmalige Kalibrierungsmessung ist für Echtzeit-Anwendungen daher empfehlenswert

12.1.3 Anwendungsbreite

Die vier detaillierten Fallstudien (Kapitel 10) demonstrierten die Vielseitigkeit:

Anwendung	Struktur	Speedup	Impakt
Roboter-MPC	Hierarchisch	$20\times$	Echtzeit-MPC @ 1 kHz
Netzwerkdynamik	Broadcasting	$40+\times$	Monte-Carlo praktikabel
Neuronale Netze	Sparse	$8+\times$	Edge-Device-Inferenz
Finanzrisiko	Sparse	$8\times$	Häufigere Stress-Tests

Gemeinsamer Erfolgsfaktor: Hohe Sparsity ($> 70\%$) kombiniert mit klarer struktureller Asymmetrie.

12.2 Praktischer Implementierungsleitfaden

Basierend auf allen theoretischen und experimentellen Erkenntnissen ergibt sich folgender Leitfaden:

12.2.1 Entscheidungsbaum

Algorithm 12.1 Strategie-Auswahl für Matrix-basierte Systeme

Eingabe: Systemmatrix A , geplante Anzahl Berechnungen N_{ops}

Ausgabe: Empfohlene Implementierungsstrategie

```

1: Messe Systemparameter:  $n$  (Größe),  $m$  (Nicht-Null-Einträge)
2: sparsity  $\leftarrow 1 - m/n^2$ 
3: if  $n < 100$  then
4:   return USE_BLAS ▷ Overhead dominiert
5: end if
6: if sparsity  $< 0.5$  then
7:   if  $N_{\text{ops}} > 100$  and  $n > 500$  then
8:     Berechne Eigenwertzerlegung (einmalig)
9:     return USE_EIGENVALUE_METHOD
10:  else
11:    return USE_DENSE_BLAS
12:  end if
13: end if
14: Analysiere Struktur: type  $\leftarrow$  Strukturerkennung( $A$ )
15: if type = BLOCK_DIAGONAL and block_size  $> 500$  then
16:   return USE_BLOCK_DECOMPOSITION
17: else if type = HIERARCHICAL then
18:   return USE_LAYER_ADAPTIVE
19: else if sparsity  $> 0.7$  then
20:   if erwarte sparse Eingabevektoren then
21:     return USE_COLUMN_ORIENTED_SPARSE
22:   else
23:     return USE_ROW_ORIENTED_SPARSE
24:   end if
25: else
26:   return USE_ADAPTIVE ▷ Wechselt dynamisch
27: end if

```

Satz 73 (Formale Laufzeitanalyse: Algorithmus 12.1). Sei $A \in \mathbb{R}^{n \times n}$ mit $m = \text{nnz}(A)$ Nicht-Null-Einträgen. Dann terminiert Algorithmus 12.1 in Zeit

$$T_{\text{DT}} = O(n + m),$$

unabhängig davon, auf welchem Pfad der Entscheidungsbaum terminiert.

Beweis. Der Algorithmus führt eine sequentielle Folge von Tests aus, von denen jeder frühestmöglich mit **Return** terminiert. Wir analysieren jeden Schritt nach Aufwand und Aufrufhäufigkeit.

Schritt 1: Berechnung von n , $m = \text{nnz}(A)$, $s = m/n^2$ (Zeilen 1–2).

Liegt A als CSR/CSC vor, ist m in $\Theta(1)$ bekannt (Header des Sparse-Formats); n und s ebenfalls. Andernfalls: linear scan $\Theta(n^2)$ für dichte Matrix oder $\Theta(n + m)$ für Sparse-Format. Best-Case: $\Theta(1)$ (Header), Worst-Case: $\Theta(n^2)$. Im relevanten Sparse-Fall: $\Theta(n + m)$.

Schritt 2: Erste Abzweigung – $n < n_0$ (Zeile 3–4).

Vergleich $n < n_0$: $\Theta(1)$. Wenn wahr: sofortige Rückgabe. Laufzeit dieses Pfads: $\Theta(1)$.

Schritt 3: Sparsity-Test $s > s_{\text{threshold}}$ (Zeile 5–6).

$\Theta(1)$. Wenn wahr: Rückgabe ROW_ORIENTED. Laufzeit: $\Theta(1)$ ab hier.

Schritt 4: Bandbreiten-/Diagonal-Test (Zeile 7–8).

Überprüfung, ob A Bandstruktur oder Diagonal hat (Pattern-Scan durch die m Nicht-Null-Einträge um Bandweite zu bestimmen): $\Theta(m)$. Wenn wahr: $\Theta(m)$.

Schritt 5: Strukturanalyse via Algorithmus 7.2 (Zeile 9).

Dieser Aufruf kostet nach Satz 32 $O(n^3)$ im Allgemeinen; für Sparse-Matrizen ist die dominante Operation (Schur-Komplement, BFS/DFS) $O(n+m)$. Symbolischer Ablauf: DFS für Zusammenhangskomponenten ($\Theta(n+m)$), Bandweiten-BFS ($\Theta(n+m)$), Hierarchie-Test ($\Theta(n+m)$). Gesamtkosten dieses Schritts: $\Theta(n+m)$ im Sparse-Fall.

Schritt 6: Verzweigung nach Strukturtyp (Zeilen 10–18).

Jeder **If/ElseIf**-Test: $\Theta(1)$. Maximal $\Theta(1)$ Tests insgesamt (konstante Zahl von Zweigen). Jeder Zweig endet sofort mit **Return**: $\Theta(1)$.

Dominanz.

Unter allen Pfaden gibt es zwei mögliche Ausführungen:

- *Frühzeitiger Exit* (Schritte 2–4): $\Theta(1)$ oder $\Theta(m)$.
- *Vollständige Strukturanalyse* (Schritte 1–6, inkl. Schritt 5): $\Theta(n+m)$.

In jedem Pfad gilt $T_{\text{DT}} = O(n+m)$.

Untere Schranke. Da mindestens m Einträge (Schritt 1) oder $n+m$ Kanten (Schritt 5) besucht werden müssen, gilt $T_{\text{DT}} = \Omega(m)$ im schlimmsten Fall. Damit: $T_{\text{DT}} = \Theta(n+m)$ auf dem vollständigen Pfad.

Korrektheit. Der Entscheidungsbaum realisiert eine Prioritätsordnung: billigste Heuristiken (Größe, Sparsity) werden zuerst ausgewertet; teurere strukturelle Analysen erfolgen nur wenn nötig. Jede Rückgabe entspricht dem syntaktisch korrekten Bezeichner einer Algorithmusvariante, die dann tatsächlich die strukturellen Anforderungen erfüllt (nach Sätzen 3–8).

Hilfsspeicher. Keine zusätzlichen Datenstrukturen außer $\Theta(n+m)$ für den internen Aufruf von Alg. 7.2; danach freigeben. Netto: $O(n+m)$ Hilfsspeicher. $\square$

12.2.2 Implementierungs-Checkliste

Für erfolgreiche Implementierung:

1. **Profilierung:** Messe reale Sparsity-Muster auf Produktionsdaten, nicht synthetischen Beispielen
2. **Schwellwerte:** Kalibriere numerische Schwellwerte ($\tau \approx 10^{-12}$ für double precision)
3. **Speicherformat:** Wähle CSC für spalten-orientiert, CSR für zeilen-orientiert
4. **Cache-Optimierung:** Berücksichtige Cache-Linien (typisch 64 Bytes)
5. **Fallback:** Implementiere dense Fallback für Fälle, wo Sparsity-Annahmen verletzt werden
6. **Validierung:** Vergleiche Ergebnisse mit Referenzimplementierung (Genauigkeit)
7. **Benchmarking:** Messe auf Zielarchitektur, nicht Entwicklungsmaschine

12.3 Forschungsagenda

Die Arbeit öffnet mehrere vielversprechende Forschungsrichtungen:

12.3.1 Kurzfristige Forschungsziele (1-2 Jahre)

1. Hardware-spezifische Optimierung

- GPU-Implementierung spalten-orientierter Algorithmen mit CUDA
- TPU-Mapping für strukturierte Sparsity
- ARM NEON SIMD-Optimierung für Edge-Devices
- Benchmarking über diverse Architekturen

2. Automatische Codegenerierung

- Compiler-Extension für strukturbewusste Optimierung
- JIT-Kompilierung basierend auf Runtime-Sparsity
- Integration in MLIR (Multi-Level Intermediate Representation)

3. Erweiterte Strukturerkennung

- Machine Learning für Struktur-Klassifikation
- Automatische Parameterwahl (τ , Algorithmus) via Bayessche Optimierung
- Online-Lernen: Adaptation an zeitvariante Sparsity

4. Empirische Schärfung der formalen Laufzeitkonstanten

Die in Sätzen 3–73 bewiesenen asymptotischen Schranken lassen verborgene Multiplikationskonstanten c offen. Experiment 8 liefert erste Messungen; für praktische Zeitplanung ist eine systematische Bestimmung von c auf verschiedenen Architekturen (x86-64, ARM, GPU) wünschenswert. Konkrete Ziele:

- Kalibrationsmessungen für alle 13 Algorithmen auf mindestens 3 repräsentativen Hardware-Plattformen
- Modellierung von c in Abhängigkeit von Cache-Größe, SIMD-Breite und Speicherbandbreite
- Einbettung in ein *Roofline-Modell*: Unterscheidung zwischen rechenbeschränkten ($\Theta(n^3)$ -Algorithmen wie Padé) und speicherbeschränkten ($\Theta(n+m)$ -Algorithmen wie SpMV)
- Publikation einer Referenz-Konstantentabelle als Teil eines offenen Benchmarks

12.3.2 Mittelfristige Forschungsziele (2-5 Jahre)

1. Theoretische Vertiefung

- Beweis optimaler Speedup-Faktoren für spezifische Strukturklassen
- Untere Schranken: Kann man besser als $O(m)$ für sparse Matrizen?
- Approximationstheorie: Trade-off zwischen Genauigkeit und Geschwindigkeit

2. Präzision der Laufzeitanalysen: von Θ zu exakten Ausdrücken

Die vorliegenden formalen Beweise (Sätze 3–73) verwenden das RAM-Modell (unit-cost Arithmetik und Speicherzugriff). Mittelfristig sind präzisere Modelle wünschenswert:

- *Cache-oblivious Analyse*: Ersetzung von $\Theta(n+m)$ durch $\Theta((n+m)/B)$ Transfers im External-Memory-Modell (Block-Größe B), besonders relevant für den Sparse Forward-Pass (Satz 13) und den Pruning-Algorithmus (Satz 61)
- *Average-Case-Analyse*: Die Beweise liefern Worst-Case-Garantien. Für zufällige Sparse-Matrizen mit i.i.d. Einträgen sind schärfere Durchschnittslaufzeiten beweisbar – etwa für die Sortierungsphase in Alg. 10.2: worst case $\Theta(N_W \log N_W)$, expected für weitgehend vorsortierte Matrizen: $\Theta(N_W)$
- *Parallele Komplexität*: Erweiterung der Beweise auf PRAM-Modell; insbesondere der SpMV-Algorithmen (3.1, 3.2) und des Layer-Adaptive-Algorithmus (4.1) versprechen lineare Tiefe $O(\log n)$ bei n Prozessoren

3. Nichtlineare Verallgemeinerung

- Asymmetrie in nichtlinearen dynamischen Systemen
- Neuronale ODEs mit adaptiver Sparsity
- Koopman-Operator-Theorie für nichtlineare Systeme

4. Stochastische Systeme

- Kalman-Filterung mit sparse Systemmatrizen
- Particle Filter: Spalten-orientierte Resampling-Strategien
- Uncertainty Quantification via sparse Polynomial Chaos

5. Quantencomputing

- Quantenalgorithmen für sparse Matrix-Operationen
- HHL-Algorithmus (Harrow-Hassidim-Lloyd) mit Strukturausnutzung
- Fehlerkorrektur in Quantensystemen

12.3.3 Langfristige Vision (5+ Jahre)

1. Hardware-Software-Co-Design

- Spezialisierte Prozessoren für spalten-orientierte Operationen
- Neuromorphe Hardware mit inhärenter Sparsity
- Optische Computer: Nutzung von Asymmetrie in photonischen Systemen

2. Foundation Models für Systemdynamik

- Pre-trained Models für Struktur-Prädiktion
- Transfer Learning zwischen verschiedenen Systemklassen
- Few-Shot Learning: Strukturerkennung mit minimalen Daten

3. Autonome Systeme

- Selbst-optimierende Algorithmen mit Online-Strukturlernen
- Adaptive Computing: Hardware rekonfiguriert sich basierend auf Struktur
- Bio-inspirierte Architekturen: Gehirn als Vorbild für asymmetrische Verarbeitung

12.4 Offene theoretische Probleme

Mehrere fundamentale Fragen bleiben unbeantwortet:

Frage 1 (Optimale Schwellwerttheorie). Existiert eine optimale Strategie zur Wahl des Sparsity-Schwellwerts τ , die den Trade-off zwischen Genauigkeit und Geschwindigkeit minimiert?

Formal: Gegeben gewünschte Genauigkeit ϵ und Matrix A , finde:

$$\tau^* = \arg \min_{\tau} T(\tau) \quad \text{s.t.} \quad \text{Error}(\tau) \leq \epsilon$$

Dies ist ein Optimierungsproblem mit unbekannten Zielfunktionen (Laufzeit und Fehler hängen komplex von τ ab).

Frage 2 (Universelle Speedup-Schranken). Existieren universelle obere Schranken für den Speedup spalten-orientierter Methoden?

Vermutung: Für sparse Matrix mit Sparsity p und Eingabevektor-Sparsity p_x gilt:

$$\text{Speedup} \leq \frac{1}{(1-p)(1-p_x)} \cdot C(\text{arch})$$

wobei $C(\text{arch})$ eine architekturspezifische Konstante ist.

Beweis steht aus.

Frage 3 (Fill-in-Vermeidung). Für welche Klassen von sparse Matrizen bleiben die Potenzen A^k sparse?

Bekannt: Tridiagonale, Band-Matrizen haben kontrolliertes Fill-in.

Unbekannt: Gibt es eine allgemeine Charakterisierung? Kann man Fill-in für beliebige Strukturen vorhersagen?

Frage 4 (Nichtlineare Asymmetrie). Lässt sich das Asymmetrie-Konzept auf nichtlineare Systeme verallgemeinern?

Für $\dot{x} = f(x)$ mit Jacobi-Matrix $J(x) = \nabla f(x)$: Die Asymmetrie von $J(x)$ variiert mit x . Wie kann man dies ausnutzen?

Möglicher Ansatz: Lokale Linearisierung + adaptive Algorithmuswahl basierend auf $J(x(t))$.

Frage 5 (Untere Schranken für bewiesene Algorithmen). Die formalen Beweise (Sätze 3–73) liefern obere Schranken. Gelten für diese Algorithmen auch *optimale* untere Schranken, d.h. ist kein asymptotisch schnellerer Algorithmus möglich?

- Für SpMV im algebraischen Berechnungsbaum-Modell: $\Omega(n+m)$ ist bekannt (jeder Nicht-Null-Eintrag muss mindestens einmal gelesen werden). Die bewiesenen Schranken $T = \Theta(n+m)$ für Algorithmen 3.1 und 3.2 sind also *asymptotisch optimal*.

- Für Matrixexponential-Methoden ($\Theta(n^3)$): Die untere Schranke $\Omega(n^3)$ für allgemeine Matrizen ist offen (folgt nicht trivial aus Matrixmultiplikation, da e^A keine bilineare Form ist). Ist $\omega < 3$ (der Matrixmultiplikations-Exponent) hier ausnutzbar?
- Für Asymmetrie-Pruning ($\Theta((1+F)|D|Ln^2)$): Fine-Tuning erfordert Vorwärts- und Rückwärtspässe, deren Komplexität durch $\Omega(Ln^2)$ pro Sample beschränkt ist. Der Fine-Tuning-Term ist somit optimal für dense Netze. Für sparse Netze nach dem Pruning: Kann man $O(Ln^2(1-p_w)(1-p_a))$ statt $O(Ln^2)$ beweisen?

Frage 6 (Kompositionsregeln für Laufzeitschranken). Die bewiesenen Sätze analysieren jeden Algorithmus isoliert. Wie propagieren Laufzeitschranken durch Algorithmus-Pipelines?

Beispiel: Der Entscheidungsbaum (Alg. 12.1, $T_{DT} = O(n+m)$) wählt dann einen spezifischen Algorithmus aus, z.B. den Sparse Forward-Pass (Alg. 5.1, $T_{SP} = \Theta(\sum_l s_l \bar{d}_l)$). Die Gesamtlaufzeit ist $T_{dt} + T_{sp}$. Für welche Strukturklassen gilt $T_{dt} = o(T_{sp})$, d.h. wann ist die Entscheidung kostenlos amortisiert? Formal gesuchte Aussage: $O(n+m) = o(\sum_l s_l \bar{d}_l)$ genau dann, wenn die Durchschnittsdichte $\bar{d}/n = \omega(1/\log n)$.

12.5 Abschließende philosophische Reflexion

Die Asymmetrie zwischen Zeilen und Spalten ist ein bemerkenswertes Beispiel dafür, wie mathematische Struktur physikalische Realität widerspiegelt:

Zeilen als Aggregatoren: In Naturgesetzen, Physik und Biologie ist Aggregation ubiquitär. Elektrisches Potential aggregiert Beiträge aller Ladungen, Populationswachstum aggregiert alle ökologischen Einflüsse, neuronale Aktivierung aggregiert alle synaptischen Inputs. Die Zeilen-Perspektive kodiert diese many-to-oneStruktur.

Spalten als Broadcaster: Gleichzeitig propagieren Einflüsse von wenigen Quellen zu vielen Zielen. Ein Influencer erreicht Millionen, eine Mutation breitet sich in einer Population aus, ein Hub-Neuron aktiviert viele nachgelagerte Neuronen. Die Spalten-Perspektive kodiert diese one-to-manyStruktur.

Diese Dualität ist nicht nur mathematisch elegant, sondern fundamentales Organisationsprinzip komplexer Systeme. Die vorliegende Arbeit hat gezeigt, dass die systematische Ausnutzung dieser Asymmetrie transformative Auswirkungen auf Berechnungseffizienz haben kann – mit Speedups von $40\times$ und mehr in realen Anwendungen.

Blickt man weiter in die Zukunft, so deutet sich ein Paradigmenwechsel an: Von symmetrischer, gleichförmiger Verarbeitung (wie in klassischen Architekturen) hin zu asymmetrischer, strukturbewusster Verarbeitung (wie im Gehirn). Das menschliche Gehirn ist das ultimative asymmetrische System: Wenige Neuronen haben tausende Verbindungen, die meisten nur hunderte. Diese Asymmetrie ist kein Fehler, sondern Feature – sie ermöglicht effiziente Verarbeitung komplexer Aufgaben mit begrenzten Ressourcen.

Die formalen Laufzeitbeweise dieser Arbeit und ihre empirische Validierung in Experiment 8 machen diesen Schritt greifbar: Strukturbewusste Algorithmen lassen sich nicht nur heuristisch entwickeln, sondern mathematisch beweisen und auf neue Hardware portieren. Lediglich eine Neukalibrierung der Konstanten c ist dabei erforderlich. Das ist das epistemologische Versprechen formaler Komplexitätsanalyse.

Die Erkenntnisse dieser Arbeit sind ein kleiner Schritt in Richtung solcher bio-inspirierter, strukturbewusster Berechnungssysteme. Sie zeigen: Asymmetrie ist nicht zu vermeiden – sie ist zu umarmen, zu verstehen und systematisch auszunutzen.

12.6 Schlusswort

Diese Arbeit begann mit der einfachen Beobachtung, dass Zeilen und Spalten einer Matrix unterschiedliche semantische Rollen spielen. Sie endet mit der Erkenntnis, dass diese Asymmetrie ein fundamentales Prinzip ist, das von der Theorie über Algorithmen bis hin zu konkreten industriellen Anwendungen durchgängig relevant ist.

Die erzielten Speedups – bis zu $40\times$ in realen Szenarien – sind nicht nur akademisch interessant. Sie ermöglichen Anwendungen, die zuvor nicht praktikabel waren: Echtzeit-MPC für komplexe Roboter, Monte-Carlo-Simulation epidemiologischer Modelle mit Millionen Szenarien, Edge-Device-Inferenz tiefer neuronaler Netze, und häufige Stress-Tests systemischer Finanzrisiken.

Darüber hinaus liefert die Arbeit mit den 13 formalen Laufzeitanalysen ein Fundament, das über reine Speedup-Zahlen hinausgeht: Es ermöglicht *a priori* Laufzeitvorhersagen, dokumentiert Garantien für Worst- und Best-Case-Szenarien und schafft eine nachprüfbare Basis für zukünftige Erweiterungen und Portierungen. Die enge Übereinstimmung von Theorie und Messung in Experiment 8 (Konstanten-Abweichung $< 15\%$) belegt, dass diese Garantien für die Praxis aussagekräftig sind.

Gleichzeitig zeigt die Arbeit auch die Grenzen: Overhead dominiert bei kleinen Systemen, hochoptimierte Bibliotheken sind schwer zu schlagen, und nicht jede strukturelle Eigenschaft lohnt die Ausnutzung. Diese Erkenntnisse sind genauso wertvoll wie die Erfolge.

Die Zukunft gehört strukturbewussten, adaptiven Systemen, die automatisch erkennen, wie Berechnungen optimal durchzuführen sind. Diese Arbeit liefert theoretische Grundlagen, algorithmische Werkzeuge und praktische Leitlinien für diesen Weg. Sie zeigt: Die Asymmetrie zwischen Zeilen und Spalten ist kein Detail – sie ist Schlüssel zu effizienter Berechnung im 21. Jahrhundert.

Die vorliegende Arbeit hat gezeigt, dass fundamentale mathematische Strukturen – wie die Asymmetrie zwischen Zeilen und Spalten – nicht nur theoretische Eleganz besitzen, sondern praktische Auswirkungen von enormer Tragweite haben. Von der Robotik über Epidemiologie bis zur Künstlichen Intelligenz: Überall, wo Matrizen die Welt beschreiben, lauert Potenzial für dramatische Effizienzgewinne.

Literaturverzeichnis

- [1] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.
- [2] D. Coppersmith und S. Winograd. Matrix multiplication via arithmetic progressions. *Journal of Symbolic Computation*, 9(3):251–280, 1990.
- [3] F. Le Gall. Powers of tensors and fast matrix multiplication. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation (ISSAC)*, Seiten 296–303, 2014.
- [4] V. V. Williams. Multiplying matrices faster than Coppersmith-Winograd. In *Proceedings of the 44th ACM Symposium on Theory of Computing (STOC)*, Seiten 887–898, 2012.
- [5] V. Ya. Pan. How to multiply matrices faster. *Lecture Notes in Mathematics*, Vol. 179, Springer, Berlin, 1984.
- [6] T. A. Davis. *Direct Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2006.
- [7] G. H. Golub und C. F. Van Loan. *Matrix Computations*, 4. Auflage. Johns Hopkins University Press, Baltimore, 2013.
- [8] R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine und H. Van der Vorst. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. SIAM, Philadelphia, 1994.
- [9] Y. Saad. *Iterative Methods for Sparse Linear Systems*, 2. Auflage. SIAM, Philadelphia, 2003.
- [10] S. C. Eisenstat und J. W. H. Liu. Exploiting structural symmetry in a sparse partial factorization scheme. *SIAM Journal on Matrix Analysis and Applications*, 14(1):253–257, 1988.
- [11] C. Moler und C. Van Loan. Nineteen dubious ways to compute the exponential of a matrix, twenty-five years later. *SIAM Review*, 45(1):3–49, 2003.
- [12] N. J. Higham. The scaling and squaring method for the matrix exponential revisited. *SIAM Journal on Matrix Analysis and Applications*, 26(4):1179–1193, 2005.
- [13] N. J. Higham. *Functions of Matrices: Theory and Computation*. SIAM, Philadelphia, 2008.

- [14] M. Al-Mohy und N. J. Higham. A new scaling and squaring algorithm for the matrix exponential. *SIAM Journal on Matrix Analysis and Applications*, 31(3):970–989, 2009.
- [15] R. C. Ward. Numerical computation of the matrix exponential with accuracy estimate. *SIAM Journal on Numerical Analysis*, 14(4):600–610, 1977.
- [16] T. Kailath. *Linear Systems*. Prentice-Hall, Englewood Cliffs, NJ, 1980.
- [17] P. J. Antsaklis und A. N. Michel. *A Linear Systems Primer*. Birkhäuser, Boston, 2006.
- [18] C.-T. Chen. *Linear System Theory and Design*. Oxford University Press, New York, 1984.
- [19] H. K. Khalil. *Nonlinear Systems*, 3. Auflage. Prentice-Hall, Upper Saddle River, NJ, 2002.
- [20] R. A. Horn und C. R. Johnson. *Matrix Analysis*, 2. Auflage. Cambridge University Press, Cambridge, 2013.
- [21] R. Diestel. *Graph Theory*, 5. Auflage. Springer, Berlin, 2017.
- [22] H. S. Warren. *Hacker’s Delight*, 2. Auflage. Addison-Wesley, Boston, 2013.
- [23] T. H. Cormen, C. E. Leiserson, R. L. Rivest und C. Stein. *Introduction to Algorithms*, 3. Auflage. MIT Press, Cambridge, MA, 2009.
- [24] R. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.
- [25] A. B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.
- [26] Y. LeCun, Y. Bengio und G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [27] I. Goodfellow, Y. Bengio und A. Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.
- [28] S. Han, J. Pool, J. Tran und W. J. Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Seiten 1135–1143, 2015.
- [29] J. Frankle und M. Carlin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.
- [30] X. Glorot, A. Bordes und Y. Bengio. Deep sparse rectifier neural networks. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, Seiten 315–323, 2011.
- [31] J. J. Dongarra, J. Du Croz, S. Hammarling und I. Duff. A set of level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software*, 16(1):1–17, 1990.

- [32] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney und D. Sorensen. *LAPACK Users' Guide*, 3. Auflage. SIAM, Philadelphia, 1999.
- [33] S. van der Walt, S. C. Colbert und G. Varoquaux. The NumPy array: A structure for efficient numerical computation. *Computing in Science & Engineering*, 13(2):22–30, 2011.
- [34] P. Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.
- [35] Intel Corporation. *Intel Math Kernel Library Reference Manual*. Intel Press, Santa Clara, CA, 2020.
- [36] J. B. Rawlings, D. Q. Mayne und M. M. Diehl. *Model Predictive Control: Theory, Computation, and Design*, 2. Auflage. Nob Hill Publishing, Madison, WI, 2017.
- [37] D. Q. Mayne, J. B. Rawlings, C. V. Rao und P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.
- [38] S. J. Qin und T. A. Badgwell. A survey of industrial model predictive control technology. *Control Engineering Practice*, 11(7):733–764, 2003.
- [39] R. P. Brent. Algorithms for matrix multiplication. Technical Report CS-TR-70-157, Stanford University, 1970.
- [40] O. H. Ibarra und S. Moran. Probabilistic algorithms for deciding equivalence of straight-line programs. *Journal of the ACM*, 30(1):217–228, 1983.
- [41] I. S. Duff, A. M. Erisman und J. K. Reid. *Direct Methods for Sparse Matrices*. Oxford University Press, Oxford, 1986.
- [42] A. George und J. W. H. Liu. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice-Hall, Englewood Cliffs, NJ, 1981.
- [43] S. V. Parter. The use of linear graphs in Gauss elimination. *SIAM Review*, 3(2):119–130, 1961.
- [44] NVIDIA Corporation. *CUDA C++ Programming Guide*, Version 12.0. NVIDIA Corporation, Santa Clara, CA, 2023.
- [45] D. B. Kirk und W. W. Hwu. *Programming Massively Parallel Processors*, 3. Auflage. Morgan Kaufmann, Burlington, MA, 2016.
- [46] N. Bell und M. Garland. Implementing sparse matrix-vector multiplication on throughput-oriented processors. In *Proceedings of the 2009 ACM/IEEE Conference on Supercomputing (SC09)*, Artikel 18, 2009.
- [47] A. Varga. *Matrix Analysis for Scientists and Engineers*. SIAM, Philadelphia, 2004.
- [48] G. Strang. *Introduction to Linear Algebra*, 5. Auflage. Wellesley-Cambridge Press, Wellesley, MA, 2016.
- [49] L. N. Trefethen und D. Bau III. *Numerical Linear Algebra*. SIAM, Philadelphia, 1997.

- [50] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2. Auflage. SIAM, Philadelphia, 2002.
- [51] J. W. Demmel. *Applied Numerical Linear Algebra*. SIAM, Philadelphia, 1997.
- [52] D. S. Watkins. *Fundamentals of Matrix Computations*, 3. Auflage. Wiley, Hoboken, NJ, 2010.
- [53] B. D. O. Anderson und J. B. Moore. *Optimal Control: Linear Quadratic Methods*. Dover Publications, Mineola, NY, 2007.
- [54] M. Mézard und A. Montanari. *Information, Physics, and Computation*. Oxford University Press, Oxford, 2009.